



Phase 02 | HTML 完整課程 + 第 1~100 題

前端網頁技術與 AI 應用甄試 | HTML 100 題訓練教材

本階段目標：

從 HTML 零基礎開始，建立「看得懂 HTML → 理解語意 → 能判斷結構 → 能解甄試題 → 能看懂 AI 產生的 HTML」的完整能力。

Part 1 | HTML 課程

Lesson 01 | HTML 是什麼？

HTML 全名：

HyperText Markup Language / 超文字標記語言

HTML 的主要功能是：

描述網頁的內容與結構。

例如：

```
<h1>公司網站</h1>
<p>這是一段介紹文字。</p>
```

可以把網站想成一棟房子：

網頁技術

房子比喻

HTML

骨架

CSS

外觀、裝潢

JavaScript

動作、功能

必考觀念

HTML 不是程式語言，而是標記語言。

Lesson 02 | HTML 文件基本結構

最基本的 HTML5：

```
<!DOCTYPE html>
<html lang="zh-Hant">
<head>
  <meta charset="UTF-8">
  <title>我的網站</title>
</head>
<body>
  <h1>Hello World</h1>
</body>
</html>
```

結構：

```
DOCTYPE
↓
html
├── head
│   ├── meta
│   └── title
└── body
    └── 網頁內容
```

必記

`<head>`：

放置網頁的設定與資訊。

`<body>`：

放置使用者看到的主要網頁內容。

Lesson 03 | DOCTYPE



```
<!DOCTYPE html>
```

表示：

此文件採用 *HTML5* 文件類型。

注意：

`DOCTYPE` 不是 **HTML** 元素。

它是文件型別宣告。

Lesson 04 | `<html>`

HTML 文件最外層元素：

```
<html lang="zh-Hant">
```

`lang` 用來描述：

網頁主要使用的語言。

例如：

```
<html lang="en">
```

代表英文。

```
<html lang="zh-Hant">
```

代表繁體中文。

Lesson 05 | `<head>`

常見內容：

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
    content="width=device-width, initial-scale=1.0">
```

```
<title>網站標題</title>
</head>
```

常見元素：

- `<title>`
- `<meta>`
- `<link>`
- `<style>`
- `<script>`

Lesson 06 | `<body>`

使用者實際看到的網頁內容主要放在：

```
<body>
    ...
</body>
```

例如：

```
<body>
    <h1>網頁設計公司</h1>
    <p>提供網站設計服務。</p>
</body>
```

Lesson 07 | Heading 標題

HTML 有：

```
<h1>
<h2>
<h3>
<h4>
<h5>
<h6>
```

`h1` 是最高層級標題。

例如：

```
<h1>公司簡介</h1>
<h2>企業願景</h2>
<h2>公司沿革</h2>
<h3>2026 年</h3>
```

SEO 注意

標題不是單純拿來調整字體大小。

應該用來建立：

文件的階層結構。

Lesson 08 | 段落、區塊與行內容器

<p>

表示段落：

```
<p>這是一個段落。</p>
```

<div>

一般區塊容器：

```
<div class="container">
  ...
</div>
```


一般行內容器：

```
<p>這是 <span>重要文字</span>。</p>
```

Lesson 09 | `<strong>` 與 `<b>`

表示：

內容具有重要性。

```
<strong>重要公告</strong>
```

```
<b>
```

主要表示：

視覺上的粗體效果。

因此：

strong 偏向語意，***b*** 偏向呈現。

Lesson 10 | `<em>` 與 `<i>`

```
<em>
```

表示強調：

```
<em>非常重要</em>
```

```
<i>
```

表示不同語氣、術語或視覺上的斜體等用途。

同樣要記：

`<em>` 有語意；`<i>` 不單純等於「強調」。

Lesson 11 | 超連結 `<a>`

基本語法：

```
<a href="https://example.com">  
  官方網站
```

```
</a>
```

href :

Hypertext Reference

指定連結目的地。

新分頁常見：

```
<a href="https://example.com"
  target="_blank">
  開啟網站
</a>
```

實務上使用 `_blank` 時，也應考慮適當的安全設定。

Lesson 12 | 圖片 `<img>`

```

```

src :

圖片來源。

alt :

圖片替代文字。

非常重要

alt 同時涉及：

- HTML
- Accessibility
- SEO
- Screen Reader

Lesson 13 | alt vs title

不要混淆。

alt

主要提供：

圖片無法顯示時的替代資訊及無障礙用途。

title

可以提供：

額外的輔助提示資訊。

不能把 title 當成 alt 的替代品。

Lesson 14 | 圖片與圖說

HTML5：

```
<figure>
  
  <figcaption>
    公司生產線
  </figcaption>
</figure>
```

figure：

圖片、圖表、程式碼等獨立內容。

figcaption：

圖說。

Lesson 15 | 清單

無序清單

```
<ul>
  <li>HTML</li>
  <li>CSS</li>
  <li>JavaScript</li>
</ul>
```

有序清單

```
<ol>
  <li>註冊</li>
  <li>登入</li>
  <li>購買</li>
</ol>
```

核心：

標籤	意義
	無序清單
	有序清單
	清單項目

Lesson 16 | 表格

基本：

```
<table>
  <tr>
    <th>姓名</th>
    <th>年齡</th>
  </tr>
  <tr>
    <td>王小明</td>
    <td>30</td>
  </tr>
</table>
```

```
</tr>
</table>
```

標籤	意義
<table>	表格
<tr>	表格列
<th>	標題儲存格
<td>	資料儲存格

Lesson 17 | 表單 Form

基本結構：

```
<form action="/submit" method="post">
  <label for="name">姓名</label>
  <input type="text" id="name" name="name">

  <button type="submit">送出</button>
</form>
```

必考

form：

收集使用者輸入資料。

Lesson 18 | Input

常見：

```
<input type="text">
<input type="email">
<input type="password">
<input type="number">
<input type="date">
```

```
<input type="checkbox">
<input type="radio">
<input type="file">
<input type="submit">
```

Lesson 19 | Label

推薦：

```
<label for="email">Email</label>
<input type="email" id="email">
```

for 的值應該對應：

```
label for
  ↓
input id
```

這對：

Accessibility

非常重要。

Lesson 20 | Select、Textarea、Button

Select

```
<select name="city">
  <option value="taipei">台北</option>
  <option value="newtaipei">新北</option>
</select>
```

Textarea

```
<textarea name="message"></textarea>
```

Button

```
<button type="submit">送出</button>
```

Lesson 21 | HTML5 表單驗證

常見屬性：

```
<input required>
```

表示：

必填。

```
<input type="email">
```

要求：

符合 email 類型格式。

```
<input minlength="8">
```

要求：

最少 8 個字元。

Lesson 22 | placeholder

```
<input  
  type="text"  
  placeholder="請輸入姓名">
```

placeholder：

提供輸入欄位的提示文字。

陷阱

placeholder 不是 label 的替代品。

Lesson 23 | GET vs POST

表單：

```
<form method="get">
```

或：

```
<form method="post">
```

GET

常用於：

取得資料。

參數通常可能出現在 URL。

POST

常用於：

提交資料。

例如：

- 登入
- 註冊
- 表單提交

注意

POST 不等於自動加密。

是否加密主要取決於：

Lesson 24 | Semantic HTML

HTML5 提供許多語意元素：

```
<header>
<nav>
<main>
<section>
<article>
<aside>
<footer>
```

這些元素不是單純「比較漂亮的 div」。

它們可以幫助：

- 瀏覽器
- 搜尋引擎
- 輔助科技
- Screen Reader
- 開發人員

理解網頁結構。

Lesson 25 | Semantic HTML 網頁架構

典型網站：

```
<header>
  Logo
  Navigation
</header>

<main>

  <section>
    Hero
```

```
</section>

<section>
  Services
</section>

<article>
  News
</article>

<aside>
  Related Content
</aside>

</main>

<footer>
  Copyright
</footer>
```

Lesson 26 | `section` vs `div`

section

表示：

一個具有主題性的內容區段。

div

表示：

一般區塊容器，沒有特殊語意。

因此：

有明確內容主題 → ***section***

只是排版／分組 → ***div***

Lesson 27 | `article`

`article` 適合：

可以獨立存在、獨立理解的內容。

例如：

- 新聞文章
- 部落格文章
- 商品評論
- 論壇文章

Lesson 28 | `nav`

用於：

主要導覽連結區。

例如：

```
<nav>
  <a href="/">首頁</a>
  <a href="/about">關於我們</a>
  <a href="/contact">聯絡我們</a>
</nav>
```

Lesson 29 | Meta

常見：

```
<meta charset="UTF-8">
```

以及：

```
<meta name="viewport"
```

```
content="width=device-width, initial-scale=1.0">
```

Viewport 對：

RWD

非常重要。

Lesson 30 | SEO 基礎

常見 SEO 元素：

```
<title>網頁設計公司 | 專業網站製作</title>
<meta name="description"
      content="提供企業網站設計、WordPress 與 SEO 服務。">
```

另外還包括：

- Heading
- Semantic HTML
- URL
- Internal Links
- Image alt
- Structured Data

Lesson 31 | HTML Entity

HTML 中某些特殊符號需要使用 Entity。

例如：

```
&copy;
```

代表：

©

```
&nbsp;
```

代表：

不換行空白。

常見：

Entity	結果
<	<
>	>
&	&
"	"
©	©

Lesson 32 | HTML 驗證

HTML 撰寫完成後可以進行：

Validation

目的：

- 找出語法問題
- 找出結構問題
- 提升標準符合度
- 協助除錯

Lesson 33 | HTML 與 Accessibility

重要觀念：

優先使用正確的原生 **HTML** 語意元素。

例如：

不應該只是：

```
<div onclick="submitForm()">送出</div>
```

如果功能本質是按鈕，優先：

```
<button type="submit">  
    送出  
</button>
```

因為原生 HTML 元素通常已具備較完整的：

- Keyboard 支援
- Accessibility 語意
- Browser 行為

Lesson 34 | ARIA

ARIA：

Accessible Rich Internet Applications

主要用來補充：

HTML 原生語意不足的情況。

重要原則：

ARIA 不是用來取代正確 HTML 的第一選擇。

Lesson 35 | AI 產生 HTML 的正確工作流程

AI 可以協助：

- 建立 HTML 架構
- 產生表單
- 建立 Semantic HTML

- 產生 SEO Meta
- 協助除錯

但是：

```
graph TD; A[AI 產生] --> B[人工閱讀]; B --> C[HTML 驗證]; C --> D[Accessibility 檢查]; D --> E[SEO 檢查]; E --> F[瀏覽器測試]; F --> G[修改]; G --> H[上線];
```

這也是本甄試「AI 應用」的重要核心。

Part 2 | HTML 100 題

以下開始正式題庫。

第 1~20 題 | HTML 基礎

第 1 題 ★

HTML 的全名為何？

- A. HyperText Markup Language
- B. HighText Machine Language
- C. Hyper Tool Markup Language
- D. Home Tool Markup Language

答案：A

解析： HTML 是 HyperText Markup Language，中文通常稱為「超文字標記語言」。

考點：HTML 基礎。

第 2 題 ★

HTML 主要用途為何？

- A. 管理資料庫
- B. 描述網頁結構與內容
- C. 製作作業系統
- D. 進行 CPU 運算

答案：B

第 3 題 ★

下列何者最適合描述 HTML？

- A. 程式語言
- B. 標記語言
- C. 資料庫語言
- D. 作業系統

答案：B

第 4 題 ★

HTML5 文件通常使用哪一個 DOCTYPE？

- A. `<!DOCTYPE HTML5>`
- B. `<!HTML5>`
- C. `<!DOCTYPE html>`
- D. `<DOCTYPE html5>`

答案：C

陷阱：不需要寫成 `DOCTYPE HTML5`。

第 5 題 ★

`<!DOCTYPE html>` 的主要作用為何？

- A. 建立標題
- B. 宣告文件類型

- C. 建立網頁內容
- D. 載入 CSS

答案：B

第 6 題 ★

下列哪一個元素是 HTML 文件的最外層元素？

- A. `<body>`
- B. `<head>`
- C. `<html>`
- D. `<main>`

答案：C

第 7 題 ★

主要放置網頁設定資訊的元素是？

- A. `<body>`
- B. `<head>`
- C. `<main>`
- D. `<footer>`

答案：B

第 8 題 ★

使用者主要看到的網頁內容通常放在哪個元素？

- A. `<head>`
- B. `<meta>`
- C. `<body>`
- D. `<title>`

答案：C

第 9 題 ★

設定 HTML 文件字元編碼最常使用：

- A. `<charset>`
- B. `<meta charset="UTF-8">`
- C. `<encoding>`
- D. `<text charset>`

答案：B

第 10 題 ★★

下列哪一段最適合作為 HTML5 基本文件？

A.

```
<html>
<body>
Hello
</body>
</html>
```

B.

```
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Test</title>
</head>
<body>
Hello
</body>
</html>
```

C.

```
<!HTML5>
<body>Hello</body>
```

D.

```
<DOCTYPE>
```

<html5>

答案：B

第 11 題 ★

HTML 中表示第一層標題的是：

- A. <head>
- B. <h1>
- C. <title>
- D. <header>

答案：B

第 12 題 ★

下列何者不是 HTML 標題元素？

- A. <h1>
- B. <h3>
- C. <h6>
- D. <h7>

答案：D

第 13 題 ★★

HTML 標題通常應該主要用來：

- A. 控制字體顏色
- B. 建立內容階層
- C. 製作圖片
- D. 建立資料庫

答案：B

第 14 題 ★

哪個元素表示段落？

- A. `<p>`
- B. `<para>`
- C. `<text>`
- D. `<paragraph>`

答案：A

第 15 題 ★

`<div>` 最主要用途是：

- A. 一般區塊容器
- B. 建立圖片
- C. 建立超連結
- D. 建立表單資料庫

答案：A

第 16 題 ★

`<span>` 最常作為：

- A. 一般行內容器
- B. 網頁標題
- C. 表格
- D. 圖片容器

答案：A

第 17 題 ★★

下列何者最能表達「這段文字具有重要性」？

- A. `<b>`
- B. `<strong>`
- C. `<big>`
- D. `<bold>`

答案：B

第 18 題 ★★

`<b>` 和 `<strong>` 最大的語意差異為：

- A. b 是圖片，strong 是文字
- B. strong 具有重要性的語意
- C. b 不能顯示
- D. strong 只能用於標題

答案：B

第 19 題 ★★

下列何者較能表達「強調」的語意？

- A. `<em>`
- B. `<italic>`
- C. `<i>`
- D. `<strong>`

答案：A

第 20 題 ★★

下列何者主要是一般視覺／不同語氣用途，而非專門表示「強調」？

- A. `<em>`
- B. `<i>`
- C. `<strong>`
- D. `<mark>`

答案：B

第 21~40 題 | 連結、圖片、清單

第 21 題 ★

建立超連結最主要使用：

- A. `<link>`
- B. `<a>`
- C. `<url>`
- D. `<href>`

答案：B

第 22 題 ★

下列哪個屬性指定 `<a>` 的連結目的地？

- A. `src`
- B. `link`
- C. `href`
- D. `url`

答案：C

第 23 題 ★

下列何者正確？

- A. `<a src="page.html">首頁</a>`
- B. `<a href="page.html">首頁</a>`
- C. `<a url="page.html">首頁</a>`
- D. `<a link="page.html">首頁</a>`

答案：B

第 24 題 ★★

若希望連結在新瀏覽器分頁開啟，常見設定為：

- A. `target="_blank"`
- B. `new="tab"`
- C. `window="new"`
- D. `open="blank"`

答案：A

第 25 題 ★

插入圖片主要使用：

- A. `<picture>`
- B. `<image>`

- C. `<img>`
- D. `<photo>`

答案：C

第 26 題 ★

`<img>` 的 `src` 主要代表：

- A. 替代文字
- B. 圖片來源
- C. 圖片標題
- D. 圖片大小

答案：B

第 27 題 ★

下列何者最適合提供圖片替代文字？

- A. `title`
- B. `description`
- C. `alt`
- D. `text`

答案：C

第 28 題 ★★

圖片：

```

```

`alt` 最主要的用途為：

- A. 設定圖片寬度
- B. 設定圖片高度
- C. 提供圖片替代資訊
- D. 設定圖片透明度

答案：C

第 29 題 ★★

若圖片只是純裝飾且沒有資訊意義，通常適合：

- A. 使用假的 alt
- B. 使用 `alt=""`
- C. 完全不能使用圖片
- D. 用 title 取代 alt

答案：B

第 30 題 ★★

下列何者正確？

- A. title 可以完全取代 alt
- B. alt 主要是圖片替代資訊
- C. alt 是設定圖片大小
- D. alt 是設定圖片透明度

答案：B

第 31 題 ★

建立無序清單使用：

- A. `<ol>`
- B. `<ul>`
- C. `<list>`
- D. `<dl>`

答案：B

第 32 題 ★

建立有序清單使用：

- A. `<ol>`
- B. `<ul>`
- C. `<order>`
- D. `<list>`

答案：A

第 33 題 ★

清單中的項目通常使用：

- A. `<item>`
- B. `<li>`
- C. `<list-item>`
- D. `<element>`

答案：B

第 34 題 ★★

下列何者正確？

```
<ul>
  <li>HTML</li>
  <li>CSS</li>
</ul>
```

- A. 建立有序清單
- B. 建立無序清單
- C. 建立表格
- D. 建立導覽列

答案：B

第 35 題 ★★

下列何者適合表示「第一步、第二步、第三步」？

- A. `<ul>`
- B. `<ol>`
- C. `<div>`
- D. `<span>`

答案：B

第 36 題 ★★

`<figure>` 最適合用於：

- A. 程式執行
- B. 可獨立呈現的圖片、圖表等內容
- C. 建立資料庫
- D. 建立 CSS

答案：B

第 37 題 ★

`<figcaption>` 主要用於：

- A. 圖片壓縮
- B. 圖片說明／圖說
- C. 圖片連結
- D. 圖片尺寸

答案：B

第 38 題 ★★

下列哪個結構最適合？

- A. `<figure><img ...><figcaption>產品照片</figcaption></figure>`
- B. `<img><caption>`
- C. `<picture><description>`
- D. `<image><text>`

答案：A

第 39 題 ★★

HTML 圖片的替代文字對哪一項特別重要？

- A. 無障礙
- B. CPU 超頻
- C. 硬碟格式化
- D. 作業系統安裝

答案：A

第 40 題 ★★★

某公司使用 AI 自動產生大量 `<img>` 元素，但所有圖片都寫：

```
alt="image"
```

最合理的評價是：

- A. 完全正確
- B. alt 應該描述圖片實際內容
- C. alt 一律不能使用英文
- D. alt 是圖片檔名

答案：B

AI 情境考點： AI 產生的 HTML 仍需人工檢查。

第 41~60 題 | 表格與表單

第 41 題 ★

建立表格使用：

- A. `<grid>`
- B. `<table>`
- C. `<tab>`
- D. `<data>`

答案：B

第 42 題 ★

表格中的一列使用：

- A. `<tr>`
- B. `<row>`
- C. `<td>`
- D. `<line>`

答案：A

第 43 題 ★

表格標題儲存格使用：

- A. `<thead>`
- B. `<th>`
- C. `<title>`
- D. `<header>`

答案：B

第 44 題 ★

表格一般資料儲存格使用：

- A. `<data>`
- B. `<cell>`
- C. `<td>`
- D. `<tr>`

答案：C

第 45 題 ★★

`<th>` 與 `<td>` 的主要差別是：

- A. th 是標題儲存格，td 是一般資料儲存格
- B. th 是圖片，td 是文字
- C. th 是表單，td 是連結
- D. 沒有任何差別

答案：A

第 46 題 ★

建立表單使用：

- A. `<input>`
- B. `<form>`
- C. `<field>`
- D. `<dataform>`

答案：B

第 47 題 ★

`form` 的 `action` 通常用來指定：

- A. 表單資料送往何處
- B. 字體大小
- C. 表單顏色
- D. 表單寬度

答案：A

第 48 題 ★★

`form` 的 `method` 主要指定：

- A. CSS 方法
- B. HTTP 資料提交方式
- C. 字型
- D. 表單大小

答案：B

第 49 題 ★★

下列何者是常見的 HTTP 表單 method？

- A. GET
- B. SEND
- C. TRANSFER
- D. PUSHHTML

答案：A

第 50 題 ★★

下列哪個通常適合用於提交不希望直接出現在 URL 查詢字串中的表單資料？

- A. GET
- B. POST
- C. LINK
- D. DISPLAY

答案：B

注意：POST 本身並不代表加密；敏感資料仍應透過 HTTPS 保護。

第 51 題 ★

文字輸入欄位：

```
<input type="text">
```

`type="text"` 表示：

- A. 文字輸入
- B. 密碼輸入
- C. 圖片輸入
- D. 日期輸入

答案：A

第 52 題 ★

密碼欄位常用：

```
<input type="password">
```

答案：

A

第 53 題 ★

Email 欄位可以使用：

```
<input type="email">
```

此設定可以讓瀏覽器進行基本的：

- A. 格式驗證
- B. 圖片壓縮
- C. SEO 分析
- D. DNS 查詢

答案：A

第 54 題 ★

`required` 的用途是：

- A. 欄位必須填寫
- B. 欄位隱藏
- C. 欄位加密
- D. 欄位刪除

答案：A

第 55 題 ★★

下列何者可以讓欄位成為必填？

- A. `important`
- B. `required`
- C. `must`
- D. `validate`

答案：B

第 56 題 ★

`placeholder` 的主要作用是：

- A. 提供輸入提示
- B. 儲存密碼
- C. 取代資料庫
- D. 建立 DNS

答案：A

第 57 題 ★★

下列關於 `placeholder` 的說法何者正確？

- A. 它可以完全取代 label
- B. 它主要是輸入提示文字
- C. 它會自動儲存資料
- D. 它會加密表單

答案：B

第 58 題 ★★

下列哪一個最適合建立多行文字輸入區？

- A. `<input>`
- B. `<textarea>`
- C. `<text>`
- D. `<multiline>`

答案：B

第 59 題 ★★

下拉選單通常使用：

- A. `<select>` + `<option>`
- B. `<dropdown>`
- C. `<menu-select>`
- D. `<choice>`

答案：A

第 60 題 ★★

下列哪一種元素最適合建立送出按鈕？

- A. `<submit>`
- B. `<button type="submit">`
- C. `<send>`
- D. `<input type="button-submit">`

答案：B

第 61~80 題 | Semantic HTML、Meta、SEO

第 61 題 ★

HTML5 引入許多語意化元素，下列何者屬於 Semantic HTML？

- A. `<header>`
- B. `<font>`

C. `<center>`

D. `<blink>`

答案：A

第 62 題 ★

`<header>` 通常表示：

A. 網頁或區域的導入／標頭內容

B. 網頁資料庫

C. CSS

D. JavaScript

答案：A

第 63 題 ★

`<footer>` 通常表示：

A. 頁尾內容

B. 頁首內容

C. 圖片

D. 表單

答案：A

第 64 題 ★

`<nav>` 主要用於：

A. 主要導覽區域

B. 圖片

C. 資料庫

D. JavaScript

答案：A

第 65 題 ★★

`<main>` 的主要用途：

- A. 表示網頁主要內容
- B. 表示瀏覽器設定
- C. 表示圖片
- D. 表示 CSS

答案：A

第 66 題 ★★

`<article>` 最適合表示：

- A. 可獨立理解的內容
- B. 純排版容器
- C. CSS 樣式
- D. JavaScript 函式

答案：A

第 67 題 ★★

`<aside>` 常用於：

- A. 主要內容旁的相關／補充內容
- B. 頁面所有內容
- C. 圖片檔案
- D. CSS 檔案

答案：A

第 68 題 ★★

`<section>` 與 `<div>` 的主要差別：

- A. section 具有內容區段的語意
- B. div 一定不能包含文字
- C. section 一定是圖片
- D. 兩者完全相同

答案：A

第 69 題 ★★★

一個企業網站首頁有「服務介紹」這個具有明確主題的內容區塊，較適合：

- A. `<section>`
- B. `<random>`
- C. `<span>`
- D. `<br>`

答案：A

第 70 題 ★★★

如果某元素只是為了 CSS 排版而需要一個容器，通常較適合：

- A. `<article>`
- B. `<div>`
- C. `<nav>`
- D. `<main>`

答案：B

第 71 題 ★

Viewport 設定通常使用：

- A. `<meta name="viewport">`
- B. `<viewport>`
- C. `<screen>`
- D. `<meta screen>`

答案：A

第 72 題 ★★

下列哪個設定常見於 RWD 網站？

```
<meta name="viewport"
      content="width=device-width, initial-scale=1.0">
```

- A. 正確
- B. 錯誤，HTML5 不支援

- C. 只能用在桌機
- D. 是 JavaScript

答案：A

第 73 題 ★

`<title>` 主要設定：

- A. 網頁文件標題
- B. 圖片尺寸
- C. CSS 顏色
- D. DNS 名稱

答案：A

第 74 題 ★★

`<title>` 對 SEO 的重要性：

- A. 通常重要
- B. 完全沒有任何關係
- C. 只控制圖片
- D. 只控制背景色

答案：A

第 75 題 ★★

Meta Description 主要用途是：

- A. 提供頁面描述資訊
- B. 設定圖片寬度
- C. 建立 JavaScript
- D. 設定 CPU

答案：A

第 76 題 ★★★

下列哪一個較符合 SEO 友善的標題結構？

- A. 所有文字都用 `<div>`
- B. 使用有階層意義的 `<h1>` ~ `<h6>`
- C. 全部使用 `<h1>`
- D. 完全不使用標題

答案：B

第 77 題 ★★★

下列何者同時可能涉及 HTML 語意、SEO 與 Accessibility？

- A. Semantic HTML
- B. CPU Clock
- C. RAM
- D. SSD

答案：A

第 78 題 ★★★

對搜尋引擎而言，Semantic HTML 的主要價值之一是：

- A. 幫助理解內容結構
- B. 增加硬碟容量
- C. 提高 CPU 頻率
- D. 自動保證排名第一

答案：A

第 79 題 ★★★★★

下列何者是較合理的 SEO 觀念？

- A. 只要塞入大量關鍵字就一定排名第一
- B. 語意清楚、對使用者有價值的內容與良好結構較重要
- C. `<h1>` 越多越好
- D. alt 應該全部塞滿關鍵字

答案：B

第 80 題 ★★★★★

使用 AI 產生網頁 SEO Meta Description 後，最適當的做法是：

- A. 不檢查直接發布
- B. 檢查內容是否準確、符合頁面內容與使用者需求
- C. 每頁都使用完全相同文字
- D. 塞入所有可能的關鍵字

答案：B

第 81~100 題 | HTML 陷阱、Accessibility、AI 情境

第 81 題 ★★★

`id` 與 `class` 的主要差異為：

- A. `id` 通常用來識別特定唯一元素，`class` 可套用於多個元素
- B. `class` 一定唯一
- C. `id` 只能用於圖片
- D. 兩者完全相同

答案：A

第 82 題 ★★★

下列何者較合理？

```
<div id="header"></div>
<div class="card"></div>
<div class="card"></div>
```

- A. 正確
- B. `class` 不可以重複
- C. `id` 應該全部重複
- D. HTML 不允許 `class`

答案：A

第 83 題 ★★★

若 JavaScript 要取得：

```
<div id="menu"></div>
```

可以使用：

```
document.getElementById("menu")
```

答案：

正確。

第 84 題 ★★★

下列哪一個不是 `id` 與 `class` 的正確觀念？

- A. id 通常應具有唯一性
- B. class 可以套用到多個元素
- C. 一個元素可以同時具有 class 與 id
- D. class 必須全網站唯一

答案：D

第 85 題 ★★★

下列哪一個最符合 Accessibility？

- A. 使用 `<div>` 偽裝所有按鈕
- B. 優先使用原生 `<button>`
- C. 所有文字轉成圖片
- D. 移除鍵盤操作

答案：B

第 86 題 ★★★

以下：

```
<label for="email">Email</label>
<input id="email" type="email">
```

`for="email"` 主要與哪個屬性建立關聯？

- A. `class`
- B. `name`
- C. `id`
- D. `src`

答案：C

第 87 題 ★★★★★

為什麼正確使用 `<label>` 很重要？

- A. 增加 CPU 速度
- B. 改善表單的可理解性與無障礙操作
- C. 自動加密密碼
- D. 自動產生 CSS

答案：B

第 88 題 ★★★

ARIA 的主要用途是：

- A. 補充 Web Accessibility 語意
- B. 取代所有 HTML
- C. 建立資料庫
- D. 加速 CPU

答案：A

第 89 題 ★★★★★

下列關於 ARIA 的說法何者較正確？

- A. ARIA 永遠比原生 HTML 好
- B. 應優先使用適當的原生 HTML，必要時再使用 ARIA
- C. ARIA 可以取代所有表單元素
- D. ARIA 只用於 SEO

答案：B

第 90 題 ★★★

下列何者最符合圖片 Accessibility？

- A. ``
- B. ``
- C. ``
- D. `<image alt="factory">`

答案：B

第 91 題 ★★★

HTML Entity：

`&`

代表：

- A. `@`
- B. `&`
- C. `<`
- D. `>`

答案：B

第 92 題 ★★★

HTML Entity：

`<`

代表：

- A. `<`
- B. `>`
- C. `&`
- D. `"`

答案：A

第 93 題 ★★★

HTML Entity :

```
&gt;
```

代表：

A. <

B. >

C. &

D. #

答案：B

第 94 題 ★★★

HTML Entity :

```
&nbsp;
```

通常表示：

A. 不換行空白

B. 換行

C. Tab

D. Enter

答案：A

第 95 題 ★★★★★

以下 HTML：

```
<form>
  <input type="text" placeholder="請輸入姓名">
</form>
```

若希望提高表單無障礙性，較適合增加：

A. <label>

B.

- C. `<blink>`
- D. `<center>`

答案：A

第 96 題 ★★★★★

一個 AI 產生的 HTML 使用大量：

```
<div>
  <div>
    <div>
      ...
    </div>
  </div>
</div>
```

而實際內容是頁首、導覽、文章與頁尾。較好的改善方式是：

- A. 增加更多 div
- B. 適當使用 Semantic HTML
- C. 全部改成圖片
- D. 移除 HTML

答案：B

第 97 題 ★★★★★

AI 產生以下程式：

```
<div onclick="submitForm()">送出</div>
```

如果這個元素實際上是一個按鈕，較適合改成：

- A. `<button type="submit">送出</button>`
- B. `<image>送出</image>`
- C. `<span>送出</span>`
- D. `<text>送出</text>`

答案：A

第 98 題 ★★★★★

使用 ChatGPT 產生 HTML 後，下列哪個做法最專業？

- A. AI 產生什麼就直接上線
- B. 檢查語意、Accessibility、SEO、結構與瀏覽器相容性
- C. 只確認畫面有出現
- D. 只確認程式碼很多

答案：B

第 99 題 ★★★★★☆

某網站表單要求使用者輸入密碼。下列觀念何者正確？

- A. `<input type="password">` 本身就代表資料傳輸一定加密
- B. 密碼欄位應搭配適當的安全傳輸，例如 HTTPS
- C. POST 一定代表加密
- D. placeholder 可以保護密碼

答案：B

陷阱解析：

`type="password"` 主要控制輸入顯示方式；POST 也不等於加密。真正的傳輸安全需要適當使用 HTTPS/TLS。

第 100 題 ★★★★★ | HTML 綜合挑戰題

某公司要求 AI 協助建立一個企業網站首頁。AI 產生：

```
<div id="header">
  <div onclick="goHome()">公司 Logo</div>
</div>

<div id="menu">
  <div>首頁</div>
  <div>服務</div>
  <div>聯絡我們</div>
</div>

<div id="content">
  <div>公司介紹</div>
```

```
</div>



<form>
  <input type="text" placeholder="姓名">
  <input type="submit" value="送出">
</form>
```

如果希望改善 **Semantic HTML**、**Accessibility**、**SEO** 與基本使用體驗，下列哪一組改善最合理？

- A. 全部保留 div，因為 div 最通用
- B. 使用 `<header>`、`<nav>`、`<main>`、適當 heading、圖片 `alt`、表單 `<label>`，並使用適當的互動元素
- C. 將所有內容改成圖片
- D. 只增加 CSS，不需要修改 HTML

答案：B

完整解析

這是一道真正的綜合題。

原始程式存在多個問題：

問題 1 | 缺少語意

```
<div id="header">
```

可以考慮：

```
<header>
```

問題 2 | 導覽沒有語意

```
<div id="menu">
```

可以改成：

```
<nav>
```

問題 3 | 主要內容

```
<div id="content">
```

可以考慮：

```
<main>
```

問題 4 | 沒有正確標題階層

公司介紹應適當使用：

```
<h1>公司介紹</h1>
```

或依頁面資訊架構使用適當 heading。

問題 5 | 圖片缺少 alt

原本：

```

```

較適合：

```

```

問題 6 | 表單缺少 label

原本：

```
<input type="text" placeholder="姓名">
```

應建立真正的 label。

問題 7 | 互動元素

原本：

```
<div onclick="goHome()">公司 Logo</div>
```

如果它是按鈕或連結，應優先使用具有原生語意的 HTML 元素。

Part 3 | HTML 100 題能力診斷

完成第 1～100 題後，不要只看總分。

應該進行以下分析。

得分	能力判斷	建議
90-100	🟢 優秀	進入 CSS
80-89	🟢 良好	複習錯題後進入 CSS
70-79	🟡 基礎尚可	強化 HTML 陷阱
60-69	🟠 不穩定	重讀 Lesson 01-35
<60	🔴 基礎不足	重新建立 HTML 基礎

但有一個重要原則：

如果第 81～100 題錯很多，即使前面得到高分，也不能認定 HTML 真正掌握。

因為後面的題目開始測：

- Semantic HTML
- Accessibility
- SEO
- AI
- 實務判斷
- 綜合能力

這才是「前端網頁技術與 AI 應用」的核心。

Part 4 | HTML 必背 30 個核心考點

考前至少要能快速回答：

#	核心考點	必記答案
1	HTML	超文字標記語言
2	DOCTYPE	HTML5 文件型別宣告
3	<code><html></code>	HTML 文件根元素
4	<code><head></code>	文件設定／資訊
5	<code><body></code>	主要頁面內容
6	<code><h1></code>	第一層標題
7	<code><p></code>	段落
8	<code><div></code>	一般區塊容器
9	<code></code>	一般行內容器
10	<code><a></code>	超連結
11	<code>href</code>	連結目的地
12	<code></code>	圖片
13	<code>src</code>	圖片來源
14	<code>alt</code>	替代文字
15	<code></code>	無序清單
16	<code></code>	有序清單
17	<code></code>	清單項目
18	<code><table></code>	表格
19	<code><tr></code>	表格列
20	<code><th></code>	標題儲存格

#	核心考點	必記答案
21	<td>	資料儲存格
22	<form>	表單
23	required	必填
24	placeholder	輸入提示
25	<label>	表單標籤
26	<nav>	導覽
27	<main>	主要內容
28	<article>	可獨立內容
29	<section>	主題內容區段
30	<footer>	頁尾

Part 5 | HTML 甄試「十大陷阱」

陷阱 01

id ≠ class

陷阱 02

alt ≠ title

陷阱 03

strong ≠ b

陷阱 04

em ≠ i

陷阱 05

section ≠ div

陷阱 06

POST ≠ 加密

陷阱 07

password ≠ 資料傳輸加密

陷阱 08

placeholder ≠ label

陷阱 09

ARIA ≠ 原生 HTML 的完全替代品

陷阱 10

AI 產生的 HTML ≠ 一定正確

Part 6 | HTML 一頁式速記

HTML

- |
- ├ 文件
 - | └ <!DOCTYPE html>
 - | └ html
 - | └ head
 - | └ body
- |
- ├ 文字
 - | └ h1~h6
 - | └ p
 - | └ strong
 - | └ em
 - | └ div
 - | └ span
- |
- ├ 連結／圖片
 - | └ a → href
 - | └ img → src

- | |─ alt
- | |─ figure / figcaption
- |
- |─ 清單
- | |─ ul
- | |─ ol
- | |─ li
- |
- |─ 表格
- | |─ table
- | |─ tr
- | |─ th
- | |─ td
- |
- |─ 表單
- | |─ form
- | |─ input
- | |─ label
- | |─ select
- | |─ option
- | |─ textarea
- | |─ button
- | |─ required
- | |─ placeholder
- |
- |─ 語意
- | |─ header
- | |─ nav
- | |─ main
- | |─ section
- | |─ article
- | |─ aside
- | |─ footer
- |
- |─ SEO
- | |─ title
- | |─ meta description
- | |─ heading
- | |─ semantic HTML
- | |─ alt
- |

- └ Accessibility
 - └ alt
 - └ label
 - └ keyboard
 - └ semantic HTML
- └ ARIA

Phase 02 | 最終任務

完成本階段後，不只是「做完 100 題」，而是要達到：

Level 1 | 看得懂

能閱讀基本 HTML。

↓

Level 2 | 說得出

能解釋每個標籤的作用。

↓

Level 3 | 判斷得出

看到 HTML 能判斷正確／錯誤。

↓

Level 4 | 改得動

能修改 AI 產生的 HTML。

↓

Level 5 | 考得出

能處理甄試中的變化題與陷阱題。

Phase 02 結業標準

建議你在進入下一階段前，達成：

HTML 100 題第一次測驗 $\geq 80\%$

並且：

第 81~100 題綜合題 $\geq 70\%$

最後再做到：

不看答案，能說明 `id / class`、`alt / title`、`strong / b`、`em / i`、`section / div`、`GET / POST`、`label / placeholder` 的差異。

達成這個標準，就可以進入：

Phase 03 | CSS / RWD 課程 + 第 101~180 題

下一階段會正式進入甄試非常重要的 **CSS Selector** → **Cascade** → **Specificity** → **Box Model** → **Position** → **Flexbox** → **Grid** → **Media Query** → **RWD**，並大量加入「給你 CSS 程式碼，判斷最後畫面會長什麼樣子」的實戰題型。

2026/8/18 下午1:15:42



Phase 03 | CSS / RWD 完整課程＋第 101～180 題

前端網頁技術與 AI 應用甄試 | CSS / RWD 80 題訓練教材

本階段目標

建立「看得懂 CSS → 理解樣式優先權 → 能推算 Box Model → 看懂 Flex / Grid → 判斷 RWD → 能閱讀 AI 產生的 CSS → 解答甄試陷阱題」的能力。

Part 1 | CSS 核心課程

Lesson 01 | CSS 是什麼？

CSS 全名：

Cascading Style Sheets / 階層式樣式表

HTML 負責：

內容與結構

CSS 負責：

外觀與版面

JavaScript 負責：

互動與行為

可以記成：

HTML → 骨架
CSS → 外觀
JavaScript → 行為

Lesson 02 | CSS 基本語法

```
p {  
  color: blue;  
  font-size: 16px;  
}
```

結構：

```
selector  
  ↓  
p {  
  property: value;  
    ↑      ↑  
  屬性  值  
}
```

例如：

```
color: blue;
```

- color → property
- blue → value

Lesson 03 | 三種 CSS 使用方式

① Inline

```
<p style="color:red;">Hello</p>
```

② Internal

```
<style>  
p {  
  color: red;  
}  
</style>
```

③ External


```
<link rel="stylesheet" href="style.css">
```

實務上通常最推薦：

External CSS

因為容易維護與重複使用。

Lesson 04 | Element Selector

```
p {  
  color: red;  
}
```

代表：

所有 `<p>` 元素。

Lesson 05 | Class Selector

HTML：

```
<p class="title">Hello</p>
```

CSS：

```
.title {  
  color: blue;  
}
```

Class Selector 前面：

.

Lesson 06 | ID Selector

HTML：

```
<p id="main-title">Hello</p>
```

CSS :

```
#main-title {  
  color: red;  
}
```

ID Selector 前面 :

#

Lesson 07 | Selector 基本比較

Selector	寫法	意義
Element	p	所有 p
Class	.box	class=box
ID	#header	id=header
Universal	*	所有元素

Lesson 08 | 後代 Selector

```
nav a {  
  color: red;  
}
```

表示 :

nav 裡面的所有 a。

Lesson 09 | 子元素 Selector

```
nav > a {  
  color: red;  
}
```

表示：

`nav` 的「直接子元素」`a`。

這和：

```
nav a
```

不同。

Lesson 10 | Pseudo-class

常見：

```
a:hover {  
  color: red;  
}
```

常見 pseudo-class：

- `:hover`
- `:focus`
- `:active`
- `:visited`
- `:first-child`
- `:nth-child()`

Lesson 11 | Pseudo-element

常見：

```
p::before {  
  content: "★";  
}
```

```
}
```

常見：

- `::before`
- `::after`
- `::first-letter`
- `::first-line`

記法：

`:` → *pseudo-class*

`::` → *pseudo-element*

Lesson 12 | Cascade

CSS 中的：

Cascading

表示當多個規則同時套用時，瀏覽器會依照規則決定最終樣式。

簡化理解：

```
!important
  ↓
Specificity
  ↓
Source Order
```

實際 Cascade 還涉及 Origin、Layer、Importance 等因素，但甄試基礎題先掌握這個核心。

Lesson 13 | Specificity

基本概念：

Inline
↓
ID
↓
Class / Attribute / Pseudo-class
↓
Element

例如：

```
p {  
  color: red;  
}  
  
.text {  
  color: blue;  
}  
  
#title {  
  color: green;  
}
```

若：

```
<p id="title" class="text">  
  Hello  
</p>
```

結果：

green

因為 ID specificity 高於 class 與 element。

Lesson 14 | Inheritance

部分 CSS 屬性會繼承。

例如：

```
body {  
  color: red;  
}
```

子元素的文字通常也會繼承這個 `color` 。

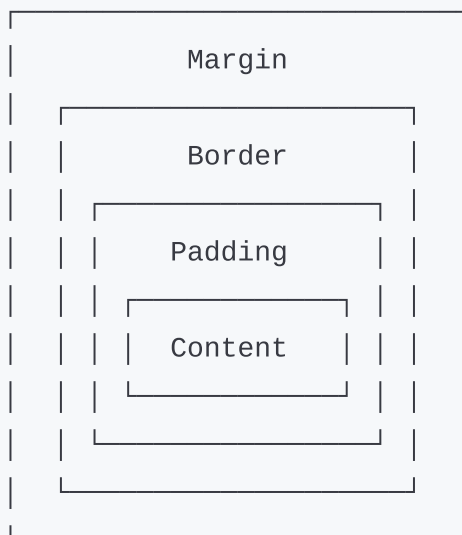
但不是所有 CSS 屬性都會繼承。

例如：

`margin` 、 `padding` 一般不會從父元素繼承。

Lesson 15 | Box Model

CSS Box Model：



順序：

Content → Padding → Border → Margin

Lesson 16 | Width 與 Box Model

預設：

```
box-sizing: content-box;
```

例如：

```
.box {  
  width: 200px;  
  padding: 20px;  
  border: 5px solid;  
}
```

實際寬度：

```
200 + 20 + 20 + 5 + 5  
= 250px
```

Lesson 17 | border-box

如果：

```
box-sizing: border-box;
```

則：

```
width: 200px;
```

代表：

Content + Padding + Border 的總寬度為 200px。

這是非常常見的 RWD 實務設定。

Lesson 18 | Margin vs Padding

Margin

元素外部的空間。

Padding

元素內部的空間。

記憶：

Margin → 外

Padding → 內

Lesson 19 | display

常見：

```
display: block;
display: inline;
display: inline-block;
display: flex;
display: grid;
display: none;
```

Lesson 20 | Block vs Inline

block

通常：

- 會另起一行
- 可設定 width / height

例如：

```
div
```

inline

通常：

- 不會強制另起一行
- width / height 行為與 block 不同

例如：

```
span
```


Lesson 21 | display:none

```
display: none;
```

元素：

不會出現在頁面版面中。

也就是：

不佔據原本的 *layout* 空間。

Lesson 22 | visibility:hidden

```
visibility: hidden;
```

元素：

看不到，但通常仍保留原本的版面空間。

這是很常見的陷阱題。

Lesson 23 | opacity

```
opacity: 0;
```

元素會：

完全透明。

但：

通常仍然存在於 *layout* 中，也可能仍能接收互動。

所以：

```
display:none  
≠  
visibility:hidden  
≠  
opacity:0
```

Lesson 24 | overflow

常見：

```
overflow: hidden;  
overflow: auto;  
overflow: scroll;
```

例如：

```
.box {  
  width: 200px;  
  height: 100px;  
  overflow: auto;  
}
```

內容超出時可以出現捲軸。

Lesson 25 | Position

CSS：

```
position: static;  
position: relative;  
position: absolute;  
position: fixed;  
position: sticky;
```

Lesson 26 | Relative

```
position: relative;
```

常用作為：

absolute 子元素的定位參考點。

例如：

```
.parent {  
    position: relative;  
}  
  
.child {  
    position: absolute;  
}
```

Lesson 27 | Absolute

```
position: absolute;  
top: 10px;  
right: 10px;
```

通常會：

脫離一般文件流。

其定位通常參考最近的已定位祖先元素。

Lesson 28 | Fixed

```
position: fixed;  
bottom: 20px;  
right: 20px;
```

通常相對於：

瀏覽器視窗。

常見用途：

- 回到頂端按鈕
- 浮動客服
- 固定導覽

Lesson 29 | Sticky

```
position: sticky;  
top: 0;
```

可以理解為：

元素在捲動過程中達到指定位置後，呈現類似固定的效果，但仍受其容器範圍限制。

Lesson 30 | z-index

```
z-index: 10;
```

主要用於：

控制定位元素的堆疊順序。

通常數值較大者會位於較上層，但實際結果也會受到 stacking context 影響。

Lesson 31 | Flexbox

Flex 是：

一維版面配置系統。

```
.container {  
  display: flex;  
}
```

預設主軸：

水平。

Lesson 32 | justify-content

```
justify-content: center;
```

控制：

主軸方向的排列。

常見：

```
flex-start  
center  
flex-end  
space-between  
space-around  
space-evenly
```

Lesson 33 | align-items

```
align-items: center;
```

控制：

交叉軸方向的對齊。

Lesson 34 | flex-direction

```
flex-direction: row;
```

水平排列。

```
flex-direction: column;
```

垂直排列。

Lesson 35 | Flex 必背

justify-content → 主軸
align-items → 交叉軸

但是：

主軸方向會受到 `flex-direction` 影響。

因此不能死背：

`justify` 永遠水平。

Lesson 36 | CSS Grid

Grid 是：

二維版面配置系統。

可以同時處理：

- Rows
- Columns

例如：

```
.container {  
  display: grid;  
  grid-template-columns: repeat(3, 1fr);  
}
```

代表：

三欄。

Lesson 37 | Flex vs Grid

Flex

一維

Row 或 Column

元件排列

導覽、按鈕

Grid

二維

Row + Column

複雜版面

網頁整體 layout

Lesson 38 | CSS Units

常見：

- px
- %
- em
- rem
- vw
- vh

Lesson 39 | px

px：

CSS 像素單位。

例如：

```
font-size: 16px;
```

Lesson 40 | %

百分比通常相對於：

某個參考尺寸，例如父元素。

```
width: 50%;
```

Lesson 41 | em vs rem

em

通常相對於：

當前元素的字體大小。

rem

相對於：

root 元素，也就是通常的 `<html>` 字體大小。

Lesson 42 | vw / vh

vw

viewport width

100vw：

視窗寬度的 100%。

vh

viewport height

100vh：

視窗高度的 100%。

Lesson 43 | Media Query

RWD 核心：


```
@media (max-width: 768px) {  
  .menu {  
    display: none;  
  }  
}
```

意思：

當 viewport 寬度符合條件時套用這些 CSS。

Lesson 44 | Responsive Web Design

RWD：

Responsive Web Design / 響應式網頁設計

目的：

讓網站適應不同螢幕尺寸與裝置。

例如：

- Desktop
- Tablet
- Mobile

Lesson 45 | Mobile First

Mobile First：

先設計手機，再逐步增加較大螢幕的樣式。

常見：

```
/* Mobile */  
.card {  
  width: 100%;
```

```
}

/* Desktop */
@media (min-width: 768px) {
  .card {
    width: 50%;
  }
}
```

Lesson 46 | Desktop First

反過來：

先設計桌機，再透過 *Media Query* 調整手機。

兩種都可以。

但現代 RWD 開發中：

Mobile First 是非常常見的策略。

Lesson 47 | CSS Variables

```
:root {
  --primary-color: #0066cc;
}
```

使用：

```
.button {
  background-color: var(--primary-color);
}
```

優點：

集中管理可重複使用的設計值。

Lesson 48 | CSS Reset / Normalize 概念

不同瀏覽器可能具有不同的預設樣式。

常見做法：

```
* {  
  box-sizing: border-box;  
}
```

可以讓 Box Model 計算更加容易控制。

Lesson 49 | CSS 與 Accessibility

不能只追求「漂亮」。

例如：

```
button {  
  outline: none;  
}
```

如果沒有其他 Focus Style：

可能造成鍵盤使用者難以知道目前焦點在哪裡。

因此：

不要隨意移除 *focus indicator*。

Lesson 50 | AI 產生 CSS 的正確流程

```
需求  
↓  
AI 產生 CSS  
↓  
人工閱讀  
↓  
檢查 Selector  
↓  
檢查 Specificity  
↓
```

檢查 Responsive



測試不同瀏覽器



檢查 Accessibility



最佳化

AI 不是：

「CSS 自動正確產生器」。

Part 2 | 第 101~180 題

第 101~120 題 | CSS 基礎與 Selector

第 101 題 ★

CSS 的全名是：

- A. Computer Style Sheets
- B. Cascading Style Sheets
- C. Creative Style System
- D. Colorful Style Sheets

答案：B

第 102 題 ★

CSS 主要負責：

- A. 網頁外觀與版面
- B. DNS
- C. 資料庫
- D. CPU 控制

答案：A

第 103 題 ★

下列哪個是正確的 CSS 語法？

- A. `p { color: red; }`
- B. `p = color:red`
- C. `p: color = red`
- D. `<p color="red">`

答案：A

第 104 題 ★

CSS 中：

```
color: red;
```

color 是：

- A. Selector
- B. Property
- C. Value
- D. Element

答案：B

第 105 題 ★

CSS 中：

```
color: red;
```

red 是：

- A. Property
- B. Selector
- C. Value
- D. Class

答案：C

第 106 題 ★

以下 CSS：

```
p {  
    color: blue;  
}
```

p 是：

- A. Selector
- B. Property
- C. Value
- D. Attribute

答案：A

第 107 题 ★★

Class Selector 的正確寫法：

- A. #box
- B. .box
- C. box.
- D. @box

答案：B

第 108 题 ★★

ID Selector：

- A. .header
- B. #header
- C. header#
- D. @header

答案：B

第 109 题 ★★

下列：

```
.card {  
    color: red;
```

```
}
```

會套用到：

- A. id="card"
- B. class="card"
- C. <card>
- D. name="card"

答案：B

第 110 題 ★★

下列：

```
#header {  
    color: blue;  
}
```

會套用到：

- A. class="header"
- B. id="header"
- C. name="header"
- D. <header> 的所有元素

答案：B

第 111 題 ★

Universal Selector 是：

- A. #
- B. .
- C. *
- D. @

答案：C

第 112 題 ★★

```
nav a {  
  color: red;  
}
```

代表：

- A. nav 本身
- B. nav 裡面的 a
- C. 所有 nav 和 a
- D. 只有直接子元素 a

答案：B

第 113 題 ★★

```
nav > a {  
  color: red;  
}
```

> 代表：

- A. 所有後代
- B. 直接子元素
- C. 父元素
- D. 相鄰元素

答案：B

第 114 題 ★★

以下哪一個是 pseudo-class？

- A. `::before`
- B. `:hover`
- C. `::after`
- D. `::first-letter`

答案：B

第 115 題 ★★

以下哪一個是 pseudo-element ？

- A. `:hover`
- B. `:focus`
- C. `::before`
- D. `:active`

答案：C

第 116 題 ★★

```
a:hover {  
  color: red;  
}
```

通常表示：

- A. 滑鼠移到連結上時
- B. 點擊後永久變紅
- C. 網頁載入時
- D. 鍵盤關閉時

答案：A

第 117 題 ★★

`:focus` 常與哪一種情境有關？

- A. 元素取得焦點
- B. 圖片壓縮
- C. DNS
- D. 資料庫

答案：A

第 118 題 ★★★

```
p::before {  
  content: "★";  
}
```

主要會：

- A. 在 p 前方產生 pseudo-element 內容
- B. 修改 HTML 原始碼
- C. 建立資料庫
- D. 刪除 p

答案：A

第 119 題 ★★

下列何者是 Element Selector？

- A. `.menu`
- B. `#menu`
- C. `nav`
- D. `*menu`

答案：C

第 120 題 ★★★

若 HTML：

```
<p class="text" id="title">Hello</p>
```

CSS：

```
p { color:red; }  
.text { color:blue; }  
#title { color:green; }
```

最後顏色通常是：

- A. red
- B. blue
- C. green
- D. black

答案：C

考點：Specificity。

第 121~140 題 | Cascade、Specificity、Box Model

第 121 題 ★★

CSS Specificity 通常較高的是：

- A. Element
- B. Class
- C. ID
- D. Universal

答案：C

第 122 題 ★★

下列一般情況下 specificity 最低：

- A. ID
- B. Class
- C. Element
- D. Inline

答案：C

第 123 題 ★★★

下列何者通常具有較高 specificity？

- A. p
- B. .box
- C. #box
- D. *

答案：C

第 124 題 ★★★

CSS：

```
p {  
  color: red;  
}
```

```
p {  
  color: blue;  
}
```

如果其他條件相同，最後通常是：

- A. red
- B. blue
- C. black
- D. 無法顯示

答案：B

原因：

後出現的規則在其他條件相同時通常勝出。

第 125 題 ★★★

CSS 的 Cascade 主要是在處理：

- A. 多個 CSS 規則如何決定最終樣式
- B. DNS
- C. JavaScript 編譯
- D. CPU 排程

答案：A

第 126 題 ★★★★★

下列何者一般優先級最高？

- A. Element Selector
- B. Class Selector
- C. ID Selector
- D. Inline Style

答案：D

第 127 題 ★★★★★

`!important` 的作用是：

- A. 提高宣告的重要性優先級
- B. 自動產生 CSS
- C. 加密 CSS
- D. 建立 ID

答案：A

第 128 題 ★★★★★

下列哪個做法最不建議大量使用？

- A. Class
- B. CSS Variables
- C. !important
- D. External CSS

答案：C

第 129 題 ★

CSS Box Model 不包含：

- A. Content
- B. Padding
- C. Border
- D. JavaScript

答案：D

第 130 題 ★

Box Model 從內向外：

- A. Margin → Border → Padding → Content
- B. Content → Padding → Border → Margin
- C. Content → Margin → Border → Padding
- D. Border → Content → Margin → Padding

答案：B

第 131 題 ★★

padding 是：

- A. 元素內部空間
- B. 元素外部空間
- C. 邊框外的空間
- D. 螢幕空間

答案：A

第 132 題 ★★

margin 是：

- A. 元素內部空間
- B. 元素外部空間
- C. 文字大小
- D. 邊框厚度

答案：B

第 133 題 ★★★

```
.box {  
  width: 200px;  
  padding: 20px;  
}
```

預設 content-box 時，若不考慮 border，實際寬度為：

- A. 200px
- B. 220px
- C. 240px
- D. 160px

答案：C

第 134 題 ★★★

```
.box {  
  width: 200px;  
  padding: 20px;
```

```
border: 5px solid;  
}
```

預設 `content-box`，實際寬度為：

- A. 200px
- B. 240px
- C. 250px
- D. 260px

答案：C

第 135 題 ★★★★★

若：

```
box-sizing: border-box;  
width: 200px;  
padding: 20px;  
border: 5px solid;
```

元素總寬度為：

- A. 200px
- B. 240px
- C. 250px
- D. 150px

答案：A

第 136 題 ★★

`box-sizing: border-box` 的重要意義：

- A. width 包含 padding 和 border
- B. width 只包含 margin
- C. 隱藏元素
- D. 自動建立 Grid

答案：A

第 137 題 ★★★

哪一項通常不會繼承父元素的 `color` ？

- A. 子元素文字
- B. CSS 的 `color` 本身
- C. `margin`
- D. 部分文字內容

答案：C

第 138 題 ★★★

`display: none`：

- A. 隱藏但保留空間
- B. 不顯示且通常不佔 layout 空間
- C. 變成透明
- D. 固定在畫面

答案：B

第 139 題 ★★★

`visibility: hidden`：

- A. 通常隱藏但保留版面空間
- B. 完全移除元素
- C. 變成 `fixed`
- D. 改成 `inline`

答案：A

第 140 題 ★★★

`opacity: 0` 通常代表：

- A. 元素完全透明
- B. 元素不存在
- C. `display none`
- D. 自動移除

答案：A

第 141~160 題 | Display、Position、Flex

第 141 題 ★

哪個通常是 block-level 元素？

- A. `<div>`
- B. `<span>`
- C. `<a>`
- D. `<strong>`

答案：A

第 142 題 ★

哪個通常是 inline 元素？

- A. `<div>`
- B. `<p>`
- C. `<span>`
- D. `<section>`

答案：C

第 143 題 ★★

```
display: flex;
```

主要用途：

- A. 啟用 Flexbox layout
- B. 隱藏元素
- C. 建立圖片
- D. 設定 DNS

答案：A

第 144 題 ★★

Flexbox 主要屬於：

- A. 一維 layout
- B. 二維 layout
- C. 三維 layout
- D. 資料庫

答案：A

第 145 題 ★★

CSS Grid 主要屬於：

- A. 一維 layout
- B. 二維 layout
- C. 只有文字 layout
- D. JavaScript

答案：B

第 146 題 ★★★

```
.container {  
  display: flex;  
  justify-content: center;  
}
```

預設 `flex-direction: row` 時，元素主要會：

- A. 垂直置中
- B. 水平置中
- C. 消失
- D. 固定在右下角

答案：B

第 147 題 ★★★

```
align-items: center;
```

主要控制：

- A. 交叉軸對齊
- B. 字型
- C. 外距
- D. 邊框

答案：A

第 148 題 ★★★

```
flex-direction: column;
```

主要軸方向變為：

- A. 水平
- B. 垂直
- C. 對角線
- D. 固定

答案：B

第 149 題 ★★★★★

若：

```
.container {  
  display: flex;  
  flex-direction: column;  
  justify-content: center;  
}
```

`justify-content` 主要控制：

- A. 垂直方向
- B. 水平方向
- C. 字型
- D. 邊框

答案：A

陷阱：`justify-content` 控制的是主軸，而主軸會隨 `flex-direction` 改變。

第 150 題 ★★★★★

若：

```
.container {  
  display: flex;  
  flex-direction: column;  
  align-items: center;  
}
```

主要會將子元素在：

- A. 垂直主軸置中
- B. 水平交叉軸置中
- C. 固定位置
- D. 隱藏

答案：B

第 151 題 ★★

`position: relative` 常見用途之一：

- A. 作為 absolute 子元素的定位參考
- B. 隱藏元素
- C. 建立 Grid
- D. 建立 DNS

答案：A

第 152 題 ★★★★★

`position: absolute` 的元素通常：

- A. 仍完全按照正常文件流排列
- B. 脫離一般文件流
- C. 永遠固定在視窗
- D. 不能設定 top

答案：B

第 153 題 ★★★★★

下列何者常用來讓元素固定在視窗右下角？

- A. `position: fixed;`
- B. `position: static;`
- C. `display: inline;`
- D. `float: none;`

答案：A

第 154 題 ★★★

`position: sticky` 的特色是：

- A. 可以在捲動時呈現類似固定的效果
- B. 永遠脫離所有容器
- C. 完全隱藏
- D. 只能用於圖片

答案：A

第 155 題 ★★★

`z-index` 主要控制：

- A. 字體大小
- B. 堆疊順序
- C. 寬度
- D. 透明度

答案：B

第 156 題 ★★★★★

兩個適當建立 stacking context 的元素重疊時，`z-index` 通常可用來：

- A. 控制前後堆疊順序
- B. 控制字體
- C. 改變 HTML
- D. 修改 DNS

答案：A

第 157 題 ★★★

以下：

```
.container {  
  display: flex;  
  justify-content: space-between;  
}
```

通常會：

- A. 將子元素平均拉開，使第一個與最後一個貼近主軸兩端
- B. 全部疊在一起
- C. 全部隱藏
- D. 變成 Grid

答案：A

第 158 題 ★★★

`align-items: stretch` 在符合條件時通常表示：

- A. 交叉軸方向延伸
- B. 主軸置中
- C. 元素隱藏
- D. 固定元素

答案：A

第 159 題 ★★★★★

下列哪個最適合三欄等寬 Grid？

- A. `grid-template-columns: repeat(3, 1fr);`
- B. `grid-column: 3px;`
- C. `grid-row: three;`
- D. `grid: columns(3);`

答案：A

第 160 題 ★★★★★

Flex 與 Grid 的正確比較：

- A. Flex 是二維，Grid 是一維
- B. Flex 通常適合一維配置，Grid 適合二維配置
- C. 兩者完全不能共存
- D. Grid 只能用於文字

答案：B

第 161~180 題 | RWD、Units、AI 情境與綜合題

第 161 題 ★

px 是：

- A. CSS 長度單位
- B. HTML 元素
- C. JavaScript 變數
- D. DNS

答案：A

第 162 題 ★★

rem 通常相對於：

- A. <html> 的字體大小
- B. <body> 寬度
- C. viewport 高度
- D. 圖片大小

答案：A

第 163 題 ★★

em 通常與哪個概念有關？

- A. 當前元素的字體大小
- B. DNS
- C. CPU
- D. 圖片解析度

答案：A

第 164 題 ★★

1vw 通常代表：

- A. viewport 寬度的 1%
- B. viewport 高度的 1%
- C. 1px
- D. 1em

答案：A

第 165 題 ★★

1vh 通常代表：

- A. viewport 寬度的 1%
- B. viewport 高度的 1%
- C. 1rem
- D. 1px

答案：B

第 166 題 ★★

RWD 的主要目的：

- A. 適應不同裝置與螢幕尺寸
- B. 只支援桌機
- C. 只支援手機
- D. 增加 CPU

答案：A

第 167 題 ★★

Media Query 主要用來：

- A. 根據條件套用不同 CSS
- B. 查詢 DNS

- C. 建立 HTML
- D. 執行 SQL

答案：A

第 168 題 ★★★

```
@media (max-width: 768px) {  
  .menu {  
    display: none;  
  }  
}
```

當 viewport 寬度為 600px 時：

- A. 規則通常會套用
- B. 規則一定不會套用
- C. CSS 無效
- D. HTML 消失

答案：A

第 169 題 ★★★

```
@media (min-width: 768px) {  
  .container {  
    width: 80%;  
  }  
}
```

當 viewport 寬度為 1000px 時：

- A. 規則符合條件
- B. 規則不符合
- C. CSS 失效
- D. HTML 被刪除

答案：A

第 170 題 ★★★

Mobile First 的主要概念：

- A. 先設計手機，再逐步擴展到較大螢幕
- B. 只能做手機網站
- C. 只使用 JavaScript
- D. 不需要 Media Query

答案：A

第 171 題 ★★★

下列哪個較符合 Mobile First？

```
.card {  
    width: 100%;  
}  
  
@media (min-width: 768px) {  
    .card {  
        width: 50%;  
    }  
}
```

- A. 正確
- B. 錯誤
- C. 這是 Desktop Only
- D. 這不是 CSS

答案：A

第 172 題 ★★★

CSS Variable 的正確寫法：

```
:root {  
    --primary: #0066cc;  
}
```

使用：

```
.button {  
  color: var(--primary);  
}
```

上述：

- A. 正確
- B. 錯誤
- C. 只能在 JavaScript 使用
- D. 是 HTML

答案：A

第 173 題 ★★★★★

使用 CSS Variables 的主要優點之一：

- A. 集中管理可重複使用的設計值
- B. 自動產生 HTML
- C. 自動建立資料庫
- D. 自動壓縮圖片

答案：A

第 174 題 ★★★★★

以下：

```
.container {  
  display: grid;  
  grid-template-columns: 1fr 1fr;  
}
```

最可能表示：

- A. 兩個等比例欄位
- B. 一個欄位
- C. 三個欄位
- D. 隱藏元素

答案：A

第 175 題 ★★★★★

以下：

```
.container {  
  display: flex;  
  flex-direction: row;  
}
```

若沒有其他特殊設定，Flex 子元素主要沿：

- A. 水平方向排列
- B. 垂直方向排列
- C. 固定位置排列
- D. 隱藏

答案：A

第 176 題 ★★★★★☆

某網站在手機上出現水平捲軸，內容超出螢幕。下列哪個方向最合理？

- A. 檢查固定寬度、overflow、圖片尺寸與 RWD CSS
- B. 直接把網站刪除
- C. 把所有文字變成圖片
- D. 增加更多固定寬度

答案：A

第 177 題 ★★★★★☆

AI 產生以下 CSS：

```
* {  
  box-sizing: border-box;  
}  
  
.container {  
  display: flex;  
  justify-content: center;
```

```
align-items: center;
}
```

若你要確認這段 CSS 是否符合需求，最重要的做法是：

- A. 只看 AI 說明
- B. 實際測試不同內容與螢幕尺寸
- C. 直接上線
- D. 把所有 CSS 刪掉

答案：B

第 178 題 ★★★★★☆

某網站 CSS：

```
.box {
  width: 100%;
}

@media (min-width: 768px) {
  .box {
    width: 50%;
  }
}
```

這種寫法最接近：

- A. Mobile First
- B. Desktop First
- C. Fixed Layout
- D. Table Layout

答案：A

第 179 題 ★★★★★☆

下列哪一種情況最適合使用 Grid？

- A. 複雜的二維頁面版面
- B. 只有一個文字段落

- C. DNS 設定
- D. JavaScript 變數

答案：A

第 180 題 ★★★★★ | CSS / RWD 綜合挑戰題

有以下 HTML：

```
<div class="container">
  <div class="card">A</div>
  <div class="card">B</div>
  <div class="card">C</div>
</div>
```

CSS：

```
.container {
  display: grid;
  grid-template-columns: 1fr;
  gap: 20px;
}

.card {
  box-sizing: border-box;
  width: 100%;
}

@media (min-width: 768px) {
  .container {
    grid-template-columns: repeat(3, 1fr);
  }
}
```

若 viewport 為 **1200px**，下列何者最正確？

- A. 三個 card 垂直排列成一欄
- B. 三個 card 排成三欄
- C. card 全部消失
- D. Grid 失效

答案：B

完整解析

初始：

```
grid-template-columns: 1fr;
```

代表：

一欄。

但是 Media Query：

```
@media (min-width: 768px)
```

1200px：

大於 768px。

因此 Media Query 生效：

```
grid-template-columns: repeat(3, 1fr);
```

也就是：



所以答案：

B

Part 3 | CSS / RWD 必背 30 個考點

#	考點	必記
1	CSS	Cascading Style Sheets
2	Element Selector	p
3	Class	.class
4	ID	#id
5	Universal	*
6	Pseudo-class	:hover
7	Pseudo-element	::before
8	Specificity	ID > Class > Element
9	Box Model	Content → Padding → Border → Margin
10	Padding	內部空間
11	Margin	外部空間
12	box-sizing	控制 width/height 的計算方式
13	border-box	width 包含 padding + border
14	display:none	不佔一般 layout 空間
15	visibility:hidden	通常保留空間
16	opacity:0	完全透明
17	relative	定位參考常見用途
18	absolute	脫離一般文件流
19	fixed	相對視窗固定
20	sticky	捲動時可呈現黏性定位
21	z-index	堆疊順序
22	Flex	一維 layout

#	考點	必記
23	Grid	二維 layout
24	justify-content	主軸
25	align-items	交叉軸
26	rem	相對 root font-size
27	vw	viewport width
28	vh	viewport height
29	Media Query	條件式 CSS
30	Mobile First	先手機、後大螢幕

Part 4 | CSS / RWD 十大甄試陷阱

陷阱 01

`justify-content` 永遠代表水平？

錯。

它控制：

主軸

主軸由 `flex-direction` 決定。

陷阱 02

`display:none` 與 `visibility:hidden` 一樣？

錯。

`display:none` 通常不佔 layout 空間。

`visibility:hidden` 通常保留空間。

陷阱 03

`opacity:0` 就等於元素不存在？

錯。

它只是透明。

陷阱 04

`width:200px` 永遠代表整個元素寬度？

不一定。

要看：

box-sizing

陷阱 05

Flex 是二維排版？

錯。

Flex 通常是一維。

Grid 才是二維。

陷阱 06

`position:absolute` 永遠相對 *viewport*？

錯。

通常會尋找最近的已定位祖先。

陷阱 07

`z-index:9999` 一定在所有元素上面？

錯。

還要考慮 stacking context。

陷阱 08

`rem` 相對於父元素？

錯。

通常相對於 root `<html>`。

陷阱 09

Mobile First 就是只做手機版？

錯。

它是：

先建立手機基礎樣式，再擴展到較大的螢幕。

陷阱 10

AI 產生 *CSS* 就一定符合 *RWD*？

錯。

必須：

實際測試。

Part 5 | CSS / RWD 一頁式速記

```
CSS
|
├─ Selector
|   ├─ element → p
|   ├─ class → .box
|   ├─ id → #box
|   ├─ pseudo-class → :hover
|   └─ pseudo-element → ::before
|
├─ Cascade
|   └─ !important
```

- | | specificity
- | | source order
- |
- | Box Model
- | | content
- | | padding
- | | border
- | | margin
- |
- | Position
- | | static
- | | relative
- | | absolute
- | | fixed
- | | sticky
- |
- | Flex
- | | display:flex
- | | justify-content → 主軸
- | | align-items → 交叉軸
- | | flex-direction
- |
- | Grid
- | | columns
- | | rows
- | | 2D layout
- |
- | Units
- | | px
- | | %
- | | em
- | | rem
- | | vw
- | | vh
- |
- | RWD
- | | Media Query
- | | Mobile First
- | | Flexible Layout
- | | Responsive Images

Part 6 | Phase 03 能力檢測

完成第 101~180 題後：

得分	能力
72-80	🟢 CSS/RWD 優秀
64-71	🟢 良好
56-63	🟡 基礎尚可
48-55	🟠 需要加強
<48	🔴 建議重新學習

但真正的過關標準是：

你應該能夠不看答案解釋：

***Selector*、*Specificity*、*Box Model*、*box-sizing*、*Flex* 主軸／交叉軸、*Grid*、*Position*、*Media Query*、*Mobile First* 的差異。**

尤其是下面 5 題必須能自己推導：

第 120、135、149、168、180 題。

因為這五類題型代表：

***Selector* → *Box Model* → *Flex* → *RWD* → 綜合判斷**

已經從「背 CSS」進入「真正理解 CSS」。

Phase 03 → Phase 04

完成 CSS / RWD 後，下一個重要關卡就是：

Phase 04 | JavaScript 課程 + 第 181~250 題

會開始進入前端甄試中最容易讓初學者失分的部分：

var / let / const → Data Types → == / === → Operators → if / switch → loop → Function → Arrow Function → Scope → Array → Object → DOM →

querySelector() → getElementById() → **Event** → addEventListener() → classList →
innerHTML / textContent → **JSON** → **localStorage** → 程式碼閱讀與錯誤判斷。

而且這一階段會特別強化「給你 5~10 行 JavaScript，請判斷最後輸出什麼」，這會是整套 500 題裡非常重要的得分能力。

2026/8/18 下午1:17:47



Phase 04 | JavaScript 課程 + 第 181~250 題

前端網頁技術與 AI 應用甄試 | JavaScript 70 題完整訓練教材

本階段核心目標

從「看得懂 JavaScript」進階到「能讀懂程式、預測輸出、找出錯誤、理解 DOM 與事件」。

這一階段特別重要，因為甄試很可能不是問你「JavaScript 是什麼」，而是直接給你一小段程式碼，要求判斷輸出結果、型別、執行順序或錯誤原因。

Part 1 | JavaScript 核心課程

Lesson 01 | JavaScript 是什麼？

JavaScript 是網頁中的：

程式語言，主要負責互動與行為。

三者可以這樣記：

HTML



結構

CSS



外觀

JavaScript



行為 / 互動

例如：

按鈕被點擊



JavaScript



顯示訊息

Lesson 02 | 變數 Variable

JavaScript 常見三種宣告：

```
var x = 10;  
let y = 20;  
const z = 30;
```

目前開發最常使用：

`let` 、 `const`

Lesson 03 | var

```
var name = "Tom";
```

`var` 是較舊的變數宣告方式。

特色：

- Function Scope
- 可以重新指定
- 可以重複宣告

例如：

```
var x = 10;  
var x = 20;
```

通常可以執行。

Lesson 04 | let

```
let age = 20;
```


特色：

- Block Scope
- 可以重新指定
- 同一作用域不能重複宣告

例如：

```
let x = 10;  
x = 20;
```

合法。

但：

```
let x = 10;  
let x = 20;
```

會造成錯誤。

Lesson 05 | const

```
const PI = 3.14159;
```

const：

不能重新指定變數所綁定的值。

例如：

```
const x = 10;  
x = 20;
```

會產生錯誤。

Lesson 06 | let / const / var 比較

特性	var	let	const
可重新指定	✓	✓	✗
Block Scope	✗	✓	✓
可同作用域重複宣告	✓	✗	✗
現代 JS 推薦	較少	✓	✓

甄試記憶：

能不改就 *const*，需要改就 *let*。

Lesson 07 | JavaScript Data Types

常見資料型別：

```
String
Number
Boolean
Undefined
Null
Object
BigInt
Symbol
```

甄試最常見：

String / Number / Boolean / Undefined / Null / Object

Lesson 08 | String

```
let name = "Alice";
```

字串可以使用：

```
"Hello"
'Hello'
```

```
`Hello`
```

Template Literal :

```
let name = "Tom";  
  
console.log(`Hello ${name}`);
```

結果：

```
Hello Tom
```

Lesson 09 | Number

JavaScript 的 Number 可以表示：

```
let age = 20;  
let price = 99.5;
```

整數與浮點數都屬於：

number

Lesson 10 | Boolean

只有兩種：

```
true  
false
```

例如：

```
let isLogin = true;
```

Lesson 11 | Undefined

```
let x;
```

```
console.log(x);
```

結果：

```
undefined
```

表示：

變數已宣告，但尚未被賦予值。

Lesson 12 | Null

```
let user = null;
```

`null` 通常表示：

明確表示「沒有值」。

注意：

```
undefined  
≠  
null
```

Lesson 13 | typeof

可以使用：

```
typeof value
```

例如：

```
typeof "Hello"
```

結果：

```
"string"
```

```
typeof 100
```

結果：

```
"number"
```

Lesson 14 | Array

陣列：

```
let fruits = ["Apple", "Banana", "Orange"];
```

索引從：

0 開始

因此：

```
fruits[0]
```

是：

```
Apple
```

Lesson 15 | Array length

```
let fruits = ["A", "B", "C"];

console.log(fruits.length);
```

結果：

```
3
```

注意：

最後一個索引是 `length - 1`。

Lesson 16 | Object

```
let user = {  
  name: "Tom",  
  age: 30  
};
```

存取：

```
user.name
```

結果：

```
Tom
```

也可以：

```
user["name"]
```

Lesson 17 | Operator

常見運算子：

```
+  
-  
*  
/  
%
```

例如：

```
10 % 3
```

結果：

```
1
```

%：

取餘數。

Lesson 18 | 比較運算子

```
>  
<  
>=  
<=  
==  
===  
!=  
!==
```

其中最重要：

```
== 寬鬆相等  
=== 嚴格相等
```

Lesson 19 | == vs ===

```
10 == "10"
```

結果：

```
true
```

因為可能進行型別轉換。

但：

```
10 === "10"
```

結果：

```
false
```

因為：

Number

≠

String

必背：

`===` 同時比較「值與型別」。

Lesson 20 | if

```
if (age >= 18) {  
    console.log("Adult");  
}
```

如果條件為 `true`：

執行區塊。

Lesson 21 | if / else

```
if (score >= 60) {  
    console.log("Pass");  
} else {  
    console.log("Fail");  
}
```

Lesson 22 | else if

```
if (score >= 90) {  
    console.log("A");  
} else if (score >= 60) {  
    console.log("B");  
} else {  
    console.log("C");  
}
```


Lesson 23 | switch

```
let day = 1;

switch (day) {
  case 1:
    console.log("Monday");
    break;

  case 2:
    console.log("Tuesday");
    break;

  default:
    console.log("Other");
}
```

Lesson 24 | break

break :

跳出迴圈或 switch。

Lesson 25 | for

```
for (let i = 0; i < 5; i++) {
  console.log(i);
}
```

輸出：

```
0
1
2
3
4
```

Lesson 26 | while

```
let i = 0;

while (i < 5) {
  console.log(i);
  i++;
}
```

只要條件為：

`true`

就繼續執行。

Lesson 27 | Function

```
function add(a, b) {
  return a + b;
}
```

呼叫：

```
add(10, 20);
```

結果：

```
30
```

Lesson 28 | return

```
function square(x) {
  return x * x;
}
```

`return`：

將結果返回給呼叫端。

Lesson 29 | Arrow Function

傳統：

```
function add(a, b) {  
  return a + b;  
}
```

Arrow Function：

```
const add = (a, b) => {  
  return a + b;  
};
```

簡寫：

```
const add = (a, b) => a + b;
```

Lesson 30 | Scope

Scope：

變數可以被存取的範圍。

例如：

```
let x = 10;  
  
if (true) {  
  let y = 20;  
}
```

y 在區塊外通常不能直接使用。

Lesson 31 | DOM

DOM：

Document Object Model

瀏覽器會把 HTML 文件表示成一個：

樹狀結構

例如：

```
document
├── html
│   ├── head
│   └── body
│       ├── h1
│       ├── p
│       └── button
```

JavaScript 可以透過 DOM：

讀取、修改 HTML 元素。

Lesson 32 | getElementById

HTML：

```
<h1 id="title">Hello</h1>
```

JavaScript：

```
const title = document.getElementById("title");
```

取得：

`id="title"` 的元素。

Lesson 33 | querySelector

```
const title = document.querySelector("#title");
```

可以使用 CSS Selector。

例如：

```
document.querySelector(".card");
```

取得符合 selector 的第一個元素。

Lesson 34 | querySelectorAll

```
const cards = document.querySelectorAll(".card");
```

取得：

| 所有符合 selector 的元素集合。

Lesson 35 | innerHTML

```
element.innerHTML = "<strong>Hello</strong>";
```

可以：

| 設定 HTML 內容。

Lesson 36 | textContent

```
element.textContent = "Hello";
```

設定：

| 純文字內容。

重要安全觀念：

| 不應把不可信使用者輸入直接塞進 `innerHTML`。

Lesson 37 | classList

常見：

```
element.classList.add("active");
```

移除：

```
element.classList.remove("active");
```

切換：

```
element.classList.toggle("active");
```

Lesson 38 | Event

常見事件：

```
click  
mouseover  
keydown  
submit  
change  
input  
load
```

例如：

```
button.addEventListener("click", function () {  
    console.log("Clicked");  
});
```

Lesson 39 | addEventListener

推薦：

```
button.addEventListener("click", handleClick);
```

相較於直接：

```
button.onclick = handleClick;
```

`addEventListener()` 更適合管理多個事件監聽器與事件處理。

Lesson 40 | localStorage

```
localStorage.setItem("name", "Tom");
```

取得：

```
localStorage.getItem("name");
```

刪除：

```
localStorage.removeItem("name");
```

清空：

```
localStorage.clear();
```

Lesson 41 | JSON

JavaScript Object：

```
const user = {  
  name: "Tom",  
  age: 30  
};
```

轉 JSON：

```
JSON.stringify(user);
```

JSON 轉回 JavaScript：

```
JSON.parse(json);
```

必背：

```
Object → JSON 字串  
JSON.stringify()
```

```
JSON 字串 → Object  
JSON.parse()
```

Lesson 42 | Array Methods

非常重要。

```
map()  
filter()  
find()  
forEach()  
push()  
pop()  
shift()  
unshift()  
includes()
```

Lesson 43 | push / pop

```
let arr = [1, 2];  
  
arr.push(3);
```

結果：

```
[1, 2, 3]
```

pop()：

移除最後一個元素。

Lesson 44 | forEach


```
const nums = [1, 2, 3];

nums.forEach(function (num) {
  console.log(num);
});
```

會依序處理每個元素。

Lesson 45 | map

```
const nums = [1, 2, 3];

const result = nums.map(num => num * 2);
```

結果：

```
[2, 4, 6]
```

map()：

建立一個新的陣列。

Lesson 46 | filter

```
const nums = [1, 2, 3, 4];

const result = nums.filter(num => num > 2);
```

結果：

```
[3, 4]
```

Lesson 47 | find

```
const nums = [10, 20, 30];
```

```
const result = nums.find(num => num > 15);
```

結果：

```
20
```

它通常回傳：

第一個符合條件的元素。

Lesson 48 | Truthy / Falsy

JavaScript 中常見 falsy：

```
false  
0  
""  
null  
undefined  
NaN
```

例如：

```
if ("" ) {  
    console.log("A");  
}
```

不會輸出 A。

Lesson 49 | 型別轉換

```
Number("123")
```

得到：

```
123
```

```
String(123)
```

得到：

```
"123"
```

```
Boolean(1)
```

得到：

```
true
```

Lesson 50 | AI × JavaScript

AI 可以：

- 解釋程式
- 產生範例
- 找 Bug
- 重構程式
- 撰寫 DOM 程式
- 產生測試案例

但：

AI 產生的 JavaScript 必須驗證。

尤其要檢查：

```
邏輯  
型別  
安全性  
效能  
瀏覽器相容性  
Accessibility
```

Part 2 | 第 181~250 題

第 181~200 題 | 變數、型別、運算子

第 181 題 ★

JavaScript 主要負責網頁的：

- A. 互動與行為
- B. DNS
- C. 硬碟分割
- D. 網路線

答案：A

第 182 題 ★

以下哪個是 JavaScript 變數宣告？

- A. `let x = 10;`
- B. `<let>x</let>`
- C. `css x = 10;`
- D. `var: x`

答案：A

第 183 題 ★★

哪一個可以重新指定？

- A. `const x = 10`
- B. `let x = 10`
- C. 都不能
- D. HTML

答案：B

第 184 題 ★★

下列哪個最適合宣告不會重新指定的變數？

- A. `var`
- B. `let`
- C. `const`
- D. `static`

答案：C

第 185 題 ★★

以下：

```
let x = 10;  
x = 20;
```

結果：

- A. 錯誤
- B. x 為 20
- C. x 為 10
- D. undefined

答案：B

第 186 題 ★★★

以下：

```
const x = 10;  
x = 20;
```

結果：

- A. x 為 20
- B. x 為 10
- C. 會造成錯誤
- D. undefined

答案：C

第 187 題 ★★

`let` 和 `const` 的重要特性：

- A. Function Scope
- B. Block Scope

- C. CSS Scope
- D. HTML Scope

答案：B

第 188 題 ★★

以下：

```
var x = 10;  
var x = 20;
```

一般情況下：

- A. 可重複宣告
- B. 一定語法錯誤
- C. CSS 錯誤
- D. DNS 錯誤

答案：A

第 189 題 ★

以下資料：

```
"Hello"
```

型別是：

- A. number
- B. string
- C. boolean
- D. object

答案：B

第 190 題 ★

以下：

```
100
```

型別是：

- A. string
- B. number
- C. boolean
- D. null

答案：B

第 191 題 ★

以下：

```
true
```

型別是：

- A. string
- B. number
- C. boolean
- D. object

答案：C

第 192 題 ★★

以下：

```
let x;
```

x 的值通常是：

- A. null
- B. undefined
- C. 0
- D. false

答案：B

第 193 題 ★★

以下：

```
let x = null;
```

表示：

- A. 明確指定沒有值
- B. x 一定是 0
- C. x 是 false
- D. x 是 undefined

答案：A

第 194 題 ★★

以下：

```
typeof "Hello"
```

結果：

- A. "text"
- B. "string"
- C. "char"
- D. "word"

答案：B

第 195 題 ★★

以下：

```
typeof 123
```

結果：

- A. "integer"
- B. "number"
- C. "numeric"
- D. "float"

答案：B

第 196 題 ★★

以下：

```
10 + 5
```

結果：

- A. 15
- B. 105
- C. "15"
- D. 5

答案：A

第 197 題 ★★

以下：

```
10 % 3
```

結果：

- A. 0
- B. 1
- C. 2
- D. 3

答案：B

第 198 題 ★★★

以下：

```
10 == "10"
```

結果：

- A. true
- B. false
- C. undefined
- D. error

答案：A

第 199 題 ★★★

以下：

```
10 === "10"
```

結果：

- A. true
- B. false
- C. 10
- D. error

答案：B

第 200 題 ★★★

=== 與 == 的主要差異：

- A. === 會比較型別
- B. == 一定不比較值
- C. === 只能比較字串
- D. 沒有差異

答案：A

第 201~220 題 | 流程控制、函式、Scope

第 201 題 ★

以下哪個用於條件判斷？

- A. if
- B. loop
- C. style
- D. html

答案：A

第 202 題 ★★

```
let age = 20;

if (age >= 18) {
  console.log("A");
}
```

結果：

- A. A
- B. B
- C. 沒有輸出
- D. Error

答案：A

第 203 題 ★★

```
let score = 50;

if (score >= 60) {
  console.log("Pass");
} else {
  console.log("Fail");
}
```

結果：

- A. Pass
- B. Fail
- C. 60
- D. Error

答案：B

第 204 題 ★★

`switch` 通常適合：

- A. 多條件分支判斷
- B. CSS 排版
- C. 圖片壓縮
- D. DNS

答案：A

第 205 題 ★★

`break` 在 `switch` 中通常用來：

- A. 跳出目前 `switch` 執行流程
- B. 建立迴圈
- C. 宣告變數
- D. 建立 CSS

答案：A

第 206 題 ★★

以下：

```
for (let i = 0; i < 3; i++) {  
  console.log(i);  
}
```

輸出：

- A. 0 1 2
- B. 1 2 3
- C. 0 1 2 3
- D. 3 2 1

答案：A

第 207 題 ★★★

以下：

```
let sum = 0;
```

```
for (let i = 1; i <= 3; i++) {  
    sum += i;  
}  
  
console.log(sum);
```

結果：

- A. 3
- B. 5
- C. 6
- D. 7

答案：C

第 208 題 ★★★

以下：

```
let i = 0;  
  
while (i < 3) {  
    i++;  
}  
  
console.log(i);
```

結果：

- A. 0
- B. 1
- C. 2
- D. 3

答案：D

第 209 題 ★

以下哪個是 Function？

- A.

```
function add(a, b) {  
    return a + b;  
}
```

- B. `function = add`
- C. `def add()`
- D. `func add()`

答案：A

第 210 題 ★★

```
function add(a, b) {  
    return a + b;  
}  
  
console.log(add(2, 3));
```

結果：

- A. 2
- B. 3
- C. 5
- D. 6

答案：C

第 211 題 ★★

`return` 主要作用：

- A. 返回結果
- B. 建立 CSS
- C. 開啟瀏覽器
- D. 建立 HTML

答案：A

第 212 題 ★★★

以下：

```
const add = (a, b) => a + b;

console.log(add(3, 4));
```

結果：

- A. 3
- B. 4
- C. 7
- D. undefined

答案：C

第 213 題 ★★

Arrow Function 的符號：

- A. =>
- B. ->
- C. ::
- D. ==>

答案：A

第 214 題 ★★★

以下：

```
let x = 10;

if (true) {
  let x = 20;
  console.log(x);
}

console.log(x);
```

輸出：

- A. 20、20
- B. 10、10
- C. 20、10
- D. 10、20

答案：C

考點：Block Scope。

第 215 題 ★★★★★

以下：

```
var x = 10;

if (true) {
    var x = 20;
}

console.log(x);
```

結果：

- A. 10
- B. 20
- C. undefined
- D. Error

答案：B

第 216 題 ★★★

下列哪個通常具有 Block Scope？

- A. var
- B. let
- C. function name only
- D. HTML

答案：B

第 217 題 ★★★★★

如果程式需要重新指定變數值，通常較適合：

- A. const
- B. let
- C. CSS
- D. JSON

答案：B

第 218 題 ★★★★★

若變數整個生命週期都不需要重新指定，通常較適合：

- A. const
- B. var
- C. let only
- D. global

答案：A

第 219 題 ★★★

下列哪一個可以減少不必要的全域變數？

- A. 使用適當的 Scope
- B. 所有變數都放 window
- C. 所有變數都用 var
- D. 不使用函式

答案：A

第 220 題 ★★★★★

JavaScript 程式設計中，Scope 的主要概念是：

- A. 變數可被存取的範圍
- B. CSS 寬度
- C. HTML 高度
- D. DNS 範圍

答案：A

第 221~240 題 | Array、Object、DOM、Event

第 221 題 ★

JavaScript Array 索引通常從：

- A. 0
- B. 1
- C. -1
- D. 10

答案：A

第 222 題 ★★

```
const fruits = ["Apple", "Banana", "Orange"];  
  
console.log(fruits[0]);
```

結果：

- A. Apple
- B. Banana
- C. Orange
- D. undefined

答案：A

第 223 題 ★★

```
const arr = [10, 20, 30];  
  
console.log(arr.length);
```

結果：

- A. 2
- B. 3
- C. 30
- D. 0

答案：B

第 224 題 ★★

```
let arr = [1, 2];

arr.push(3);

console.log(arr);
```

結果：

- A. [1]
- B. [1, 2]
- C. [1, 2, 3]
- D. [3]

答案：C

第 225 題 ★★

pop() 通常：

- A. 移除最後一個元素
- B. 加入第一個元素
- C. 移除第一個元素
- D. 排序

答案：A

第 226 題 ★★★

```
const nums = [1, 2, 3];

const result = nums.map(n => n * 2);

console.log(result);
```

結果：

- A. [1, 2, 3]
- B. [2, 4, 6]
- C. [3, 4, 5]
- D. 6

答案：B

第 227 題 ★★★

`filter()` 主要用途：

- A. 根據條件篩選元素
- B. 只能排序
- C. 建立 DOM
- D. 建立 CSS

答案：A

第 228 題 ★★★

```
const nums = [1, 2, 3, 4];  
  
const result = nums.filter(n => n > 2);
```

結果：

- A. [1, 2]
- B. [2, 3]
- C. [3, 4]
- D. [4]

答案：C

第 229 題 ★★★

`find()` 通常會回傳：

- A. 所有符合元素
- B. 第一個符合條件的元素
- C. 陣列長度
- D. Boolean only

答案：B

第 230 題 ★★

Object 通常使用：

```
const user = {  
  name: "Tom",  
  age: 30  
};
```

取得姓名：

- A. `user.name`
- B. `user->name`
- C. `user/name`
- D. `user:name`

答案：A

第 231 題 ★

DOM 全名：

- A. Document Object Model
- B. Data Object Method
- C. Document Operation Module
- D. Digital Object Model

答案：A

第 232 題 ★★

JavaScript 可以透過 DOM：

- A. 修改 HTML 元素
- B. 製造 CPU
- C. 更換 RAM
- D. 修改 DNS Server

答案：A

第 233 題 ★★

HTML：

```
<h1 id="title">Hello</h1>
```

取得它：

```
document.getElementById("title");
```

答案：

- A. 正確
- B. 錯誤

答案：A

第 234 題 ★★

以下：

```
document.querySelector(".card");
```

主要是：

- A. 取得符合 `.card` Selector 的第一個元素
- B. 取得所有 card
- C. 建立 card
- D. 刪除 card

答案：A

第 235 題 ★★★

取得所有 `.card`：

```
document.querySelectorAll(".card");
```

其目的：

- A. 取得所有符合元素
- B. 取得第一個元素

- C. 刪除所有元素
- D. 建立 CSS

答案：A

第 236 題 ★★★

以下：

```
element.textContent = "Hello";
```

主要用途：

- A. 設定文字內容
- B. 執行 SQL
- C. 修改 DNS
- D. 建立圖片

答案：A

第 237 題 ★★★

以下：

```
element.innerHTML = "<strong>Hello</strong>";
```

會：

- A. 將內容視為 HTML 解析
- B. 永遠視為純文字
- C. 刪除元素
- D. 建立 JavaScript

答案：A

第 238 題 ★★★★★

若使用者輸入內容不可信，直接：

```
element.innerHTML = userInput;
```

可能產生：

- A. XSS 風險
- B. DNS 風險
- C. CPU 超頻
- D. RWD 自動化

答案：A

第 239 題 ★★

以下：

```
button.addEventListener("click", handleClick);
```

表示：

- A. 監聽 click 事件
- B. 建立 CSS
- C. 建立 HTML
- D. 讀取 DNS

答案：A

第 240 題 ★★★

下列何者是常見 JavaScript Event ？

- A. click
- B. DNS
- C. HTTP
- D. CSS

答案：A

第 241~250 題 | JSON、localStorage、綜合與 AI

第 241 題 ★★

```
classList.add("active")：
```


- A. 新增 class
- B. 移除 class
- C. 建立 HTML
- D. 清除 CSS

答案：A

第 242 題 ★★

`classList.remove("active")`：

- A. 新增 class
- B. 移除 class
- C. 隱藏所有元素
- D. 建立 ID

答案：B

第 243 題 ★★

`classList.toggle("active")`：

- A. 在存在與不存在之間切換 class
- B. 永遠新增
- C. 永遠移除
- D. 建立 CSS

答案：A

第 244 題 ★★

`localStorage.setItem()`：

- A. 儲存資料
- B. 刪除資料
- C. 清除所有 Cookie
- D. 修改 DOM

答案：A

第 245 題 ★★

```
localStorage.setItem("name", "Tom");  
  
console.log(localStorage.getItem("name"));
```

結果：

- A. Tom
- B. name
- C. null
- D. undefined

答案：A

第 246 題 ★★★

`JSON.stringify()` 的主要作用：

- A. JavaScript 值轉成 JSON 字串
- B. JSON 轉 Object
- C. CSS 轉 HTML
- D. HTML 轉 CSS

答案：A

第 247 題 ★★★

`JSON.parse()` 的主要作用：

- A. JSON 字串轉成 JavaScript 值
- B. JavaScript 轉 CSS
- C. HTML 轉 JSON
- D. CSS 轉 JavaScript

答案：A

第 248 題 ★★★★★

以下：

```
console.log(5 + "5");
```

結果通常是：

- A. 10
- B. 55
- C. "55"
- D. Error

答案：C

原因：

JavaScript 在這種情況會發生字串串接。

第 249 題 ★★★★★☆

以下：

```
let x = 10;  
let y = "10";  
  
console.log(x == y);  
console.log(x === y);
```

輸出為：

A.

```
true  
true
```

B.

```
false  
false
```

C.

```
true  
false
```

D.

```
false
true
```

答案：C

解析

```
x == y
```

會進行寬鬆比較：

```
10 == "10"
→ true
```

但：

```
x === y
```

同時比較型別：

```
number !== string
→ false
```

因此：

C

第 250 題 ★★★★★ | AI × JavaScript 綜合題

你使用 AI 產生以下 JavaScript：

```
const button = document.querySelector("#btn");

button.addEventListener("click", () => {
  const name = document.querySelector("#name").value;

  document.querySelector("#result").innerHTML =
```

```
    "Hello " + name;  
});
```

以下哪個做法最適當？

- A. AI 寫的程式可以直接上線，不需要檢查
- B. 只要可以執行，就代表安全
- C. 應檢查 DOM 是否存在、輸入資料處理、XSS 風險、瀏覽器相容性與實際功能
- D. AI 可以取代所有前端測試

答案：C

解析

這題是本階段非常重要的：

AI × Front-End Professional Judgment

AI 可以幫助你產生程式：

```
需求  
↓  
AI 產生程式  
↓  
人類理解  
↓  
測試  
↓  
安全檢查  
↓  
效能檢查  
↓  
相容性檢查  
↓  
正式使用
```

尤其：

```
innerHTML
```

如果 `name` 是使用者可以控制的內容，就需要注意：

XSS (Cross-Site Scripting) 風險。

因此不能因為：

「AI 寫出來了」

就認定：

「一定正確」。

Part 3 | JavaScript 必背 40 個考點

#	考點	必記
1	JavaScript	行為 / 互動
2	var	Function Scope
3	let	Block Scope，可重新指定
4	const	Block Scope，不可重新指定
5	String	"Hello"
6	Number	100
7	Boolean	true / false
8	undefined	宣告但尚未賦值
9	null	明確表示無值
10	Array	索引從 0
11	Object	key / value
12	typeof	查詢型別
13	==	寬鬆相等
14	===	嚴格相等
15	%	餘數

#	考點	必記
16	if	條件判斷
17	switch	多分支
18	break	跳出流程
19	for	迴圈
20	while	條件迴圈
21	function	函式
22	return	返回值
23	Arrow Function	=>
24	Scope	變數可存取範圍
25	DOM	Document Object Model
26	getElementById	依 ID 找元素
27	querySelector	第一個符合元素
28	querySelectorAll	所有符合元素
29	textContent	純文字
30	innerHTML	HTML 內容
31	classList	操作 class
32	addEventListener	事件監聽
33	click	常見事件
34	push	尾端加入
35	pop	尾端移除
36	map	產生新陣列
37	filter	篩選陣列
38	JSON.stringify	Object → JSON 字串

#	考點	必記
39	JSON.parse	JSON → JS 值
40	localStorage	瀏覽器端儲存

Part 4 | 甄試最容易出現的 12 個 JavaScript 陷阱

陷阱 01 | `==` 與 `===`

```
10 == "10"    // true
10 === "10"   // false
```

陷阱 02 | Array 從 0 開始

```
const a = ["A", "B", "C"];

a[0] // A
a[2] // C
```

陷阱 03 | length 與最後索引不同

```
const a = [10, 20, 30];
```

```
length = 3
last index = 2
```

陷阱 04 | const 不是「內容完全不可變」

例如：

```
const user = {
  name: "Tom"
};

user.name = "John";
```


這種情況可以修改物件內容。

但：

```
user = {};
```

則不能重新指定 `user`。

這是進階甄試非常值得注意的觀念。

陷阱 05 | `map()` 不等於 `forEach()`

`map()`：

通常產生新的陣列。

`forEach()`：

主要逐一執行函式，不以建立新陣列為目的。

陷阱 06 | `filter()` 不是找第一個

```
[1,2,3,4].filter(x => x > 2)
```

結果：

```
[3,4]
```

而：

```
[1,2,3,4].find(x => x > 2)
```

結果：

```
3
```

陷阱 07 | `textContent` VS `innerHTML`

```
element.textContent = "<strong>Hello</strong>";
```

會當成文字。

而：

```
element.innerHTML = "<strong>Hello</strong>";
```

會解析 HTML。

陷阱 08 | `querySelector()`

```
document.querySelector(".box");
```

通常取得：

第一個符合元素。

不是全部。

陷阱 09 | `querySelectorAll()`

```
document.querySelectorAll(".box");
```

取得：

所有符合元素。

陷阱 10 | `null` **vs** `undefined`

`undefined`

→ 沒有被賦值

`null`

→ 明確表示沒有值

陷阱 11 | `5 + "5"`

不是 10：

```
5 + "5"
```

結果：

```
"55"
```

陷阱 12 | AI 產生的 JavaScript

最危險的觀念：

「可以執行 = 正確」

錯。

真正流程：

```
AI Code
↓
Understand
↓
Test
↓
Security
↓
Compatibility
↓
Performance
↓
Deploy
```

Part 5 | JavaScript 程式碼閱讀訓練

這部分對甄試非常重要。

題型 A

```
let x = 5;
let y = 2;
```

```
console.log(x + y);
```

先算：

5 + 2

答案：

7

題型 B

```
let x = 10;

if (x > 5) {
  console.log("A");
} else {
  console.log("B");
}
```

判斷：

10 > 5
→ true

答案：

A

題型 C

```
for (let i = 0; i < 3; i++) {
  console.log(i);
}
```

不要看成：

```
1 2 3
```

而是：

```
0 1 2
```

題型 D

```
const arr = [10, 20, 30];  
  
console.log(arr[1]);
```

答案：

```
20
```

題型 E

```
const nums = [1, 2, 3];  
  
const result = nums.map(n => n + 1);  
  
console.log(result);
```

答案：

```
[2, 3, 4]
```

Part 6 | Phase 04 最重要的「程式閱讀公式」

看到 JavaScript 題目時，不要急著選答案。

使用：

5 步驟判讀法

① 先找變數

↓

② 確認資料型別



③ 找運算子



④ 找執行順序



⑤ 最後判斷輸出

例如：

```
let x = 10;  
let y = "10";  
  
console.log(x == y);  
console.log(x === y);
```

先看：

```
x → number  
y → string
```

再看：

```
== → 寬鬆比較  
=== → 嚴格比較
```

最後：

```
true  
false
```

Phase 04 | 考前一頁速記

```
JavaScript  
|  
├─ Variables  
|   ├─ var  
|   ├─ let  
|   └─ const
```

```
|
├─ Data Types
|   ├─ String
|   ├─ Number
|   ├─ Boolean
|   ├─ Undefined
|   ├─ Null
|   ├─ Array
|   └─ Object
|
├─ Operators
|   ├─ +
|   ├─ -
|   ├─ *
|   ├─ /
|   ├─ %
|   ├─ ==
|   └─ ===
|
├─ Control
|   ├─ if
|   ├─ switch
|   ├─ for
|   └─ while
|
├─ Function
|   ├─ function
|   ├─ return
|   └─ =>
|
├─ DOM
|   ├─ getElementById()
|   ├─ querySelector()
|   ├─ querySelectorAll()
|   ├─ textContent
|   ├─ innerHTML
|   └─ classList
|
├─ Event
|   └─ addEventListener()
|
```

```
└─ Array
  │ └─ push
  │ └─ pop
  │ └─ map
  │ └─ filter
  │ └─ find
  │ └─ forEach
  │
└─ Data
  │ └─ JSON.stringify()
  │ └─ JSON.parse()
  │ └─ localStorage
  │
└─ AI
  │ └─ Understand
  │ └─ Test
  │ └─ Security
  │ └─ Performance
  │ └─ Compatibility
```

Phase 04 | 自我檢測標準

完成 **181~250** 題後：

得分	評價	建議
63-70	🟢 優秀	可以進入網路與 Internet
56-62	🟢 良好	加強程式碼閱讀
49-55	🟡 普通	重新複習 JS 基礎
42-48	🟠 偏弱	加強變數、型別、DOM
<42	🔴 需要重訓	先不要急著進下一階段

特別注意

如果你：

- HTML 很好

- CSS 很好
- 但是 JavaScript 程式閱讀題錯很多

不要只背答案。

你真正需要補的是：

「執行流程思維」

因為從 Phase 04 開始，題目會逐漸從：

「你知道什麼？」

轉成：

「你能不能看懂程式正在做什麼？」

Phase 04 最終學習成果

完成本階段後，你應該可以獨立回答：

基礎

- `var`、`let`、`const` 有什麼差別？
- `String`、`Number`、`Boolean` 是什麼？
- `Array` 為什麼從 0 開始？
- `Object` 如何存取？

程式判讀

- `==` 與 `===` ？
- `5 + "5"` 結果？
- `for` 會跑幾次？
- `if/else` 最後執行哪一段？

DOM

- 如何取得 HTML ?
- `querySelector()` 與 `querySelectorAll()` ?
- `textContent` 與 `innerHTML` ?
- 如何新增／移除 class ?

事件

- `click` 是什麼 ?
- `addEventListener()` 做什麼 ?

資料

- `map()` ?
- `filter()` ?
- `find()` ?
- `JSON.stringify / parse` ?
- `localStorage` ?

AI

最重要的是：

AI 可以幫你寫 *JavaScript*，但你必須有能力判斷 AI 寫的 *JavaScript* 是否正確、安全、合理。

這正是「**前端網頁技術+AI 應用**」與單純傳統網頁設計甄試最大的差異之一。

下一階段

Phase 05 | 網路與 Internet 課程 + 第 251~320 題

這一階段會開始建立前端工程師非常重要的「**瀏覽器 → DNS → IP → Server → HTTP → HTTPS → Response → Browser Render**」完整觀念，並大量訓練：

「當使用者在瀏覽器輸入 `https://example.com` 後，到網頁真正出現在螢幕上，中間到底發生了什麼？」

同時會完整掌握 **HTTP Methods**、**200/301/302/400/401/403/404/500**、**DNS**、**IPv4/IPv6**、**Port**、**TCP/IP**、**TLS**、**Cookie**、**Session**、**Cache**、**CDN**、**Hosting**、**Web Server**。

2026/8/18 下午1:21:22



Phase 05 | 網路與 Internet 課程 + 第 251~320 題

前端網頁技術與 AI 應用甄試 | 70 題完整訓練教材

本階段核心目標

把「我會做網頁」提升到「我知道網頁到底是怎麼運作的」。

甄試中這一區非常容易出現觀念辨識題、情境題、流程題與陷阱題。尤其要熟悉：

URL → DNS → IP → TCP/TLS → HTTP → Web Server → Response → Browser Render

Part 1 | Internet 基礎

Lesson 01 | Internet 是什麼？

Internet：

全球互相連接的電腦網路系統。

它不是單一一台電腦，也不是單一家公司所擁有。

可以簡化理解：

```
graph TD; A[你的電腦] --> B[網路]; B --> C[Internet]; C --> D[Web Server];
```

Lesson 02 | WWW 是什麼？

WWW：

World Wide Web，全球資訊網。

它是建立在 Internet 之上的服務之一。

因此：

Internet ≠ WWW

Internet 是整體網路基礎；WWW 是利用網路提供網頁等資訊服務的系統。

Lesson 03 | Internet vs WWW

項目	Internet	WWW
意義	全球網路基礎	全球資訊網
範圍	較廣	Internet 上的一種服務
主要技術	TCP/IP 等	HTTP/HTTPS、HTML 等
網頁	不等於網頁	主要與網頁相關

甄試陷阱

「Internet 就等於 WWW」

✗ 錯誤。

Lesson 04 | Client / Server

網站基本架構：

```
Client
  ↓ Request
Server
  ↓ Response
Client
```

Client 通常指：

使用者端的瀏覽器或裝置。

Server：

提供服務或資料的電腦／系統。

Lesson 05 | Browser

常見瀏覽器：

- Chrome
- Edge
- Firefox
- Safari

瀏覽器主要負責：

向 **Web Server** 發送請求、接收資料並呈現網頁。

Lesson 06 | Web Server

Web Server：

接收 *HTTP/HTTPS Request*，並提供相對應資源或服務。

例如：

```
Browser
  ↓
GET /index.html
  ↓
Web Server
  ↓
index.html
  ↓
Browser
```

Lesson 07 | HTTP

HTTP：

HyperText Transfer Protocol

中文：

超文字傳輸協定。

它是 Web 中 Client 與 Server 溝通的重要協定。

Lesson 08 | HTTPS

HTTPS：

HTTP + TLS 加密保護

主要目的：

- 保護傳輸資料
- 驗證網站身分
- 降低資料遭竄改或竊聽的風險

網址通常：

```
https://
```

Lesson 09 | HTTP vs HTTPS

HTTP

未提供 TLS 加密保護

資料保護較弱

```
http://
```

HTTPS

使用 TLS

傳輸安全性較高

```
https://
```

必背：

HTTPS 不是另一種完全不同的 HTTP，而是 HTTP 搭配 TLS 保護。

Lesson 10 | HTTP Request

瀏覽器向 Server 發出：

Request (請求)

例如：

```
GET /index.html HTTP/1.1
```

Lesson 11 | HTTP Response

Server 回覆：

Response (回應)

例如：

```
HTTP/1.1 200 OK
```

以及：

```
HTML  
CSS  
JavaScript  
圖片  
JSON
```

Lesson 12 | HTTP Request / Response

最重要的流程：

```
Browser  
|  
| HTTP Request  
↓  
Web Server  
|  
| HTTP Response
```




Browser

甄試看到：

「Client 向 Server 要求網頁」

通常就是：

Request

看到：

「Server 將網頁資料傳回」

通常就是：

Response

Part 2 | HTTP Methods

Lesson 13 | GET

GET：

主要用於取得資源。

例如：

```
GET /products.html
```

常見特性：

- 取得資料
- 參數常可出現在 URL
- 不適合用來傳送敏感資料

Lesson 14 | POST

POST：

通常用於提交資料給 *Server*。

例如：

登入
註冊
表單提交
建立資料

例如：

POST /login

重要陷阱

POST 本身不等於「加密」。

是否安全主要還要看：

HTTPS / TLS

Lesson 15 | PUT

PUT：

常用於建立或完整更新指定資源。

甄試只需要先掌握：

GET → 取得
POST → 提交 / 建立
PUT → 更新 / 替換
DELETE → 刪除

Lesson 16 | DELETE

DELETE：

用於刪除指定資源。

例如：

```
DELETE /users/123
```

Lesson 17 | 四大 HTTP Method

超高頻記憶：

GET

↓

拿資料

POST

↓

送資料

PUT

↓

更新資料

DELETE

↓

刪除資料

Part 3 | HTTP Status Code

Lesson 18 | Status Code

HTTP Response 通常包含：

Status Code (狀態碼)

例如：

```
200
404
500
```

Lesson 19 | 2xx

代表：

成功

最重要：

```
200 OK
```

Lesson 20 | 200 OK

`200 OK`：

Request 成功。

例如：

```
Browser
  ↓
GET /index.html
  ↓
Server
  ↓
200 OK
```

Lesson 21 | 3xx

代表：

重新導向或其他重新處理

常見：

301

302

Lesson 22 | 301

301 Moved Permanently

表示：

資源永久移動到新的 URL。

SEO 中非常重要。

例如：

old-page.html

↓

301

↓

new-page.html

Lesson 23 | 302

302：

暫時重新導向。

因此：

301 → Permanent

302 → Temporary

必背。

Lesson 24 | 4xx

代表：

Client 端 Request 有問題或無法被授權處理。

常見：

400
401
403
404

Lesson 25 | 400

400 Bad Request

表示：

Request 格式或內容有問題。

Lesson 26 | 401

401 Unauthorized

通常表示：

需要身分驗證或目前沒有通過驗證。

例如：

需要登入

Lesson 27 | 403

403 Forbidden

表示：

Server 理解 *Request*，但拒絕提供存取權。

401 vs 403

401

↓

身分驗證問題

403

↓

拒絕存取

Lesson 28 | 404

404 Not Found

表示：

找不到要求的資源。

例如：

`https://example.com/about.html`

但 Server 找不到：

`about.html`

Lesson 29 | 500

500 Internal Server Error

表示：

Server 發生內部錯誤。

必背：

404 → 找不到資源

500 → Server 內部錯誤

Lesson 30 | HTTP Status Code 快速記憶

1xx → 資訊
2xx → 成功
3xx → 重新導向
4xx → Client 端問題
5xx → Server 端問題

最重要：

200 → OK
301 → 永久重新導向
302 → 暫時重新導向
400 → Bad Request
401 → 需要驗證
403 → Forbidden
404 → Not Found
500 → Server Error

Part 4 | URL、Domain、DNS、IP

Lesson 31 | URL

URL：

Uniform Resource Locator

例如：

`https://www.example.com/products/index.html`

可以拆成：

https
↓
Protocol

www.example.com
↓
Domain / Host


```
/products/index.html
```

↓

Path

Lesson 32 | Domain Name

Domain :

給人容易閱讀與記憶的名稱。

例如：

```
example.com
```

```
google.com
```

人比較容易記：

```
example.com
```

而不是：

```
142.250.xxx.xxx
```

Lesson 33 | IP Address

IP Address :

網路上的位址，用於識別網路中的主機或介面。

例如 IPv4：

```
192.168.1.1
```

Lesson 34 | IPv4

IPv4 使用：

32-bit

常見形式：

192.168.1.1

由四組數字組成。

Lesson 35 | IPv6

IPv6 使用：

128-bit

例如：

2001:db8::1

主要原因之一：

IPv4 位址數量有限，因此發展 IPv6。

Lesson 36 | DNS

DNS：

Domain Name System

主要作用：

將 Domain Name 對應到 IP Address。

例如：

www.example.com

↓

DNS

↓
93.xxx.xxx.xxx

超高頻考點：

DNS 就像 Internet 的「電話簿」。

Lesson 37 | DNS 不是 Web Server

注意：

DNS
↓
負責名稱解析

Web Server
↓
負責提供 Web 資源

兩者角色不同。

Part 5 | Port、TCP/IP

Lesson 38 | Port

Port：

用來區分同一台主機上的不同網路服務。

例如常見：

HTTP → 80
HTTPS → 443

Lesson 39 | HTTP 80 / HTTPS 443

必背：

HTTP

↓

Port 80

HTTPS

↓

Port 443

Lesson 40 | TCP/IP

TCP/IP：

Internet 通訊的重要協定組合。

甄試初階理解：

IP

↓

負責位址與封包傳遞

TCP

↓

提供可靠的傳輸機制

Lesson 41 | TCP

TCP 的重要特性：

可靠、具連線導向的傳輸。

可以協助：

- 確認資料是否收到
- 維持資料順序
- 重傳遺失資料

Lesson 42 | IP

IP 主要負責：

定址與封包路由。

簡單記：

IP

↓

去哪裡？

TCP

↓

怎麼可靠地送？

Part 6 | SSL / TLS / HTTPS

Lesson 43 | TLS

TLS：

Transport Layer Security

用於提供網路傳輸安全保護。

HTTPS 通常就是：

HTTP

+

TLS

Lesson 44 | HTTPS 可以提供什麼？

主要可以提供：

① 機密性

降低傳輸內容被竊聽的風險。

② 完整性

降低資料被竄改的風險。

③ 身分驗證

透過憑證等機制協助驗證網站身分。

Lesson 45 | HTTPS ≠ 網站一定安全

這是非常重要的陷阱。

HTTPS：

保護傳輸通道。

但如果網站本身有：

- XSS
- SQL Injection
- 弱密碼
- 錯誤權限設定
- 惡意內容

仍然可能不安全。

Part 7 | Cookie、Session、Cache

Lesson 46 | Cookie

Cookie：

瀏覽器可以儲存的小型資料。

常見用途：

- 登入狀態
- 使用者偏好

- Session Identifier
- 追蹤資訊

Lesson 47 | Session

Session :

Server 端維持使用者狀態的一種機制。

簡化：

```
Browser
  ↓
Cookie : Session ID
  ↓
Server
  ↓
Session Data
```

Lesson 48 | Cookie vs Session

Cookie

常儲存在 Client

瀏覽器會保存 Cookie

可攜帶識別資訊

Session

通常由 Server 維護狀態

Server 保存 Session 資訊

可保存使用者狀態

Lesson 49 | Cache

Cache :

暫存已取得的資源，以便未來更快取得。

例如：

第一次

Browser → Server → CSS

第二次

Browser → Cache

可以減少：

網路傳輸與等待時間。

Lesson 50 | Cookie ≠ Cache

這是常見陷阱。

Cookie

↓

資料 / 狀態

Cache

↓

暫存資源，加快載入

用途不同。

Part 8 | CDN、Hosting、Web Server

Lesson 51 | CDN

CDN：

Content Delivery Network

將內容分散到不同地理位置的節點。

目的：

讓使用者從較接近自己的節點取得內容。

Lesson 52 | CDN 適合什麼？

例如：

- 圖片
- CSS
- JavaScript
- 影片
- 靜態檔案

可以降低：

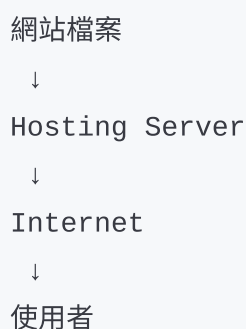
使用者與內容伺服器之間的距離與延遲。

Lesson 53 | Hosting

Web Hosting：

提供網站所需的伺服器與網路環境。

例如：



```
graph TD; A[網站檔案] --> B[Hosting Server]; B --> C[Internet]; C --> D[使用者];
```

Lesson 54 | Web Server 軟體

常見：

- Apache
- Nginx
- IIS

它們可以負責：

接收 HTTP Request、處理請求並回應資源。

Part 9 | 最重要流程：輸入網址後發生什麼？

Lesson 55 | 完整流程

假設使用者輸入：

```
https://www.example.com
```

大致可以理解為：

- ① 輸入 URL
- ↓
- ② Browser 判斷 URL
- ↓
- ③ DNS 查詢 Domain 對應的 IP
- ↓
- ④ 建立網路連線
- ↓
- ⑤ HTTPS 情況下進行 TLS 建立
- ↓
- ⑥ Browser 發送 HTTP Request
- ↓
- ⑦ Web Server 接收 Request
- ↓
- ⑧ Server 回傳 HTTP Response
- ↓
- ⑨ Browser 取得 HTML
- ↓
- ⑩ Browser 再取得 CSS / JS / Images
- ↓
- ⑪ Browser 解析並渲染
- ↓
- ⑫ 使用者看到網頁

這是本階段最重要的流程題。

Lesson 56 | 第一次輸入網址

例如：

```
https://www.tayulee.com
```

第一個重要問題：

`tayulee.com` 對應哪個 IP？

這時候會涉及：

DNS

Lesson 57 | DNS 查詢

概念上：

```
Domain
  ↓
DNS
  ↓
IP Address
```

例如：

```
www.example.com
  ↓
DNS
  ↓
93.xxx.xxx.xxx
```

Lesson 58 | 建立連線

取得 IP 後，Client 才能與對應的 Server 建立網路通訊。

HTTPS 情況下，還需要：

TLS 安全連線建立。

Lesson 59 | Browser 發送 Request

例如：

```
GET / HTTP/1.1
Host: www.example.com
```

Server 收到後：

判斷要提供什麼資源。

Lesson 60 | Server Response

例如：

```
HTTP/1.1 200 OK
Content-Type: text/html
```

後面可能包含：

```
<!DOCTYPE html>
<html>
...
</html>
```

Lesson 61 | Browser Render

瀏覽器取得 HTML 後：

```
HTML
↓
DOM
```

取得 CSS：

```
CSS
↓
CSSOM
```

最後瀏覽器會進行：

Render / Layout / Paint 等工作

將網頁呈現在螢幕上。

Part 10 | 第 251~320 題

第 251~270 題 | Internet / HTTP

第 251 題 ★

Internet 是：

- A. 全球互相連接的網路系統
- B. 一種 CSS
- C. 一種瀏覽器
- D. 一種圖片格式

答案：A

第 252 題 ★

WWW 的全名：

- A. World Wide Web
- B. World Web Window
- C. Web Wide World
- D. Wide World Website

答案：A

第 253 題 ★★

以下何者正確？

- A. Internet 與 WWW 完全相同
- B. WWW 是 Internet 上的重要服務之一
- C. Internet 是一種瀏覽器
- D. WWW 是 CPU

答案：B

第 254 題 ★

Client 通常是：

- A. 使用服務的一方
- B. DNS
- C. 網路線
- D. 硬碟

答案：A

第 255 題 ★

Server 主要是：

- A. 提供服務或資源的一方
- B. CSS Selector
- C. 圖片格式
- D. HTML Tag

答案：A

第 256 題 ★

瀏覽器通常屬於：

- A. Client
- B. Server
- C. DNS
- D. Router only

答案：A

第 257 題 ★

HTTP 全名：

- A. HyperText Transfer Protocol
- B. High Transfer Text Program

- C. Hyperlink Text Process
- D. Host Transfer Protocol

答案：A

第 258 題 ★

HTTP 主要用途：

- A. Client 與 Server 進行 Web 通訊
- B. 控制 CPU 溫度
- C. 編輯圖片
- D. 壓縮硬碟

答案：A

第 259 題 ★★

瀏覽器向 Server 要求網頁時稱為：

- A. Request
- B. Response
- C. Cache
- D. Cookie

答案：A

第 260 題 ★★

Server 回傳資料給 Browser 稱為：

- A. Request
- B. Response
- C. Query only
- D. DNS

答案：B

第 261 題 ★★

以下何者通常用來取得資源？

- A. GET
- B. DELETE
- C. CSS
- D. DNS

答案：A

第 262 題 ★★

以下何者通常用來提交資料？

- A. POST
- B. GET only
- C. DNS
- D. TCP

答案：A

第 263 題 ★★

以下何者通常與刪除資源有關？

- A. GET
- B. POST
- C. DELETE
- D. DNS

答案：C

第 264 題 ★★★

下列配對何者正確？

- A. GET → 取得
- B. DELETE → 查詢 DNS
- C. POST → CSS
- D. PUT → 建立瀏覽器

答案：A

第 265 題 ★★★

HTTP 的 POST 本身是否代表資料一定經過加密？

- A. 是
- B. 否
- C. 只有 Chrome 才是
- D. 只有手機才是

答案：B

解析：

是否加密主要取決於：

HTTPS / TLS

第 266 題 ★★

200 OK 表示：

- A. 成功
- B. 找不到
- C. Server 錯誤
- D. 永久轉址

答案：A

第 267 題 ★★

404 Not Found 表示：

- A. 找不到要求的資源
- B. 成功
- C. DNS 一定故障
- D. HTTPS 成功

答案：A

第 268 題 ★★

500 Internal Server Error 通常表示：

- A. Server 端發生內部錯誤
- B. 找不到網址

- C. Client 一定離線
- D. CSS 錯誤

答案：A

第 269 題 ★★★

301 最主要表示：

- A. 暫時重新導向
- B. 永久重新導向
- C. Server 錯誤
- D. 找不到資源

答案：B

第 270 題 ★★★

302 最主要表示：

- A. 永久重新導向
- B. 暫時重新導向
- C. Server Error
- D. Not Found

答案：B

第 271~290 題 | Status Code / URL / DNS

第 271 題 ★★

HTTP Status Code 2xx 通常代表：

- A. 成功
- B. Client Error
- C. Server Error
- D. Redirect only

答案：A

第 272 題 ★★

HTTP Status Code **3xx** 通常與：

- A. Redirect
- B. CPU
- C. 密碼
- D. CSS

有關。

答案：A

第 273 題 ★★

HTTP Status Code **4xx** 通常表示：

- A. Client Request / Authorization 等方面的問題
- B. 一定是 Server 硬體故障
- C. 成功
- D. DNS 一定正常

答案：A

第 274 題 ★★

HTTP Status Code **5xx** 通常表示：

- A. Server 端錯誤
- B. Client 一定正常
- C. CSS 錯誤
- D. URL 一定正確

答案：A

第 275 題 ★★★

401 Unauthorized 常與什麼有關？

- A. 身分驗證
- B. CSS
- C. DNS
- D. 圖片格式

答案：A

第 276 題 ★★★

403 Forbidden 表示：

- A. Server 拒絕存取
- B. 資源一定不存在
- C. Request 成功
- D. 永久轉址

答案：A

第 277 題 ★★★

401 與 403 的概念差異：

- A. 401 通常涉及驗證；403 表示拒絕存取
- B. 兩者完全相同
- C. 401 是 Server Error
- D. 403 是永久轉址

答案：A

第 278 題 ★★

URL 全名：

- A. Uniform Resource Locator
- B. Universal Router Link
- C. Uniform Router Location
- D. User Resource Link

答案：A

第 279 題 ★★

以下：

```
https://example.com/about/index.html
```

其中 `https` 是：

- A. Protocol
- B. Domain
- C. Path
- D. File extension

答案：A

第 280 题 ★★

以下：

```
https://example.com/about/index.html
```

其中 `example.com` 是：

- A. Domain / Host
- B. Protocol
- C. Path
- D. Port only

答案：A

第 281 题 ★★

以下：

```
https://example.com/about/index.html
```

其中 `/about/index.html` 主要是：

- A. Path
- B. Protocol
- C. IP
- D. DNS

答案：A

第 282 题 ★★

DNS 全名：

- A. Domain Name System
- B. Domain Network Server
- C. Digital Name System
- D. Data Name Service

答案：A

第 283 題 ★

DNS 主要作用：

- A. Domain Name 對應 IP
- B. 建立 HTML
- C. 編輯 CSS
- D. 加密硬碟

答案：A

第 284 題 ★★

以下哪個最符合 DNS 的用途？

- A. example.com → IP Address
- B. HTML → CSS
- C. JPG → PNG
- D. RAM → SSD

答案：A

第 285 題 ★★★

為什麼人類需要 Domain Name？

- A. 比直接記 IP 容易
- B. 因為 IP 不存在
- C. 因為瀏覽器不能使用 IP
- D. 因為 DNS 是瀏覽器

答案：A

第 286 題 ★★

IPv4 使用：

- A. 32-bit
- B. 64-bit
- C. 128-bit
- D. 256-bit

答案：A

第 287 題 ★★

IPv6 使用：

- A. 32-bit
- B. 64-bit
- C. 128-bit
- D. 256-bit

答案：C

第 288 題 ★★★

IPv6 發展的重要原因之一：

- A. IPv4 位址空間有限
- B. HTML 太慢
- C. CSS 太大
- D. CPU 太快

答案：A

第 289 題 ★★

以下哪一個是 IPv4 格式？

- A. 192.168.1.1
- B. 2001:db8::1
- C. www.example.com
- D. https://example.com

答案：A

第 290 題 ★★

以下哪一個較符合 IPv6 格式？

- A. 192.168.0.1
- B. 2001:db8::1
- C. example.com
- D. 443

答案：B

第 291~305 題 | Port / TCP/IP / HTTPS

第 291 題 ★★

HTTP 常見 Port：

- A. 21
- B. 25
- C. 80
- D. 443

答案：C

第 292 題 ★★

HTTPS 常見 Port：

- A. 80
- B. 110
- C. 443
- D. 8080 only

答案：C

第 293 題 ★★

Port 的主要用途：

- A. 區分主機上的不同網路服務
- B. 儲存圖片

- C. 產生 HTML
- D. 取代 DNS

答案：A

第 294 題 ★★★

下列配對正確的是：

- A. HTTP → 80
- B. HTTPS → 80 only
- C. DNS → 443
- D. CSS → 80

答案：A

第 295 題 ★★★

TCP 的重要特性：

- A. 提供可靠的傳輸機制
- B. 負責 HTML 排版
- C. 負責 DNS 名稱本身
- D. 負責圖片壓縮

答案：A

第 296 題 ★★★

IP 主要與什麼有關？

- A. 定址與封包路由
- B. CSS Selector
- C. HTML Tag
- D. Cookie

答案：A

第 297 題 ★★★

下列哪個最適合描述 TCP/IP？

- A. Internet 通訊的重要協定組合
- B. 一種 CSS Framework
- C. 一種圖片格式
- D. 一種瀏覽器

答案：A

第 298 題 ★★

TLS 的全名：

- A. Transport Layer Security
- B. Transfer Link System
- C. Text Layer Security
- D. Transport Login Service

答案：A

第 299 題 ★★★

HTTPS 通常可以理解為：

- A. HTTP + TLS
- B. HTTP + CSS
- C. HTTP + HTML
- D. HTTP + DNS

答案：A

第 300 題 ★★★

HTTPS 的主要目的之一：

- A. 保護資料傳輸
- B. 讓 HTML 自動產生
- C. 取代 JavaScript
- D. 取代 DNS

答案：A

第 301 題 ★★★★★

HTTPS 可以協助提供：

- A. 機密性、完整性、身分驗證
- B. CPU 超頻
- C. RAM 擴充
- D. HTML 排版

答案：A

第 302 題 ★★★★★

「網站使用 HTTPS，所以網站絕對沒有任何安全漏洞。」

這句話：

- A. 正確
- B. 錯誤
- C. 只有手機正確
- D. 只有 Chrome 正確

答案：B

解析：

HTTPS 主要保護：

傳輸通道。

並不能保證網站程式沒有：

- XSS
 - SQL Injection
 - 弱密碼
 - 權限漏洞
-

第 303 題 ★★★

SSL 與現代 HTTPS 安全通訊最相關的技術：

- A. TLS
- B. HTML

- C. CSS
- D. DOM

答案：A

第 304 題 ★★★★★

如果使用者在公共 Wi-Fi 上登入網站，下列哪一項通常可以提高傳輸安全性？

- A. 使用 HTTPS
- B. 使用 HTTP
- C. 關閉瀏覽器
- D. 使用 `<div>`

答案：A

第 305 題 ★★★★★

下列何者最正確？

- A. HTTPS 可以降低傳輸被竊聽或竄改的風險
- B. HTTPS 可以保證 Server 沒有 Bug
- C. HTTPS 可以防止所有惡意程式
- D. HTTPS 可以取代防火牆

答案：A

第 306~320 題 | Cookie / Cache / CDN / 完整流程

第 306 題 ★★

Cookie 通常是：

- A. 瀏覽器可保存的小型資料
- B. CPU
- C. CSS
- D. DNS Server

答案：A

第 307 題 ★★

Cookie 常見用途：

- A. 儲存登入或偏好等資訊
- B. 增加 CPU 核心
- C. 取代 HTML
- D. 建立 SSD

答案：A

第 308 題 ★★★

Session 通常主要用於：

- A. 維持使用者狀態
- B. 修改 CSS
- C. 壓縮圖片
- D. DNS 查詢

答案：A

第 309 題 ★★★

以下何者較正確？

- A. Session 通常由 Server 維護狀態
- B. Session 就是 CSS
- C. Session 是 DNS
- D. Session 一定儲存在圖片中

答案：A

第 310 題 ★★★

Cache 的主要目的：

- A. 暫存資源以加快後續存取
- B. 建立 DNS
- C. 取代 HTML
- D. 建立 CPU

答案：A

第 311 題 ★★★

以下何者正確？

- A. Cookie 與 Cache 用途完全相同
- B. Cookie 偏向資料／狀態；Cache 偏向暫存資源
- C. Cache 就是 Cookie
- D. Cookie 就是 DNS

答案：B

第 312 題 ★★★

CDN 全名：

- A. Content Delivery Network
- B. Computer Data Network
- C. Content Domain Name
- D. Central Delivery Node

答案：A

第 313 題 ★★★

CDN 的主要優點之一：

- A. 讓內容可從較接近使用者的節點提供
- B. 取代 HTML
- C. 取代 JavaScript
- D. 取代 DNS

答案：A

第 314 題 ★★★

以下哪一項特別適合透過 CDN 提供？

- A. 圖片、CSS、JavaScript 等靜態資源
- B. CPU
- C. RAM
- D. 主機板

答案：A

第 315 題 ★★★

Hosting 主要提供：

- A. 網站所需的伺服器與網路環境
- B. HTML 標籤
- C. CSS Selector
- D. JavaScript Function

答案：A

第 316 題 ★★★

以下何者是常見 Web Server 軟體？

- A. Nginx
- B. Photoshop
- C. Excel
- D. PowerPoint

答案：A

第 317 題 ★★★★★

使用者輸入：

```
https://example.com
```

下列哪個步驟通常會先涉及將 Domain 對應到 IP？

- A. DNS
- B. CSS
- C. DOM
- D. JSON

答案：A

第 318 題 ★★★★★

完整流程中，DNS 的位置最適合描述為：

Domain

↓

?

↓

IP

- A. DNS
- B. CSS
- C. HTML
- D. Cookie

答案：A

第 319 題 ★★★★★☆

使用者輸入：

https://example.com

下列流程何者最合理？

A.

URL

- DNS
- IP
- HTTPS/TLS
- HTTP Request
- Server
- Response
- Browser Render

B.

URL

- CSS
- CPU
- DNS
- HTML

C.

URL

→ Photoshop

→ DNS

→ RAM

D.

URL

→ HTML

→ DNS

→ SSD

答案：A

第 320 題 ★★★★★ | 超級綜合題

小王在瀏覽器輸入：

https://www.example.com/index.html

下列哪個流程最合理？

A.

Browser

↓

DNS 將 Domain 解析成 IP

↓

建立網路連線 / HTTPS 的 TLS 保護

↓

Browser 發送 HTTP Request

↓

Web Server 處理

↓

Server 回傳 HTTP Response

↓

Browser 取得 HTML

↓

再取得 CSS、JavaScript、圖片等資源

↓
Browser 解析與渲染
↓
使用者看到網頁

B.

Browser
↓
CSS
↓
DNS
↓
CPU
↓
使用者看到網頁

C.

Browser
↓
HTML
↓
直接變成 IP
↓
RAM

D.

Browser
↓
Server
↓
DNS
↓
CSS

答案：A

解析

這是 Phase 05 最重要的綜合觀念。

可以把它背成：

「網址 → 找地址 → 建連線 → 發請求 → *Server* 回應 → *Browser* 下載資源 → *Render*」

也就是：

```
URL
↓
DNS
↓
IP
↓
TCP / TLS
↓
HTTP Request
↓
Web Server
↓
HTTP Response
↓
HTML / CSS / JS / Images
↓
Browser
↓
Render
```

Part 11 | 70 題高頻考點總整理

HTTP

```
GET
→ 取得

POST
→ 提交

PUT
→ 更新
```

DELETE

→ 刪除

Status Code

200

→ OK

301

→ 永久重新導向

302

→ 暫時重新導向

400

→ Bad Request

401

→ Authentication

403

→ Forbidden

404

→ Not Found

500

→ Server Error

Internet

Domain

↓

DNS

↓

IP

Protocol

HTTP

→ 80

HTTPS

→ 443

Security

HTTPS

→ HTTP + TLS

Client / Server

Client

↓ Request

Server

↓ Response

Client

Cookie / Session / Cache

Cookie

→ Client 資料 / 狀態

Session

→ Server 端狀態

Cache

→ 暫存資源

CDN

使用者

↓

較接近的 CDN 節點

↓

內容

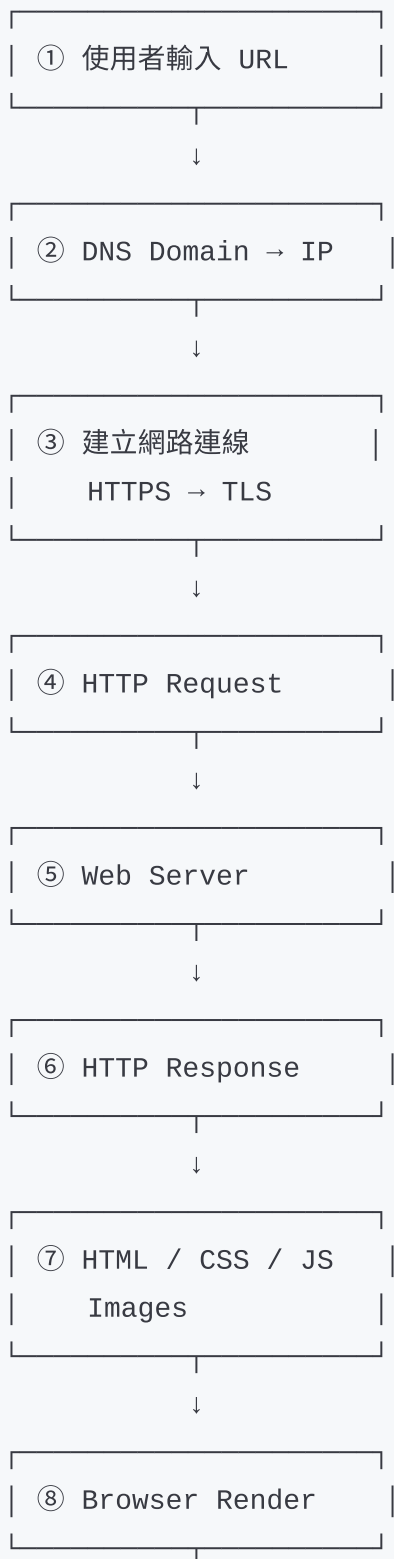


Part 12 | 「輸入網址後發生什麼？」考試版

如果甄試考：

「使用者在瀏覽器輸入網址後，發生什麼事情？」

你腦中直接跑這張圖：



↓

⑨ 使用者看到網頁

Phase 05 | 考前 1 分鐘速記

如果考試時間只剩 1 分鐘，優先背這 20 個答案：

- 01 Internet = 全球互聯網路
- 02 WWW = World Wide Web
- 03 Browser = Client
- 04 Web Server = 提供 Web 資源
- 05 HTTP = Web 通訊協定
- 06 HTTPS = HTTP + TLS
- 07 Request = Client → Server
- 08 Response = Server → Client
- 09 GET = 取得
- 10 POST = 提交
- 11 PUT = 更新
- 12 DELETE = 刪除
- 13 200 = OK
- 14 301 = 永久轉址
- 15 302 = 暫時轉址
- 16 404 = 找不到
- 17 500 = Server Error
- 18 DNS = Domain → IP
- 19 HTTP 80 / HTTPS 443
- 20 Cookie ≠ Cache

Phase 05 | 自我檢測標準

完成 251~320 題後：

得分	評價	建議
63-70	● 優秀	進入電腦與作業系統
56-62	● 良好	加強 HTTP / DNS

得分	評價	建議
49-55	● 普通	重新複習 Status Code
42-48	● 偏弱	重新建立 Internet 基礎
<42	● 需要重訓	先熟記核心名詞再進階

特別提醒

Phase 05 有三組一定要做到**看到題目就能立即反應**：

① DNS → Domain → IP

② GET → 取得 / POST → 提交 / PUT → 更新 / DELETE → 刪除

③ 200 / 301 / 302 / 400 / 401 / 403 / 404 / 500

如果這三組熟練，這 70 題的基礎分數會非常穩。

Phase 05 → Phase 06

下一階段：

Phase 06 | 電腦與作業系統課程＋第 321～370 題

將從「網頁是如何透過 Internet 運作」進一步建立「**電腦本身是如何運作**」的觀念。

核心會包括：

CPU → RAM → SSD/HDD → GPU → 主機板 → OS → Process → Browser → Driver → File System → 32/64-bit → 二進位 → KB/MB/GB/TB → Virtual Memory → 基礎故障排除

並特別加入甄試很容易考的：

「看到電腦症狀 → 判斷最可能是哪個硬體、軟體或作業系統問題」

情境題。

2026/8/18 下午1:58:02



Phase 06 | 電腦與作業系統課程＋第 321～370 題

前端網頁技術與 AI 應用甄試 | 50 題完整訓練教材

本階段核心目標

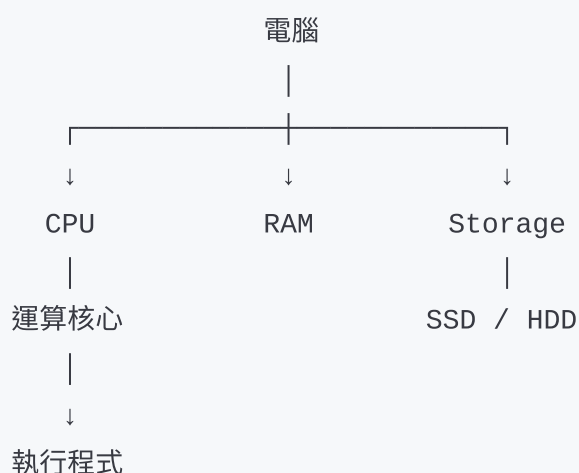
建立「電腦到底是怎麼運作的」基本觀念。

前端網頁設計師雖然不一定要成為硬體工程師，但甄試很常從 **CPU**、**RAM**、**SSD**、**OS**、檔案、瀏覽器、**Process**、**Driver**、儲存單位、**32/64-bit**、故障排除出題。

Part 1 | Computer 基礎架構

Lesson 01 | 電腦的基本組成

可以先把電腦想成：



常見重要元件：

- CPU
- RAM
- SSD / HDD
- GPU
- 主機板
- 電源供應器
- 網路介面

- 作業系統

Lesson 02 | CPU

CPU：

Central Processing Unit，中央處理器。

可以把它想成：

電腦主要的運算與指令執行中心。

例如：

```
graph TD; A[程式] --> B[CPU]; B --> C[執行指令]; C --> D[產生結果];
```

Lesson 03 | CPU 核心

CPU Core：

CPU 中可以執行指令的處理單元。

一般而言：

單核心
雙核心
四核心
六核心
八核心

核心數增加通常有助於：

同時處理更多工作。

但：

核心數越多 $\neq$ 所有程式一定快相同倍數。

這是常見陷阱。

Lesson 04 | CPU Clock Speed

CPU 時脈常以：

GHz

表示。

例如：

3.5 GHz

4.2 GHz

一般來說，時脈越高代表：

單位時間可以進行更多時脈週期。

但電腦效能不能只看 GHz。

還受到：

- CPU 架構
- 核心數
- Cache
- 指令集
- 工作負載

等因素影響。

Lesson 05 | RAM

RAM：

Random Access Memory

中文：

隨機存取記憶體。

它主要用於：

暫時儲存目前正在使用的程式與資料。

例如：

Chrome
Photoshop
VS Code
Word

執行時都會使用 RAM。

Lesson 06 | RAM 的特性

RAM 是：

揮發性記憶體 (Volatile Memory)

意思是：

電源關閉後，RAM 中的資料通常會消失。

因此：

RAM
↓
暫時工作空間

而：

SSD / HDD

↓

長期儲存

Lesson 07 | RAM 不夠會怎樣？

例如同時開：

Chrome
Photoshop
VS Code
Teams
大量分頁

RAM 不足時可能：

- 系統變慢
- 程式切換變慢
- 使用虛擬記憶體
- 磁碟 I/O 增加

因此：

RAM 主要影響多工能力與程式執行時的工作空間。

Lesson 08 | ROM

ROM：

Read-Only Memory

傳統概念上：

主要用於儲存不容易被修改的資料。

甄試通常要能區分：

RAM → 工作中的暫存資料

ROM → 非揮發性、傳統上偏向唯讀的記憶體

Part 2 | Storage

Lesson 09 | SSD

SSD：

Solid State Drive

使用：

Flash Memory 等非揮發性儲存技術。

主要特色：

- 速度快
- 無傳統機械磁碟的旋轉結構
- 抗震性通常較佳
- 適合當作系統碟

Lesson 10 | HDD

HDD：

Hard Disk Drive

傳統硬碟使用：

磁性碟片與機械讀寫結構。

特色：

- 容量通常較大且每 GB 成本較低
- 有機械結構

- 通常比 SSD 慢

Lesson 11 | SSD vs HDD

項目	SSD	HDD
技術	Flash	磁碟
機械結構	無傳統旋轉碟片	有
速度	通常較快	通常較慢
抗震	通常較佳	較容易受機械結構影響
噪音	幾乎無機械噪音	有機械運轉聲

甄試必背

SSD 快、HDD 有機械結構。

Lesson 12 | SSD 不是 RAM

非常重要。

RAM



暫時工作空間

SSD



長期儲存

即使關機：

SSD 中的資料仍然存在。

Part 3 | GPU / 主機板

Lesson 13 | GPU

GPU：

Graphics Processing Unit

中文：

圖形處理器。

主要負責：

- 圖形運算
- 影像處理
- 遊戲圖形
- 影片相關運算
- 部分平行運算
- AI 運算

Lesson 14 | CPU vs GPU

簡化：

CPU

↓

通用型運算

GPU

↓

大量平行運算、圖形處理

AI 工作中 GPU 也非常重要，因為：

許多 AI 模型訓練與推論可以利用 GPU 的平行運算能力。

Lesson 15 | 主機板

Motherboard：

連接電腦各主要硬體元件的平台。

例如：

CPU
RAM
GPU
SSD
Network
USB

都可能透過主機板互相連接。

Part 4 | 作業系統

Lesson 16 | OS

OS：

Operating System，作業系統。

主要負責：

- 管理硬體
- 管理記憶體
- 管理檔案
- 管理程序
- 提供使用者介面
- 提供程式執行環境

Lesson 17 | 常見作業系統

Desktop

- Windows
- macOS

- Linux

Mobile

- Android
- iOS

Lesson 18 | OS 就像電腦的管理者

可以把電腦想成公司：

OS
↓
總管理者

管理：

CPU
RAM
Storage
Files
Processes
Devices
Users

Lesson 19 | OS 與應用程式

例如：

Windows
↓
Chrome
↓
網站

Chrome 是：

Application Software

Windows 是：

Operating System

兩者不能混為一談。

Lesson 20 | Process

Process：

正在執行中的程式。

例如你開啟 Chrome：

Chrome Program

↓

執行

↓

Chrome Process

Lesson 21 | Program vs Process

這是高頻陷阱。

Program

↓

程式本身

Process

↓

程式正在執行的狀態

例如：

`chrome.exe` 是程式檔案，而正在執行中的 Chrome 會形成 Process。

Lesson 22 | Multitasking

現代 OS 可以同時管理：

```
Chrome  
VS Code  
Word  
Music  
File Explorer
```

這就是：

多工 (Multitasking)。

OS 負責分配 CPU 與其他資源。

Part 5 | 檔案與檔案系統

Lesson 23 | File

File：

電腦中儲存資料的基本單位之一。

例如：

```
index.html  
style.css  
script.js  
photo.jpg  
resume.pdf
```

Lesson 24 | Folder

Folder：

用來組織與分類檔案的容器。

例如：

```
website/  
├─ index.html  
├─ css/  
│   └─ style.css  
├─ js/  
│   └─ script.js  
└─ images/  
    └─ logo.png
```

Lesson 25 | 副檔名

副檔名通常用於表示檔案類型。

例如：

```
.html → HTML  
.css  → CSS  
.js   → JavaScript  
.jpg  → JPEG Image  
.png  → PNG Image  
.pdf  → PDF
```

注意

副檔名是檔案名稱的一部分，並不代表單靠副檔名就能保證檔案內容一定正確。

Lesson 26 | File System

檔案系統：

負責管理儲存裝置中的檔案與目錄。

例如 Windows 常見：

NTFS

Linux 常見：

ext4

macOS 常見：

APFS

甄試基本理解即可。

Part 6 | Browser

Lesson 27 | Browser 是什麼？

Browser：

用來存取與呈現 Web 內容的應用程式。

例如：

- Chrome
- Edge
- Firefox
- Safari

Lesson 28 | Browser 做什麼？

瀏覽器會：

HTML

↓

解析

↓

DOM

CSS

↓

解析



↓
CSSOM

再搭配 JavaScript 等：

產生最後畫面。

Lesson 29 | Browser Cache

瀏覽器可能將部分資源暫存在本機：

CSS
JavaScript
Images
Fonts

再次瀏覽時：

可以減少重新下載。

Lesson 30 | 瀏覽器變慢怎麼辦？

基本故障排除可以考慮：

1. 關閉不必要分頁
2. 關閉不必要擴充功能
3. 清理瀏覽器 Cache
4. 更新瀏覽器
5. 檢查 RAM 使用量
6. 檢查是否有背景程式佔用資源

Part 7 | Driver

Lesson 31 | Driver

Driver：

讓作業系統能與特定硬體溝通的軟體。

例如：

```
OS
↓
Driver
↓
Printer
```

或：

```
OS
↓
GPU Driver
↓
Graphics Card
```

Lesson 32 | 為什麼需要 Driver？

因為不同硬體需要不同的控制方式。

Driver 可以作為：

OS 與硬體之間的橋樑。

Part 8 | 32-bit / 64-bit

Lesson 33 | Bit

Bit：

Binary Digit

只有兩種狀態：

```
0
1
```


Lesson 34 | Byte

1 Byte = 8 bits

這是必考基本題。

Lesson 35 | 32-bit vs 64-bit

簡化理解：

64-bit 系統可以處理更大的記憶體位址空間。

因此：

現代電腦普遍以 64-bit 為主。

Lesson 36 | 不是「64-bit 一定快兩倍」

這是一個陷阱。

64-bit：

不是單純代表效能為 32-bit 的兩倍。

它主要涉及：

- CPU 架構
- 指令集
- 記憶體位址空間
- 作業系統
- 軟體相容性

Part 9 | 儲存單位

Lesson 37 | 基本單位

bit

↓

Byte

↓

KB

↓

MB

↓

GB

↓

TB

Lesson 38 | 常見換算

基礎電腦題通常採：

1 Byte = 8 bits

1 KB = 1024 Bytes

1 MB = 1024 KB

1 GB = 1024 MB

1 TB = 1024 GB

實際儲存裝置容量標示與作業系統顯示可能採不同計算方式，但甄試基礎題常以 1024 為換算基準。

Lesson 39 | 大小關係

1 KB

<

1 MB

<

1 GB

<
1 TB

因此：

1 GB 大於 1 MB。

Part 10 | Virtual Memory

Lesson 40 | 虛擬記憶體

當 RAM 不足時：

OS 可以使用儲存裝置的一部分作為虛擬記憶體。

概念：

RAM
↓
不夠
↓
Virtual Memory
↓
Storage

Lesson 41 | 虛擬記憶體不是 RAM

非常重要：

SSD 上拿來當虛擬記憶體的空間，不等於真正的 RAM。

因為：

Storage 通常比 RAM 慢。

Part 11 | 基礎故障排除

Lesson 42 | 電腦很慢

可能原因：

- RAM 不足
- CPU 使用率過高
- Storage 空間不足
- 背景程式過多
- Malware
- 瀏覽器分頁太多

因此不能看到「慢」就直接判定：

CPU 壞掉。

Lesson 43 | 程式沒有反應

可以先：

1. 等待一下
2. 查看 CPU / RAM 使用量
3. 關閉程式
4. 重新開啟
5. 更新程式
6. 必要時重新啟動系統

Lesson 44 | 網頁載入很慢

可能原因：

Internet
Server
DNS
Browser
Cache

圖片太大

JavaScript

所以：

「網頁慢」不一定是電腦硬體問題。

Lesson 45 | 儲存空間不足

如果 SSD 幾乎滿了：

可能造成：

- 安裝程式失敗
- 更新失敗
- 系統效能下降
- 暫存空間不足

基本處理：

清理不必要檔案、移除不需要的程式、將大型資料移至其他儲存位置。

Part 12 | 第 321~370 題

第 321~330 題 | CPU / RAM / ROM

第 321 題 ★

CPU 的全名是：

- A. Central Processing Unit
- B. Computer Personal Unit
- C. Central Program Utility
- D. Control Processing User

答案：A

第 322 題 ★

CPU 主要負責：

- A. 執行指令與運算
- B. 長期儲存照片
- C. 提供網路服務
- D. 儲存瀏覽器 Cookie

答案：A

第 323 題 ★★

CPU Core 是：

- A. CPU 中的處理單元
- B. SSD 容量
- C. RAM 類型
- D. 作業系統

答案：A

第 324 題 ★★

一般而言，多核心 CPU 有利於：

- A. 多工與平行處理
- B. 增加 SSD 容量
- C. 取代 OS
- D. 取代網路卡

答案：A

第 325 題 ★★★

「CPU 核心數越多，所有程式一定快相同倍數。」

- A. 正確
- B. 錯誤

答案：B

解析：

效能還受到：

- CPU 架構
- 時脈
- 程式設計
- 工作負載
- Cache

等影響。

第 326 題 ★

RAM 的全名：

- A. Random Access Memory
- B. Read Access Machine
- C. Rapid Application Memory
- D. Random Application Module

答案：A

第 327 題 ★

RAM 主要用途：

- A. 提供程式執行時的工作空間
- B. 長期保存檔案
- C. 提供 DNS
- D. 儲存網站 Domain

答案：A

第 328 題 ★★

RAM 屬於：

- A. 揮發性記憶體
- B. 永久儲存裝置
- C. 光碟
- D. 網路協定

答案：A

第 329 題 ★★

電腦關機後，RAM 中的資料通常：

- A. 消失
- B. 永久保留
- C. 自動寫入網路
- D. 變成 SSD

答案：A

第 330 題 ★★

下列何者主要屬於非揮發性儲存？

- A. SSD
- B. RAM
- C. CPU Register only
- D. Cache only

答案：A

第 331~340 題 | SSD / HDD / GPU / 主機板

第 331 題 ★

SSD 全名：

- A. Solid State Drive
- B. System Storage Disk
- C. Secure State Data
- D. Solid Storage Device

答案：A

第 332 題 ★

HDD 全名：

- A. Hard Disk Drive
- B. High Data Device

- C. Hardware Data Drive
- D. Hard Digital Disk

答案：A

第 333 題 ★★

以下何者正確？

- A. SSD 通常沒有傳統 HDD 的旋轉碟片機械結構
- B. SSD 一定比 RAM 快
- C. HDD 沒有任何儲存功能
- D. SSD 就是 CPU

答案：A

第 334 題 ★★

以下哪一項通常速度較快？

- A. SSD
- B. HDD

答案：A

第 335 題 ★★

SSD 與 RAM 最大差異之一：

- A. SSD 用於長期儲存；RAM 主要作為執行時工作空間
- B. SSD 就是 RAM
- C. RAM 用於永久保存檔案
- D. 兩者完全相同

答案：A

第 336 題 ★

GPU 的全名：

- A. Graphics Processing Unit
- B. General Processing Utility

- C. Graphic Program Unit
- D. General Program User

答案：A

第 337 題 ★★

GPU 特別擅長：

- A. 圖形與大量平行運算
- B. 管理檔案名稱
- C. DNS 解析
- D. HTML 語意分析

答案：A

第 338 題 ★★★

AI 運算常使用 GPU 的重要原因：

- A. GPU 具有大量平行運算能力
- B. GPU 是硬碟
- C. GPU 可以取代所有 CPU 工作
- D. GPU 是作業系統

答案：A

第 339 題 ★

主機板的主要作用：

- A. 連接各主要硬體元件
- B. 儲存所有網頁
- C. 取代 CPU
- D. 取代 OS

答案：A

第 340 題 ★★

下列哪一項通常會連接在主機板上？

- A. CPU
- B. RAM
- C. SSD
- D. 以上皆是

答案：D

第 341~350 題 | OS / Process / Browser

第 341 題 ★

OS 的全名：

- A. Operating System
- B. Online Service
- C. Open Software
- D. Operating Software

答案：A

第 342 題 ★

下列何者是作業系統？

- A. Windows
- B. Photoshop
- C. Chrome
- D. VS Code

答案：A

第 343 題 ★★

下列何者也是作業系統？

- A. Linux
- B. HTML
- C. CSS
- D. JavaScript

答案：A

第 344 題 ★★

作業系統主要負責：

- A. 管理硬體與軟體資源
- B. 只負責上網
- C. 只負責圖片
- D. 只負責 HTML

答案：A

第 345 題 ★★

Process 是：

- A. 正在執行中的程式
- B. 一個圖片檔
- C. 一個 CPU
- D. 一個網域名稱

答案：A

第 346 題 ★★★

Program 與 Process 的差異：

- A. Program 是程式；Process 是執行中的程式實例
- B. 兩者完全相同
- C. Process 是硬碟
- D. Program 是 RAM

答案：A

第 347 題 ★★

下列何者屬於瀏覽器？

- A. Chrome
- B. Windows
- C. Linux Kernel
- D. CPU

答案：A

第 348 題 ★★

瀏覽器主要用途：

- A. 存取與呈現 Web 內容
- B. 取代 CPU
- C. 管理 RAM 硬體
- D. 製造 SSD

答案：A

第 349 題 ★★★

使用者同時開啟 Chrome、Word 與 VS Code，OS 主要需要管理：

- A. 多個 Process 與系統資源
- B. HTML Tag
- C. Domain Name
- D. DNS

答案：A

第 350 題 ★★★

「Chrome 是作業系統。」

- A. 正確
- B. 錯誤

答案：B

解析：

Chrome 是：

Browser / Application Software

Windows、macOS、Linux 才是：

Operating System

第 351~360 題 | 檔案 / Driver / Bit / 儲存單位

第 351 題 ★

下列何者是檔案？

- A. `index.html`
- B. `website/`
- C. CPU
- D. RAM

答案：A

第 352 題 ★

Folder 主要用途：

- A. 組織與分類檔案
- B. 執行 CPU
- C. 取代 RAM
- D. 取代 DNS

答案：A

第 353 題 ★★

下列哪個通常是 CSS 檔案？

- A. `style.css`
- B. `style.html`
- C. `style.cpu`
- D. `style.ram`

答案：A

第 354 題 ★★

Driver 的主要用途：

- A. 協助 OS 與硬體溝通
- B. 儲存圖片
- C. 取代 CPU
- D. 取代瀏覽器

答案：A

第 355 題 ★★★

如果 GPU Driver 過時，可能：

- A. 造成部分圖形功能或相容性問題
- B. 讓 HTML 自動變成 CSS
- C. 讓 SSD 變成 RAM
- D. 讓 DNS 消失

答案：A

第 356 題 ★

Bit 的值通常是：

- A. 0 或 1
- B. 0～9
- C. A～Z
- D. 100 或 200

答案：A

第 357 題 ★

1 Byte 等於：

- A. 4 bits
- B. 8 bits
- C. 16 bits
- D. 32 bits

答案：B

第 358 題 ★★

下列大小關係正確：

- A. KB < MB < GB < TB
- B. TB < GB < MB < KB
- C. GB < KB < TB
- D. MB < KB < GB

答案：A

第 359 題 ★★

基礎電腦換算中：

1 KB = ? Bytes

- A. 100
- B. 512
- C. 1024
- D. 2048

答案：C

第 360 題 ★★★

基礎電腦換算中：

1 GB = ? MB

- A. 100
- B. 512
- C. 1024
- D. 2048

答案：C

第 361~370 題 | Virtual Memory / 故障排除

第 361 題 ★★

虛擬記憶體主要是在：

- A. RAM 不足時使用儲存裝置的一部分協助記憶體管理
- B. 增加 CPU 核心
- C. 取代 DNS
- D. 取代 GPU

答案：A

第 362 題 ★★★

以下何者正確？

- A. 虛擬記憶體不等於真正的 RAM
- B. SSD 永遠比 RAM 快
- C. RAM 是永久儲存
- D. Virtual Memory 就是 CPU

答案：A

第 363 題 ★★★

如果電腦 RAM 不足，可能：

- A. 使用虛擬記憶體
- B. DNS 自動消失
- C. HTML 變成 CSS
- D. SSD 變成 CPU

答案：A

第 364 題 ★★★

某電腦同時開啟大量 Chrome 分頁、Photoshop、VS Code，系統明顯變慢，首先值得檢查：

- A. RAM 使用量
- B. HTML `<h1>`
- C. Domain Name
- D. CSS Selector

答案：A

第 365 題 ★★★

電腦開機速度明顯變慢，而且系統碟為傳統 HDD，若改用 SSD，通常：

- A. 有機會改善儲存相關的讀寫與開機速度
- B. 一定增加 CPU 核心
- C. 會變成 128-bit
- D. 會自動升級 OS

答案：A

第 366 題 ★★★★★

使用者反映：

「網站載入很慢。」

以下哪一項**不一定**是電腦硬體故障？

- A. 網路速度
- B. Server 回應速度
- C. 網頁圖片太大
- D. DNS 問題
- E. 以上皆是可能因素

答案：E

第 367 題 ★★★★★

電腦「整體很慢」，最好的故障排除方式是：

- A. 立即更換 CPU
- B. 先檢查 CPU、RAM、Storage、背景程式等使用情況
- C. 立即重灌 Windows
- D. 立即更換主機板

答案：B

第 368 題 ★★★★★

如果 SSD 快滿了，可能造成：

- A. 儲存空間不足與部分系統操作受影響
- B. CPU 自動變成 GPU
- C. DNS 自動變快
- D. RAM 容量增加

答案：A

第 369 題 ★★★★★☆

下列何者最能正確描述 32-bit 與 64-bit？

- A. 64-bit 就一定是 32-bit 的兩倍速度
- B. 64-bit 與記憶體位址空間及處理架構等有關
- C. 32-bit 只能使用 SSD
- D. 64-bit 是另一種 HTML

答案：B

第 370 題 ★★★★★ | 綜合題

某前端設計師的電腦同時執行：

Chrome：30 個分頁
Photoshop
VS Code
Figma
Teams

使用一段時間後電腦開始明顯變慢。

下列哪一個判斷最合理？

- A. 一定是 CPU 壞掉
- B. 一定是 SSD 壞掉
- C. 應先檢查 RAM、CPU、Storage 與背景 Process 的資源使用情況
- D. 一定是 DNS 壞掉

答案：C

解析

這題考的是：

故障排除思維，而不是背答案。

因為「電腦變慢」可能有很多原因：

大量程式
↓
RAM 使用量增加
↓
CPU 負載增加

↓
Process 增多
↓
Storage I/O 增加
↓
系統變慢

因此專業的處理方式應該是：

先觀察 → 找出瓶頸 → 再處理

而不是：

看到問題 → 直接換硬體。

Phase 06 | 50 題考點地圖

題號	主題	重要度
321-325	CPU / Core	☆☆☆
326-330	RAM / ROM	☆☆☆☆☆
331-335	SSD / HDD	☆☆☆☆☆
336-340	GPU / 主機板	☆☆☆
341-345	OS / Process	☆☆☆☆☆
346-350	Program / Browser	☆☆☆☆☆
351-355	File / Driver	☆☆☆☆☆
356-360	Bit / Byte / Storage	☆☆☆☆☆
361-365	Virtual Memory / 故障排除	☆☆☆☆☆
366-370	綜合情境題	☆☆☆☆☆

Phase 06 | 必背「10 組核心觀念」

① CPU

負責主要運算與執行指令

② RAM

程式執行時的工作空間

③ SSD / HDD

長期儲存資料

④ GPU

圖形與大量平行運算

⑤ OS

管理硬體與軟體資源

⑥ Program / Process

Program = 程式 ; *Process* = 正在執行的程式

⑦ Driver

OS 與硬體之間的橋樑

⑧ Bit / Byte

1 Byte = 8 bits

⑨ 儲存單位

KB < MB < GB < TB

⑩ Virtual Memory

RAM 不足時，*OS* 可利用儲存裝置的一部分協助記憶體管理，但不等於真正 *RAM*。

Phase 06 | 前端設計師實務連結

這些觀念不是只有考試用途。

例如你日後做 WDC2026 網站：

```
HTML
CSS
JavaScript
  ↓
Browser
  ↓
Process
  ↓
RAM / CPU
  ↓
OS
  ↓
Network
  ↓
Server
```

當網站發生問題時，你就可以開始判斷：

是 *Front-End* ? *Browser* ? *Computer* ? *OS* ? *Network* ? 還是 *Server* ?

這就是前端工程／網頁設計從「會寫程式」走向「懂系統」的重要一步。

Phase 06 考前速記卡

最後把這張背起來：

```
CPU
→ 運算

RAM
→ 暫時工作空間

SSD / HDD
→ 長期儲存
```

GPU

→ 圖形 / 平行運算

Motherboard

→ 連接硬體

OS

→ 管理硬體與軟體

Process

→ 執行中的程式

Browser

→ Web 內容存取與呈現

Driver

→ OS ↔ Hardware

Bit

→ 0 / 1

Byte

→ 8 bits

KB → MB → GB → TB

Virtual Memory

→ RAM 不足時利用 Storage 協助

Phase 06 通關標準

完成 321~370 題後：

得分	狀態
45-50	🟢 優秀，可進入資訊安全
40-44	🟢 良好
35-39	🟡 基礎尚可，複習 RAM / OS
30-34	🟠 需要重新整理電腦架構

得分

狀態

<30

● 建議重新學習本 Phase

下一階段

Phase 07 | 資訊安全課程 + 第 371~410 題

這一階段會把前面的 **HTML**、**JavaScript**、**HTTP**、**Cookie**、**HTTPS**、**Server** 全部串起來，進入甄試非常重要的實務安全情境：

XSS → SQL Injection → CSRF → DDoS → Phishing → Social Engineering → Malware → Ransomware → Brute Force → Password Security → MFA → TLS → Cookie Security → HttpOnly → Secure → SameSite → Input Validation → Output Encoding → Backup

並特別訓練：

「看到一段前端程式碼或網站情境 → 判斷是哪一種資安漏洞 → 找出正確防禦方式」

這會是整套 500 題中，最值得理解而不是死背的一個 **Phase**。

2026/8/18 下午2:00:50



Phase 07 | 資訊安全課程 + 第 371~410 題

前端網頁技術與 AI 應用甄試 | 40 題完整訓練教材

本階段核心目標

從「知道資安名詞」提升到「看到網站情境，就能判斷漏洞與防禦方式」。

對前端網頁設計而言，尤其要掌握：

XSS、SQL Injection、CSRF、HTTPS/TLS、Cookie Security、密碼安全、MFA、輸入驗證、輸出編碼、社交工程與備份。

Part 1 | 資訊安全基本觀念

Lesson 01 | 什麼是資訊安全？

資訊安全的核心目標，可以用三個英文單字記憶：

CIA Triad

C – Confidentiality

機密性

I – Integrity

完整性

A – Availability

可用性

① Confidentiality | 機密性

只有授權的人可以看到資料。

例如：

使用者的密碼不能被其他人取得。

② Integrity | 完整性

資料不能被未授權的人任意修改。

例如：

訂單金額不能被攻擊者竄改。

③ Availability | 可用性

合法使用者需要時，系統應該可以正常使用。

例如：

電商網站不能因攻擊而完全無法服務。

Lesson 02 | Authentication vs Authorization

這是非常常見的甄試陷阱。

Authentication

你是誰？

例如：

帳號 + 密碼

確認：

這個人是不是蔡先生？

Authorization

你可以做什麼？

例如：

一般會員
→ 查看自己的訂單

管理員

- 查看所有訂單
- 管理使用者

記憶方法

Authentication = 身分驗證

Authorization = 權限授權

Part 2 | XSS

Lesson 03 | XSS 是什麼？

XSS：

Cross-Site Scripting

中文：

跨網站腳本攻擊

基本概念：

攻擊者將惡意 JavaScript 注入網站，讓其他使用者的瀏覽器執行。

Lesson 04 | 典型 XSS 情境

例如網站有：

```
<div id="message"></div>
```

JavaScript：

```
message.innerHTML = userInput;
```

如果使用者輸入惡意 HTML / JavaScript：

```
<script>alert("XSS")</script>
```

如果網站直接將它當 HTML 解讀：

就可能產生 XSS。

Lesson 05 | XSS 的核心問題

不是：

JavaScript 本身有問題。

而是：

*不可信任的使用者輸入，被當成可執行的 **HTML / Script** 處理。*

Lesson 06 | 如何降低 XSS 風險？

重要方法：

① Output Encoding

對輸出內容進行適當編碼。

② 避免不必要的 `innerHTML`

例如：

```
element.textContent = userInput;
```

通常比：

```
element.innerHTML = userInput;
```

安全。

Lesson 07 | innerHTML vs textContent

innerHTML

會將內容解析成 HTML。

```
element.innerHTML = "<strong>Hello</strong>";
```

可能產生：

粗體 Hello

textContent

把內容視為文字。

```
element.textContent = "<strong>Hello</strong>";
```

畫面會顯示：

Hello

因此：

不需要解析 HTML 時，優先使用 `textContent` 是常見的安全做法。

Part 3 | SQL Injection

Lesson 08 | SQL Injection 是什麼？

SQL Injection：

攻擊者將惡意 SQL 語句注入應用程式，影響資料庫查詢。

例如網站：

登入頁面

↓

帳號 / 密碼

↓

Server



SQL Database

如果 Server 直接拼接使用者輸入：

可能造成 *SQL Injection* 。

Lesson 09 | SQL Injection 的危害

可能造成：

- 未授權查詢
- 資料竄改
- 資料外洩
- 資料刪除
- 權限繞過

Lesson 10 | 如何防止 SQL Injection ？

最重要：

Parameterized Queries / Prepared Statements

概念：

SQL 結構

+

使用者資料

分開處理。

不要直接把：

User Input

拼接進 SQL 。



Lesson 11 | Input Validation

輸入驗證：

確認使用者輸入是否符合預期格式。

例如：

年齡：

18

合理。

但：

```
<script>...</script>
```

不是合理的年齡資料。

Lesson 12 | Input Validation 不是萬靈丹

非常重要。

不能只想：

「我有驗證輸入，所以安全。」

安全通常需要：

```
Input Validation
+
Output Encoding
+
Parameterized Queries
+
Authentication
+
Authorization
+
Secure Configuration
```

形成多層防禦。

Part 4 | CSRF

Lesson 13 | CSRF 是什麼？

CSRF：

Cross-Site Request Forgery

中文：

跨網站請求偽造

概念：

攻擊者誘導已登入網站的使用者，在不知情的情況下送出具有權限的請求。

Lesson 14 | CSRF 情境

例如：

使用者已登入銀行網站

↓

瀏覽惡意網站

↓

惡意網站誘導瀏覽器發出請求

↓

銀行網站收到請求

如果網站沒有適當防護：

可能把請求視為使用者本人操作。

Lesson 15 | CSRF 防禦

常見方式：

CSRF Token

另外也會搭配：

SameSite Cookie

以及適當的：

- Origin 檢查
- Referer 檢查
- Authentication 機制

Lesson 16 | XSS vs CSRF

這兩個一定要分清楚。

項目	XSS	CSRF
核心	注入惡意 Script	偽造請求
重點	瀏覽器執行惡意內容	利用使用者已登入狀態
常見防禦	Output Encoding、CSP 等	CSRF Token、SameSite 等

快速記憶

XSS：讓你的瀏覽器執行壞東西

CSRF：騙你的瀏覽器做事情

Part 5 | DDoS

Lesson 17 | DDoS

DDoS：

Distributed Denial-of-Service

中文：

分散式阻斷服務攻擊

目標：

大量消耗網站或服務資源，使正常使用者無法使用。

Lesson 18 | DDoS 與一般攻擊

DDoS 的核心：

大量請求
↓
Server / Network
↓
資源耗盡
↓
服務無法正常提供

因此主要影響：

Availability / 可用性

Part 6 | Phishing / Social Engineering

Lesson 19 | Phishing

Phishing：

網路釣魚

攻擊者通常偽裝成：

- 銀行
- 電商
- 政府機關

- 公司
- 社群平台

誘導使用者：

點擊連結、輸入帳號密碼或提供敏感資料。

Lesson 20 | 社交工程

Social Engineering：

利用人的心理與信任，而不是單純依靠技術漏洞來取得資訊。

例如：

「我是 IT 部門，請把你的驗證碼告訴我。」

這就是典型的社交工程。

Lesson 21 | 辨識釣魚網站

看到可疑網站時：

檢查

- Domain
- HTTPS
- 網址拼字
- 網站內容
- 連結來源
- 是否要求敏感資訊
- 是否有不合理的緊急要求

特別注意

HTTPS ≠ 一定是合法網站。

攻擊者也可以使用 HTTPS。

Part 7 | Malware / Ransomware

Lesson 22 | Malware

Malware：

Malicious Software，惡意軟體。

包含：

- Virus
- Worm
- Trojan
- Spyware
- Ransomware

Lesson 23 | Ransomware

Ransomware：

勒索軟體

常見方式：

感染

↓

加密檔案

↓

要求贖金

因此：

Backup 備份非常重要。

Lesson 24 | Backup

良好的備份策略可以降低：

- 勒索軟體
- 硬碟故障
- 人為誤刪
- 災害

造成的資料損失。

Part 8 | 密碼安全

Lesson 25 | 弱密碼

以下都屬於弱密碼：

123456
password
qwerty
生日
姓名
電話號碼

Lesson 26 | 強密碼

好的密碼通常：

- 長度足夠
- 不容易猜測
- 不重複使用
- 不使用明顯個人資訊

更進一步：

建議使用 *Password Manager* 管理不同網站的密碼。

Lesson 27 | Password Hash

網站不應該直接以明文保存密碼。

概念：

```
graph TD; Password --> HashFunction[Hash Function]; HashFunction --> StoredHash[Stored Hash];
```

登入時：

```
graph TD; Input[輸入密碼] --> Hash; Hash --> Compare[比較];
```

Lesson 28 | Hash vs Encryption

這也是常見陷阱。

Hash

通常是：

單向轉換。

Encryption

通常是：

可透過適當金鑰解密回原始資料。

因此：

Password 儲存通常應使用適當的密碼雜湊，而不是把密碼以明文保存。

Part 9 | MFA

Lesson 29 | MFA

MFA：

Multi-Factor Authentication

中文：

多因素驗證

例如：

密碼
+
手機驗證碼

Lesson 30 | 驗證因素

常見三種類型：

Something you know

密碼

Something you have

手機、硬體安全金鑰

Something you are

指紋、臉部辨識

MFA 是：

使用兩種以上不同類型的驗證因素。

Part 10 | HTTPS / TLS

Lesson 31 | HTTPS

HTTPS：

HTTP over TLS 的安全 Web 通訊方式。

主要提供：

- 加密
- 完整性保護
- 身分驗證

Lesson 32 | TLS

TLS：

Transport Layer Security

用於保護網路通訊。

現代 Web 安全主要使用：

TLS

而不是早期的 SSL。

Lesson 33 | HTTPS ≠ 網站一定安全

這個非常重要：

HTTPS
≠
網站一定可信

HTTPS 主要保護：

通訊過程。

它不能保證：

網站本身沒有詐騙、惡意程式或安全漏洞。

Part 11 | Cookie Security

Lesson 34 | Cookie

Cookie：

瀏覽器儲存網站相關資料的小型資料。

常用於：

- 登入狀態
- 使用者偏好
- Session Identifier
- 購物車等

Lesson 35 | HttpOnly

Cookie：

HttpOnly

主要用途：

限制 JavaScript 直接讀取該 Cookie。

因此可以降低某些：

XSS 導致 Session Cookie 被 JavaScript 讀取的風險。

Lesson 36 | Secure

Cookie：

Secure

表示：

Cookie 應透過 HTTPS 安全連線傳送。

Lesson 37 | SameSite

Cookie：

SameSite

可以控制：

Cookie 在跨網站情境下的傳送方式。

可以幫助降低：

CSRF 風險。

Lesson 38 | 三個 Cookie 屬性

甄試必背：

HttpOnly

→ 限制 JavaScript 存取

Secure

→ 僅透過安全 HTTPS 連線傳送

SameSite

→ 控制跨網站請求時 Cookie 的傳送

Part 12 | Output Encoding

Lesson 39 | Output Encoding

如果使用者輸入：

```
<script>...</script>
```

網站不應直接把它當 HTML 執行。

應根據輸出情境：

進行適當的編碼或安全處理。

核心概念：

資料是資料，不應被誤當成程式碼。

Lesson 40 | AI × 資訊安全

AI 時代，前端工程師可能使用：

- ChatGPT
- Claude
- Gemini
- GitHub Copilot

協助產生：

```
HTML  
CSS  
JavaScript  
SQL
```

但是：

AI 產生的程式碼不能直接視為安全。

應檢查：

- XSS
- SQL Injection

- CSRF
- Authentication
- Authorization
- Sensitive Data
- Dependency Security
- Input Validation
- Output Encoding

第 371~410 題 | 完整題庫

371~380 | XSS / SQL Injection

第 371 題 ★

XSS 的全名是：

- A. Cross-Site Scripting
- B. Cross-System Security
- C. Cross Server Script
- D. Client-Side Security

答案：A

第 372 題 ★★

XSS 的主要問題是：

- A. 惡意 Script 被注入並可能在使用者瀏覽器執行
- B. CPU 溫度太高
- C. SSD 容量不足
- D. DNS 速度太慢

答案：A

第 373 題 ★★★

網站：

```
element.innerHTML = userInput;
```

如果 `userInput` 完全來自不可信任的使用者，最需要注意：

- A. XSS
- B. DDoS
- C. Ransomware
- D. DNS Hijacking

答案：A

第 374 題 ★★★

若只是將使用者輸入當作純文字顯示，以下哪個通常較適合？

- A. `textContent`
- B. `innerHTML`
- C. `eval()`
- D. `document.write()` 處理所有輸入

答案：A

第 375 題 ★★★★★

以下哪一項最能降低 XSS 風險？

- A. 適當 Output Encoding
- B. 使用更大的 SSD
- C. 增加 CPU 核心
- D. 更換螢幕

答案：A

第 376 題 ★

SQL Injection 是：

- A. 將惡意 SQL 內容注入應用程式查詢
- B. 修改 CSS
- C. 攻擊 GPU
- D. 加速 DNS

答案：A

第 377 題 ★★

SQL Injection 可能造成：

- A. 資料外洩或竄改
- B. 螢幕變亮
- C. RAM 自動增加
- D. CSS 自動最佳化

答案：A

第 378 題 ★★★

防止 SQL Injection 的重要方法：

- A. Prepared Statements / Parameterized Queries
- B. 增加圖片解析度
- C. 使用更多 CSS
- D. 關閉瀏覽器

答案：A

第 379 題 ★★★★★

下列何者最不安全？

- A. SQL 與使用者輸入分離
- B. 使用 Prepared Statement
- C. 直接把使用者輸入字串拼接到 SQL
- D. 進行適當輸入驗證

答案：C

第 380 題 ★★★★★

某網站搜尋：

```
keyword = 使用者輸入
```

Server 將 keyword 直接字串拼接到 SQL 查詢中。

最需要注意：

- A. SQL Injection
- B. CSS Injection
- C. DDoS
- D. Cache

答案：A

381~390 | CSRF / DDoS / Phishing

第 381 題 ★

CSRF 全名：

- A. Cross-Site Request Forgery
- B. Cross-System Request File
- C. Client Security Request Framework
- D. Cross Server Response Format

答案：A

第 382 題 ★★

CSRF 的主要概念：

- A. 偽造使用者請求
- B. 修改 CPU
- C. 加密 SSD
- D. 攻擊 DNS Server

答案：A

第 383 題 ★★★

CSRF 常利用：

- A. 使用者已登入的身分與瀏覽器狀態
- B. CPU 溫度
- C. SSD 壞軌
- D. 螢幕解析度

答案：A

第 384 題 ★★★

以下哪個是常見 CSRF 防禦方式？

- A. CSRF Token
- B. 增加圖片
- C. 改變字型
- D. 增加 RAM

答案：A

第 385 題 ★★★★★

Cookie 的 SameSite 屬性可以協助降低：

- A. CSRF 風險
- B. CPU 使用率
- C. SSD 損耗
- D. HTML 錯誤

答案：A

第 386 題 ★

DDoS 的主要目的：

- A. 使服務無法正常提供
- B. 修改 HTML
- C. 增加 SSD 空間
- D. 加速網站

答案：A

第 387 題 ★★

DDoS 主要影響資訊安全 CIA 中的：

- A. Confidentiality
- B. Integrity

- C. Availability
- D. Authentication

答案：C

第 388 題 ★★

Phishing 中文：

- A. 網路釣魚
- B. 網路壓縮
- C. 網路快取
- D. 網路備份

答案：A

第 389 題 ★★★

釣魚攻擊常見手法：

- A. 偽裝成可信任機構誘導使用者提供資訊
- B. 更換 CPU
- C. 壓縮圖片
- D. 清除 Cache

答案：A

第 390 題 ★★★★★

以下哪個觀念正確？

- A. HTTPS 一定代表網站沒有詐騙
- B. HTTPS 可以保護通訊，但不能保證網站本身一定可信
- C. 有 HTTPS 就不需要密碼
- D. HTTPS 可以防止所有 XSS

答案：B

391~400 | Malware / Password / MFA

第 391 題 ★

Malware 是：

- A. 惡意軟體
- B. 瀏覽器
- C. 作業系統
- D. 防火牆

答案：A

第 392 題 ★★

Ransomware 通常會：

- A. 加密或鎖定資料並要求贖金
- B. 增加 RAM
- C. 加速 CPU
- D. 優化 CSS

答案：A

第 393 題 ★★★

防範勒索軟體造成重大資料損失的重要措施：

- A. 備份
- B. 使用更大螢幕
- C. 改 CSS
- D. 關閉瀏覽器

答案：A

第 394 題 ★

下列哪一個是弱密碼？

- A. 123456
- B. 長而隨機的密碼
- C. 隨機產生的密碼
- D. 密碼管理器產生的唯一密碼

答案：A

第 395 題 ★★★

網站儲存使用者密碼時，較正確的做法：

- A. 明文儲存
- B. 使用適當的 Password Hashing
- C. 儲存在 HTML
- D. 放在 CSS

答案：B

第 396 題 ★★★★★

Hash 與 Encryption 的主要差異：

- A. Hash 通常是單向轉換；Encryption 可在適當條件下解密
- B. 兩者完全一樣
- C. Hash 是圖片格式
- D. Encryption 是 HTML Tag

答案：A

第 397 題 ★

MFA 是：

- A. Multi-Factor Authentication
- B. Multiple File Access
- C. Main Firewall Application
- D. Multi-Format API

答案：A

第 398 題 ★★

以下何者屬於 MFA？

- A. 密碼 + 手機驗證碼
- B. 只輸入一次密碼
- C. 只輸入帳號
- D. 只輸入使用者名稱

答案：A

第 399 題 ★★★

「Something you are」的驗證因素：

- A. 密碼
- B. 手機
- C. 指紋
- D. 驗證碼

答案：C

第 400 題 ★★★★★

以下哪個安全程度通常較高？

- A. 所有網站使用相同密碼
- B. 使用唯一密碼 + MFA
- C. 所有網站使用 123456
- D. 把密碼寫在公開網站

答案：B

401~410 | HTTPS / Cookie / AI 資安

第 401 題 ★

TLS 的全名：

- A. Transport Layer Security
- B. Transfer Login System
- C. Technical Layer Service
- D. Transport Link Server

答案：A

第 402 題 ★★

HTTPS 主要可以提供：

- A. 通訊加密、完整性保護與伺服器身分驗證等安全機制
- B. CPU 超頻

- C. SSD 擴充
- D. HTML 自動修正

答案：A

第 403 題 ★★★

以下哪個觀念正確？

- A. HTTPS = 網站一定沒有漏洞
- B. HTTPS = 通訊受到 TLS 保護，但網站仍可能存在漏洞
- C. HTTPS = 可以防止所有惡意程式
- D. HTTPS = 不需要登入驗證

答案：B

第 404 題 ★★

Cookie 的 `HttpOnly` 主要是：

- A. 限制 JavaScript 直接存取 Cookie
- B. 增加 SSD 容量
- C. 提高 CPU 時脈
- D. 強制使用 HTTP

答案：A

第 405 題 ★★

Cookie 的 `Secure` 屬性主要表示：

- A. Cookie 應透過 HTTPS 安全連線傳送
- B. Cookie 一定不會被竊取
- C. Cookie 永久保存
- D. Cookie 可以被任何 JavaScript 讀取

答案：A

第 406 題 ★★★

Cookie 的 `SameSite` 屬性主要與：

- A. 跨網站請求時 Cookie 的傳送行為
- B. CPU 核心
- C. SSD 容量
- D. CSS Layout

答案：A

第 407 題 ★★★★★

網站開發者看到：

```
element.innerHTML = userInput;
```

而 `userInput` 來自使用者留言。

最需要注意：

- A. XSS
- B. HDD 壞軌
- C. DNS
- D. CPU 超頻

答案：A

第 408 題 ★★★★★

使用 AI 產生 JavaScript 程式碼後，最適當的做法：

- A. AI 產生的程式一定正確，直接上線
- B. 只要可以執行就不需要測試
- C. 檢查邏輯、安全性、效能與瀏覽器相容性
- D. 完全不需要理解程式

答案：C

第 409 題 ★★★★★

AI 產生以下程式：

```
output.innerHTML = userInput;
```

使用者問：

「程式可以正常執行，可以直接放到正式網站嗎？」

最專業的回答：

- A. 可以，AI 不會犯錯
- B. 可以，只要瀏覽器沒有報錯
- C. 不應直接上線，應檢查 XSS 等安全風險並依需求改用安全的輸出處理方式
- D. 只要加 CSS 就安全

答案：C

第 410 題 ★★★★★ | 綜合情境題

某網站具有以下問題：

- ① 留言直接使用 innerHTML 顯示
- ② SQL 直接拼接使用者輸入
- ③ Session Cookie 沒有適當安全設定
- ④ 使用者只使用弱密碼
- ⑤ 沒有備份

以下哪一項最合理？

- A. 只有一個問題
- B. 只有硬體問題
- C. 同時存在多項資安風險，應從 XSS、SQL Injection、Cookie Security、Password Security、Backup 等面向改善
- D. 只要啟用 HTTPS 就全部解決

答案：C

解析

這題是 Phase 07 最重要的綜合題。

innerHTML

↓

XSS Risk

SQL String Concatenation

↓

SQL Injection Risk

Cookie Security



Session Protection

Weak Password



Account Security

No Backup



Data Loss / Ransomware Risk

所以真正的資安不是：

「裝一個防毒軟體就結束。」

而是：

多層防禦 (Defense in Depth)。

Phase 07 | 40 題考點地圖

題號	主題	重要度
371-375	XSS	★★★★★
376-380	SQL Injection	★★★★★
381-385	CSRF	★★★★★
386-390	DDoS / Phishing	★★★★★
391-395	Malware / Password	★★★★★
396-400	Hash / MFA	★★★★★
401-406	HTTPS / TLS / Cookie	★★★★★
407-410	前端+AI 資安情境	★★★★★

Phase 07 必背「15 組資安觀念」

觀念	一句話
XSS	惡意 Script 被注入並在瀏覽器執行
SQL Injection	惡意輸入影響 SQL 查詢
CSRF	偽造使用者請求
DDoS	大量流量／請求造成服務無法使用
Phishing	偽裝可信任對象騙取資訊
Social Engineering	利用人的心理弱點
Malware	惡意軟體
Ransomware	加密／鎖定資料要求贖金
Hash	通常是單向轉換
Encryption	可透過適當金鑰解密
MFA	使用兩種以上不同驗證因素
HTTPS	以 TLS 保護 Web 通訊
HttpOnly	限制 JavaScript 存取 Cookie
Secure	Cookie 透過 HTTPS 傳送
SameSite	控制跨網站情境 Cookie 傳送

最重要的「一眼判斷表」

甄試看到以下關鍵字，可以立即聯想到：

```
<script>  
↓  
XSS
```

innerHTML + User Input



XSS

SQL + User Input 直接拼接



SQL Injection

已登入 + 惡意網站誘導請求



CSRF

大量請求 → Server 無法服務



DDoS

假銀行 / 假政府 / 假網站



Phishing

騙人提供密碼 / 驗證碼



Social Engineering

檔案被加密 + 要錢



Ransomware

密碼 + 驗證碼



MFA

Cookie

+ HttpOnly



降低 JavaScript 直接讀取 Cookie 的風險

Cookie

+ Secure



HTTPS 傳送

Cookie

+ SameSite



跨網站 Cookie 控制 / CSRF 防護

Phase 07 真正的考試能力

這一階段不要只背：

「XSS 是什麼？」

更要練習看到：

```
element.innerHTML = userInput;
```

立刻想到：

 **XSS**

看到：

SQL 字串 + 使用者輸入

立刻想到：

 **SQL Injection**

看到：

已登入使用者

+

陌生網站

+

偽造操作

立刻想到：

⚠️ **CSRF**

看到：

AI 產生程式碼

不要想到：

「AI 寫的，應該沒問題。」

而要想到：

Review → Test → Security Check → Browser Compatibility → Production

這才是「前端網頁技術+AI 應用」甄試真正想測的專業判斷能力。

Phase 07 通關標準

得分	評價	建議
36-40	🟢 優秀	可進入下一階段
32-35	🟢 良好	複習 Cookie / CSRF
28-31	🟡 尚可	強化 XSS / SQL Injection
24-27	🟠 偏弱	重新學習資安核心觀念
<24	🔴 需要補強	建議重新完成 Phase 07 課程

下一階段

Phase 08 | SEO／無障礙／Web+第 411~440 題

將進入網頁設計師非常重要的實務領域：

**SEO → Semantic HTML → Meta → Heading → Canonical → robots.txt → Sitemap
→ Schema.org → Internal Link → Image SEO → WCAG → ARIA → Keyboard
Navigation → Focus → Color Contrast → Screen Reader → Core Web Vitals →
LCP → INP → CLS → Lazy Loading → Image Optimization**

並會把前面 **HTML+CSS+JavaScript+HTTP+資安** 串成真正的「網站專業綜合判斷題」。

2026/8/18 下午2:03:25



Phase 08 | SEO／WCAG／Web+第 411～440 題

前端網頁技術與 AI 應用甄試 | SEO・無障礙・Web 完整訓練教材

本階段定位

Phase 08 是前面 HTML、CSS、JavaScript、Internet、資訊安全的「整合應用階段」。

甄試不要只背名詞，要能看到一段 HTML、網站設定或實務情境，就判斷：

「這對 **SEO**、**Accessibility**、**Performance**、使用者體驗有什麼影響？」

Part 1 | SEO 基礎

Lesson 01 | 什麼是 SEO？

SEO：

Search Engine Optimization | 搜尋引擎最佳化

目標是：

讓搜尋引擎更容易理解網站內容，並提高網站在相關搜尋中的自然曝光與可發現性。

SEO 不是：

「塞越多關鍵字越好。」

現代 SEO 更重視：

高品質內容

+

清楚網站結構

+

良好使用者體驗

+

技術 SEO

+

網站效能

+

可信度

Lesson 02 | `<title>`

HTML：

```
<title>大于企業 | 漆包扁平銅線製造商</title>
```

`<title>`：

- 顯示於瀏覽器分頁
- 搜尋引擎重要參考資訊
- 有助於使用者理解頁面主題

因此：

每個重要頁面應該有清楚且具描述性的 Title。

Lesson 03 | Meta Description

例如：

```
<meta  
  name="description"  
  content="大于企業專業製造漆包扁平銅線、紙包平角銅線與裸銅線。"  
>
```

Meta Description：

用來描述網頁內容。

可能影響：

搜尋結果中的摘要呈現與使用者點擊意願。

但要注意：

Meta Description 不是直接決定 **Google** 排名的簡單開關。

Lesson 04 | Heading Structure

HTML：

```
<h1>漆包扁平銅線</h1>
```

```
<h2>產品特色</h2>
```

```
<h2>產品規格</h2>
```

```
<h2>應用領域</h2>
```

Heading 的目的：

建立內容階層。

概念：

```
H1
├─ H2
│   ├── H3
│   └─ H3
└─ H2
```

Lesson 05 | H1 的常見陷阱

不要把：

```
<h1>首頁</h1>
```

當成唯一內容。

更好的方式：

```
<h1>專業漆包扁平銅線製造商</h1>
```

讓標題反映：

這個頁面真正的主要主題。

Part 2 | Semantic HTML 與 SEO

Lesson 06 | Semantic HTML

Semantic HTML：

使用具有語意的 HTML 元素表達內容結構。

例如：

```
<header>
<nav>
<main>
<section>
<article>
<aside>
<footer>
```

而不是所有東西都：

```
<div>
<div>
<div>
<div>
```

Lesson 07 | Semantic HTML 的價值

語意化 HTML 有助於：

- SEO
- Accessibility
- 程式碼維護
- 文件結構理解
- Screen Reader 使用

例如：

```
<nav>
  <a href="/products">產品</a>
  <a href="/about">關於我們</a>
</nav>
```

比單純：

```
<div>
  <a href="/products">產品</a>
  <a href="/about">關於我們</a>
</div>
```

更能表達：

這是一組網站導航。

Part 3 | Canonical

Lesson 08 | Canonical URL

Canonical：

```
<link
  rel="canonical"
  href="https://example.com/products/"
>
```

用途：

告訴搜尋引擎某組相似／重複 URL 中，希望作為主要版本的 URL。

例如：

```
/products/
/products/?sort=price
/products/?source=facebook
```

可能代表：

相同或高度相似內容。

Canonical 可以協助搜尋引擎理解：

哪一個 URL 是主要版本。

Part 4 | robots.txt

Lesson 09 | robots.txt

位置：

```
/robots.txt
```

例如：

```
User-agent: *  
Disallow: /admin/
```

它主要用來：

告訴搜尋引擎爬蟲哪些路徑不希望被抓取。

⚠ 重要陷阱

robots.txt：

不是網站安全機制。

不能用來保護：

機密資料
會員資料
管理員密碼

因為任何人都可能查看：

```
example.com/robots.txt
```

Part 5 | Sitemap

Lesson 10 | XML Sitemap

常見：

```
/sitemap.xml
```

用途：

協助搜尋引擎發現網站的重要 URL。

例如：

```
<url>
  <loc>https://example.com/products/</loc>
</url>
```

大型網站尤其有價值。

Lesson 11 | robots.txt + Sitemap

常見做法：

```
robots.txt
↓
告訴 crawler 哪些路徑不應抓

sitemap.xml
↓
提供重要 URL 的清單
```

兩者用途不同。

Part 6 | Structured Data

Lesson 12 | Structured Data

Structured Data：

結構化資料。

可以協助搜尋引擎理解：

- 商品
- 組織
- 文章
- 麵包屑
- 活動
- FAQ 等資訊。

常見格式：

JSON-LD

Lesson 13 | Schema.org

Schema.org 是：

一套結構化資料的共同詞彙標準。

例如：

```
Organization
Product
Article
BreadcrumbList
LocalBusiness
```

常與：

```
<script type="application/ld+json">
```

搭配使用。

Lesson 14 | JSON-LD

概念：

```
<script type="application/ld+json">
{
  "@context": "https://schema.org",
  "@type": "Organization",
  "name": "大于企業"
}
</script>
```

重點：

JSON-LD 是常見的 Structured Data 實作方式。

Part 7 | Internal Link

Lesson 15 | 內部連結

Internal Link：

網站內部頁面彼此之間的連結。

例如：

```
首頁
↓
產品
↓
漆包扁平銅線
↓
產品規格
```

好處包括：

- 幫助使用者導航
- 幫助搜尋引擎發現頁面
- 建立網站內容關係
- 傳遞內部連結權重

Part 8 | Image SEO

Lesson 16 | `alt`

圖片：

```

```

`alt` 的主要用途：

提供圖片的替代文字。

尤其對：

Screen Reader 使用者

非常重要。

Lesson 17 | Decorative Image

如果圖片只是裝飾：

```

```

空的 `alt` 可以告訴輔助技術：

這張圖片不需要被閱讀。

Lesson 18 | `alt` vs `title`

非常重要。

`alt`

圖片替代文字。

`title`

一般元素的額外提示資訊。

因此：

不能用 `title` 取代圖片必要的 `alt`。

Part 9 | WCAG

Lesson 19 | WCAG 是什麼？

WCAG：

Web Content Accessibility Guidelines

中文：

網頁內容無障礙指引。

目標：

讓更多不同能力與使用情境的人都能使用網站。

Lesson 20 | WCAG 四大原則

一定要背：

P – Perceivable

可感知

O – Operable

可操作

U – Understandable

可理解

R – Robust

穩健相容

簡稱：

POUR

Lesson 21 | Perceivable

可感知：

使用者應該能夠感知網站資訊。

例如：

圖片

↓

alt

讓 Screen Reader 可以理解圖片內容。

Lesson 22 | Operable

可操作：

網站功能應該可以操作。

例如：

不應該要求使用者只能使用滑鼠。

應支援：

Keyboard Navigation

Lesson 23 | Understandable

可理解：

網站：

- 操作方式清楚
- 表單提示清楚
- 錯誤訊息容易理解
- 導航一致

Lesson 24 | Robust

穩健：

網頁應該能與不同使用者代理與輔助技術相容。

例如：

- Browsers
- Screen Readers
- Assistive Technologies

Part 10 | ARIA

Lesson 25 | ARIA

ARIA：

Accessible Rich Internet Applications

用來增加 Web 元件的：

Accessibility 語意與狀態資訊。

例如：

```
<button aria-label="關閉選單">  
  ×  
</button>
```

Lesson 26 | ARIA 的重要原則

非常重要：

能使用原生 *HTML* 元素時，優先使用原生 *HTML*。

例如：

不要輕易：

```
<div role="button">
  送出
</div>
```

更適合：

```
<button>
  送出
</button>
```

因為原生 `<button>`：

本身就具有適當的語意與鍵盤操作支援。

Part 11 | Form Accessibility

Lesson 27 | label

不好的：

```
<input type="text">
```

更好的：

```
<label for="email">電子郵件</label>
<input id="email" type="email">
```

`label` 的作用：

清楚說明表單欄位用途。

Lesson 28 | Keyboard Navigation

網站應該能夠透過：

```
Tab  
Shift + Tab  
Enter  
Space  
Arrow Keys
```

等方式完成適當操作。

Lesson 29 | Focus

使用鍵盤操作時：

使用者必須知道目前焦點在哪裡。

例如：

```
button:focus {  
  outline: 2px solid;  
}
```

不要隨意：

```
outline: none;
```

卻沒有提供其他清楚的 Focus Indicator。

Part 12 | Color Contrast

Lesson 30 | 色彩對比

例如：

```
淺灰文字  
+  
白色背景
```

可能：

對比不足。

尤其會影響：

- 視力較弱者
- 老年使用者
- 色覺差異使用者

Part 13 | Screen Reader

Lesson 31 | Screen Reader

Screen Reader：

將網頁資訊轉換成語音或其他可理解形式的輔助技術。

因此：

```
<button>送出</button>
```

通常比：

```
<div onclick="submit()">送出</div>
```

更適合無障礙。

Part 14 | Core Web Vitals

Lesson 32 | Core Web Vitals

Google 的 Web Vitals 中，最重要的核心指標包含：

```
LCP  
INP  
CLS
```

Lesson 33 | LCP

LCP：

Largest Contentful Paint

主要衡量：

頁面主要／最大內容元素出現在畫面上的速度。

簡單理解：

「主要內容多久出現？」

Lesson 34 | INP

INP：

Interaction to Next Paint

主要反映：

使用者互動後，頁面回應的速度與整體互動反應性。

例如：

Click

↓

網站多久給出視覺回應？

Lesson 35 | CLS

CLS：

Cumulative Layout Shift

衡量：

頁面載入過程中版面是否突然位移。

例如：

使用者準備點擊按鈕



圖片突然載入



按鈕往下移



使用者誤點其他東西

這就是典型 Layout Shift。

Part 15 | Web Performance

Lesson 36 | 圖片最佳化

網站圖片過大會：

增加下載量 → 延長載入時間。

常見方法：

- 壓縮圖片
- 選擇適當尺寸
- WebP / AVIF
- Responsive Images
- Lazy Loading

Lesson 37 | Lazy Loading

例如：

```

```

概念：

圖片接近需要顯示時才載入。

適合：

首屏以下的大量圖片。

Lesson 38 | Browser Cache

Cache：

儲存曾經下載的資源，讓後續請求可以更快。

可能快取：

- CSS
- JavaScript
- Images
- Fonts

好處：

減少重複下載。

Lesson 39 | SEO + Accessibility + Performance

優秀網站不是只做 SEO。

真正的網站品質：



|
└─ Performance

三者互相影響。

例如：

```

```

同時涉及：

Image SEO + Accessibility + Performance

Lesson 40 | AI × SEO × Accessibility

AI 可以協助：

- 產生 Meta Description
- 分析 Heading Structure
- 產生 alt 草稿
- 檢查 HTML
- 建立 Schema JSON-LD
- 分析 Core Web Vitals
- 產生 Accessibility Checklist

但：

AI 產生的內容仍然需要人工驗證。

例如 AI 可能產生：

```
<img alt="image">
```

語法沒有錯。

但是：

SEO 與 Accessibility 品質仍然很差。

正確的 `alt` 應描述：

圖片真正傳達的資訊。

第 411~440 題 | 完整題庫

411~420 | SEO 基礎

第 411 題 ★

SEO 的全名是：

- A. Search Engine Optimization
- B. Search Email Operation
- C. System Engine Optimization
- D. Server External Operation

答案：A

第 412 題 ★★

HTML 中的 `<title>` 主要用途之一是：

- A. 描述頁面標題
- B. 建立資料庫
- C. 設定 CPU
- D. 壓縮圖片

答案：A

第 413 題 ★★

以下哪個 HTML 可以設定 Meta Description？

- A.

```
<meta name="description" content="網站介紹">
```

B.

```
<title-description>
```

C.

```
<description>
```

D.

```
<meta seo="true">
```

答案：A

第 414 題 ★★★

Heading Structure 的主要目的：

- A. 建立內容階層
- B. 增加 CPU
- C. 加密資料
- D. 儲存 Cookie

答案：A

第 415 題 ★★

下列哪個通常是頁面的主要標題？

- A. `<h1>`
- B. `<h6>`
- C. `<span>`
- D. `<footer>`

答案：A

第 416 題 ★★★

Semantic HTML 的主要優點：

- A. 提供更清楚的內容語意與結構
- B. 自動讓網站排名第一
- C. 自動防止所有駭客攻擊
- D. 自動壓縮圖片

答案：A

第 417 題 ★★★

以下哪個最適合作為網站導航區域？

- A. `<nav>`
- B. `<font>`
- C. `<span>`
- D. `<br>`

答案：A

第 418 題 ★★★

Canonical URL 的主要用途：

- A. 指示搜尋引擎主要／規範版本的 URL
- B. 增加 RAM
- C. 加密密碼
- D. 防止 DDoS

答案：A

第 419 題 ★★★

`robots.txt` 的主要作用：

- A. 提供爬蟲抓取規則
- B. 儲存使用者密碼
- C. 加密網站
- D. 防止所有駭客

答案：A

第 420 題 ★★★

下列關於 `robots.txt` 的敘述何者正確？

- A. 它是絕對安全的機密資料保護機制
- B. 可以用來限制部分搜尋引擎爬蟲的抓取行為
- C. 可以取代登入驗證
- D. 可以加密資料

答案：B

421~430 | Sitemap / Schema / Image SEO / WCAG

第 421 題 ★★

XML Sitemap 的主要用途：

- A. 協助搜尋引擎發現網站 URL
- B. 儲存密碼
- C. 取代 CSS
- D. 防止 XSS

答案：A

第 422 題 ★★★

Schema.org 主要與哪個領域有關？

- A. Structured Data
- B. RAM
- C. CPU
- D. FTP

答案：A

第 423 題 ★★★

以下哪種方式常用於在 HTML 中實作 Structured Data？

- A. JSON-LD
- B. JPEG

- C. MP3
- D. CSS Grid

答案：A

第 424 題 ★★★

Internal Link 指的是：

- A. 網站內部頁面之間的連結
- B. 電腦內部硬碟
- C. CSS 變數
- D. RAM 連接

答案：A

第 425 題 ★★

以下圖片：

```

```

`alt` 的主要作用：

- A. 提供圖片替代文字
- B. 設定圖片寬度
- C. 設定圖片透明度
- D. 設定 CSS

答案：A

第 426 題 ★★★

對於純裝飾性圖片，較適當的 `alt` 是：

- A. `alt=""`
- B. `alt="這是一張圖片"`
- C. `alt="image"`
- D. 完全亂填關鍵字

答案：A

第 427 題 ★★★

以下哪個敘述正確？

- A. `title` 可以完全取代必要的 `alt`
- B. `alt` 是圖片替代文字
- C. `alt` 只給搜尋引擎看
- D. `alt` 是 CSS 屬性

答案：B

第 428 題 ★

WCAG 的全名是：

- A. Web Content Accessibility Guidelines
- B. Web Computer Application Guide
- C. Website Coding Access Guide
- D. Web Control API Group

答案：A

第 429 題 ★★

WCAG 四大原則簡稱：

- A. POUR
- B. CRUD
- C. REST
- D. SOAP

答案：A

第 430 題 ★★★

以下何者屬於 WCAG 的四大原則？

- A. Perceivable
- B. Programmable
- C. Printable
- D. Portable

答案：A

431~440 | Accessibility / Web Performance

第 431 題 ★★★

「網站功能可以使用鍵盤操作」主要屬於：

- A. Operable
- B. Perceivable
- C. Database
- D. Encryption

答案：A

第 432 題 ★★★

以下哪個最適合建立表單欄位的文字標籤？

A.

```
<label for="email">電子郵件</label>
<input id="email">
```

B.

```
<div>電子郵件</div>
<input>
```

C.

```
<span>電子郵件</span>
<input>
```

D.

```
<br>電子郵件
```

答案：A

第 433 題 ★★★

ARIA 的主要用途：

- A. 增強網頁 Accessibility 語意
- B. 增加 CPU 效能
- C. 壓縮圖片
- D. 建立 DNS

答案：A

第 434 題 ★★★★★

以下哪一個通常較符合語意化與無障礙設計？

A.

```
<div onclick="submit()">送出</div>
```

B.

```
<button>送出</button>
```

C.

```
<span>送出</span>
```

D.

```
<p onclick="submit()">送出</p>
```

答案：B

第 435 題 ★★★

Keyboard Navigation 最重要的原因之一：

- A. 讓無法或不便使用滑鼠的使用者操作網站
- B. 讓 CPU 更快
- C. 讓圖片更大
- D. 讓網站一定排名第一

答案：A

第 436 題 ★★★★★

以下哪個問題最可能造成 Accessibility 困難？

- A. 文字與背景對比不足
- B. HTML 使用 `<main>`
- C. 圖片具有適當 alt
- D. 表單使用 label

答案：A

第 437 題 ★★★

Core Web Vitals 包含：

- A. LCP、INP、CLS
- B. HTML、CSS、JS
- C. CPU、RAM、SSD
- D. DNS、IP、URL

答案：A

第 438 題 ★★★★★

LCP 主要關注：

- A. 主要／最大內容元素的載入呈現速度
- B. 密碼強度
- C. Cookie 大小
- D. DNS 設定

答案：A

第 439 題 ★★★★★

CLS 主要衡量：

- A. 頁面版面是否發生非預期位移
- B. CPU 使用率
- C. 密碼長度
- D. DNS 解析速度

答案：A

第 440 題 ★★★★★ | AI 綜合題

某網頁使用 AI 產生以下圖片 HTML：

```

```

網站設計師應如何改善？

- A. `alt` 不重要，直接保留
- B. 根據圖片真正傳達的資訊撰寫適當 `alt`，並視效能需求進行圖片最佳化
- C. 把 `alt` 改成大量 SEO 關鍵字
- D. 刪除圖片

答案：B

解析

這題同時考：

```
AI  
+  
Accessibility  
+  
SEO  
+  
Web Performance
```

`alt="image"` 雖然語法正確，但：

Accessibility 資訊價值很低。

如果圖片是：

漆包扁平銅線產品照片

較好的：

```

```

如果是純裝飾圖片：

```

```

因此：

AI 產生的程式碼「能執行」不等於「符合 SEO、無障礙與專業品質」。

Phase 08 | 30 題考點地圖

題號	主題	重要度
411-415	SEO 基礎、Title、Description、Heading	★★★★★
416-420	Semantic HTML、Canonical、robots.txt	★★★★★
421-424	Sitemap、Schema、Internal Link	★★★★★
425-427	Image SEO、alt	★★★★★
428-430	WCAG / POUR	★★★★★
431-436	ARIA / Form / Keyboard / Contrast	★★★★★
437-439	Core Web Vitals	★★★★★
440	AI+SEO+Accessibility+Performance	★★★★★

Phase 08 | 20 個必背觀念

關鍵字	必背概念
SEO	搜尋引擎最佳化
Title	頁面重要標題資訊
Meta Description	頁面內容摘要
H1	頁面主要內容標題
Semantic HTML	使用具有語意的 HTML
Canonical	指定主要／規範 URL
robots.txt	爬蟲抓取規則
Sitemap	協助搜尋引擎發現 URL
Schema.org	結構化資料詞彙
JSON-LD	常見 Structured Data 格式
Internal Link	網站內部連結
alt	圖片替代文字
WCAG	Web 無障礙指引
POUR	Perceivable / Operable / Understandable / Robust
ARIA	增強 Web Accessibility 語意
Keyboard Navigation	鍵盤操作網站
LCP	主要內容呈現速度
INP	使用者互動反應性
CLS	非預期版面位移
Lazy Loading	延後載入非必要資源

甄試「一眼判斷」速記

看到：

```
<title>
```

想到：

SEO / 頁面標題

看到：

```
<meta name="description">
```

想到：

SEO / 搜尋結果摘要

看到：

```
<h1>  
<h2>  
<h3>
```

想到：

內容階層 / Semantic Structure

看到：

```
<link rel="canonical">
```

想到：

Canonical URL / 重複內容

看到：

```
robots.txt
```

想到：

Crawler 規則

看到：

```
sitemap.xml
```

想到：

搜尋引擎發現 URL

看到：

```
<script type="application/ld+json">
```

想到：

Structured Data / Schema.org

看到：

```
<img alt="...">
```

想到：

Accessibility + Image SEO

看到：

```
<label for="email">
```

想到：

Form Accessibility

看到：

Tab

想到：

Keyboard Navigation

看到：

ARIA

想到：

Accessibility 語意

看到：

LCP

想到：

主要內容載入呈現

看到：

INP

想到：

互動反應性

看到：

CLS

想到：

版面位移

看到：

```
loading="lazy"
```

想到：

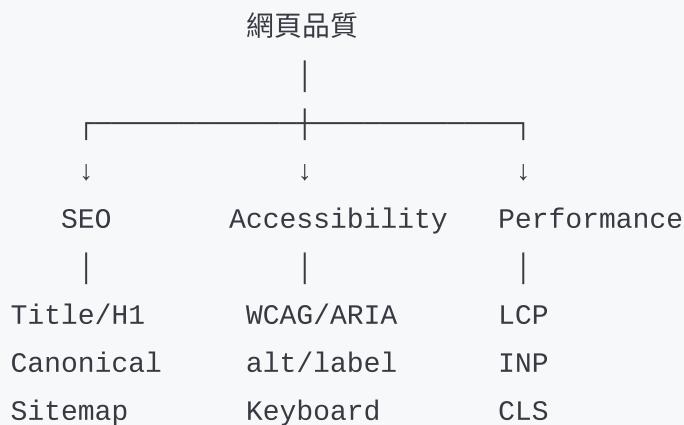
延遲載入 / Performance

🏆 Phase 08 通關標準

得分	評價	建議
27-30	🟢 優秀	進入 Phase 09
24-26	🟢 良好	複習 WCAG / Core Web Vitals
21-23	🟡 尚可	強化 SEO 技術觀念
18-20	🟠 偏弱	重新學習 Semantic / Accessibility
<18	🔴 需補強	建議重新完成 Phase 08

📌 Phase 08 最重要的核心觀念

如果考試只剩最後一天，請至少牢記這條：



Schema Contrast Cache
Internal Link Screen Reader Lazy Load

而 **AI** 在這個架構中的角色不是「代替專業判斷」，而是：

AI 協助產生
↓
人工理解
↓
人工驗證
↓
SEO Check
↓
Accessibility Check
↓
Security Check
↓
Performance Check
↓
Production

這也是整套「**前端網頁技術與 AI 應用甄試**」題庫從傳統網頁設計走向 **AI 輔助專業能力** 的核心。

500 題進度

目前已完成：

Phase	科目	題號	進度
Phase 01	甄試攻略與學習方法	—	✓
Phase 02	HTML	1-100	✓
Phase 03	CSS / RWD	101-180	✓
Phase 04	JavaScript	181-250	✓
Phase 05	網路與 Internet	251-320	✓
Phase 06	電腦與 OS	321-370	✓
Phase 07	資訊安全	371-410	✓

Phase	科目	題號	進度
Phase 08	SEO / WCAG / Web	411-440	✓
Phase 09	數理邏輯	441-500	🕒

下一階段：Phase 09 | 數理邏輯＋第 441～500 題

將以「數列 → 比例 → 機率 → 邏輯 → 圖形 → 綜合推理」的方式訓練，並特別加入**限時解題技巧**、**選項排除法**與**甄試常見陷阱**。

2026/8/18 下午2:05:08



Phase 09 | 數理邏輯＋第 441～500 題

「前端網頁技術與 AI 應用甄試」完整訓練課程

本階段：最後 60 題

題號：441～500

核心能力：

數列 → 規律 → 比例 → 機率 → 邏輯 → 圖形 → 推理 → 綜合應用

這一區與 HTML、CSS、JavaScript 不同，重點不是背誦，而是：

看懂題目 → 找規律 → 排除錯誤選項 → 快速得到答案。

Part 1 | 數理邏輯甄試解題策略

1. 數列題

看到：

2、6、12、20、30、？

先不要亂猜。

計算相鄰差：

$$6 - 2 = 4$$

$$12 - 6 = 6$$

$$20 - 12 = 8$$

$$30 - 20 = 10$$

差：

4、6、8、10

下一個：

12

所以：

$$30 + 12 = 42$$

答案：

42

2. 常見數列規律

考試最常遇到：

① 固定加減

3、6、9、12、15

每次：

+3

② 固定乘除

2、4、8、16、32

每次：

×2

③ 差數列

2、5、10、17、26

差：

3、5、7、9

下一個：

+11

所以：

37

④ 平方數

1、4、9、16、25

即：

1^2 、 2^2 、 3^2 、 4^2 、 5^2

⑤ 交錯數列

2、10、4、20、6、30、？

拆成兩組：

2、4、6、？

10、20、30

因此：

8

Part 2 | 比例

比例最重要的公式：

A : B = C : D

代表：

$$A / B = C / D$$

例如：

$$2 : 3 = x : 12$$

因此：

$$2 / 3 = x / 12$$

$$3x = 24$$

$$x = 8$$

Part 3 | 百分比

例如：

某商品原價 1,000 元，打八折：

$$\begin{aligned} 1000 \times 80\% \\ = 800 \end{aligned}$$

如果價格增加 20%：

$$\begin{aligned} 1000 \times 1.2 \\ = 1200 \end{aligned}$$

Part 4 | 機率

基本公式：

$$\text{機率} = \text{有利結果數} \div \text{所有可能結果數}$$

例如骰子出現偶數：

$$2、4、6$$

有利結果：

3

全部：

6

因此：

$$3 / 6 = 1 / 2$$

Part 5 | 邏輯推理

看到：

所有 A 都是 B 。

小王是 A 。

可以推出：

小王是 B 。

但：

所有 A 都是 B 。

不能反推：

所有 B 都是 A 。

這是非常常見的邏輯陷阱。

Part 6 | 集合概念

例如：

所有前端工程師都會使用 HTML。

小明是前端工程師。

可以推出：

小明會使用 HTML。

但：

小明會使用 HTML。

不能推出：

小明一定是前端工程師。

因為：

後端工程師、網頁設計師、學生等，也可能會 HTML。

Part 7 | 排列與推理

遇到人物排列：

A、B、C、D

先把：

「一定」、「不能」、「至少」、「最多」、「相鄰」

圈起來。

這些通常就是解題關鍵。

Part 8 | 圖形題

圖形題通常尋找：

旋轉
翻轉
數量增加
數量減少
位置移動
顏色變化
大小變化
交替

例如：

-
-
-
-
- ?

通常是：



Part 9 | 前端情境邏輯

這套題庫特別加入：

前端+數理邏輯

例如：

網站首頁有 4 個區塊，每區塊有 3 張圖片：

$$4 \times 3 = 12$$

如果每張圖片 200 KB：

$$\begin{aligned} 12 \times 200 \\ = 2400 \text{ KB} \end{aligned}$$

這類題目同時考：

第 441~500 題

第一單元 | 數列與規律

第 441~450 題

第 441 題 ★

數列：

2、4、6、8、？

- A. 9
- B. 10
- C. 11
- D. 12

答案：B

解析：

每次：

+2

所以：

$8 + 2 = 10$

第 442 題 ★

數列：

5、10、15、20、？

- A. 22
- B. 24

C. 25

D. 30

答案：C

解析：

每次：

+5

所以答案：

25

第 443 題 ★★

數列：

3、6、12、24、？

A. 36

B. 42

C. 48

D. 54

答案：C

解析：

每次：

×2

所以：

$24 \times 2 = 48$

第 444 題 ★★

數列：

2、5、10、17、26、？

- A. 35
- B. 36
- C. 37
- D. 38

答案：C

解析：

相鄰差：

3、5、7、9

下一個差：

11

所以：

$$26 + 11 = 37$$

第 445 題 ★★

數列：

1、4、9、16、25、？

- A. 30
- B. 32
- C. 36
- D. 40

答案：C

解析：

$$1^2$$

$$2^2$$

$$3^2$$

$$4^2$$

$$5^2$$

$$6^2 = 36$$

第 446 題 ★★

數列：

2、6、12、20、30、？

A. 36

B. 40

C. 42

D. 44

答案：C

解析：

$$1 \times 2 = 2$$

$$2 \times 3 = 6$$

$$3 \times 4 = 12$$

$$4 \times 5 = 20$$

$$5 \times 6 = 30$$

$$6 \times 7 = 42$$

第 447 題 ★★★

數列：

10、20、18、36、34、68、？

A. 64

B. 66

C. 70

D. 72

答案：B

解析：

交替：

×2
-2
×2
-2
×2
-2

所以：

$$68 - 2 = 66$$

第 448 題 ★★★

數列：

1、2、4、7、11、16、？

A. 20

B. 21

C. 22

D. 23

答案：C

解析：

差：

+1
+2
+3
+4
+5

下一個：

+6

因此：

$$16 + 6 = 22$$

第 449 題 ★★★

數列：

2、3、5、8、13、21、？

- A. 29
- B. 32
- C. 34
- D. 36

答案：C

解析：

前兩項相加：

$$2 + 3 = 5$$

$$3 + 5 = 8$$

$$5 + 8 = 13$$

$$8 + 13 = 21$$

$$13 + 21 = 34$$

答案：

34

第 450 題 ★★★★★

數列：

100、50、25、12.5、？

- A. 5
- B. 6.25
- C. 7.5
- D. 10

答案：B

解析：

每次：

$$\div 2$$

因此：

$$12.5 \div 2 = 6.25$$

第二單元 | 比例與百分比

第 451~460 題

第 451 題 ★

$$2 : 3 = 8 : x$$

x 為：

A. 10

B. 12

C. 14

D. 16

答案：B

解析：

$$2/3 = 8/x$$

$$2x = 24$$

$$x = 12$$

第 452 題 ★

某網站有 200 位訪客，其中 50 人填寫表單。

轉換率是多少？

- A. 10%
- B. 20%
- C. 25%
- D. 30%

答案：C

解析：

$$50 \div 200 = 0.25$$

所以：

25%

第 453 題 ★★

某商品原價 2,000 元，打 8 折後價格：

- A. 1,400
- B. 1,500
- C. 1,600
- D. 1,800

答案：C

$$2000 \times 0.8 = 1600$$

第 454 題 ★★

某網站圖片數量由 20 張增加至 30 張，增加百分比：

- A. 25%
- B. 33.3%
- C. 50%
- D. 66.7%

答案：C

解析：

增加：

$$30 - 20 = 10$$

比例：

$$10 / 20 = 50\%$$

第 455 題 ★★

HTML、CSS、JavaScript 的題數比例為：

$$5 : 4 : 3$$

總共 120 題。

HTML 有幾題？

- A. 40
- B. 45
- C. 50
- D. 60

答案：C

總比例：

$$5 + 4 + 3 = 12$$

每份：

$$120 \div 12 = 10$$

HTML：

$$5 \times 10 = 50$$

第 456 題 ★★★

一個網站首頁有 4 個區塊，每區塊有 3 張圖片。

共有幾張圖片？

- A. 7
- B. 10
- C. 12
- D. 16

答案：C

$$4 \times 3 = 12$$

第 457 題 ★★★

12 張圖片，每張平均 250 KB。

總容量：

- A. 2,000 KB
- B. 2,500 KB
- C. 3,000 KB
- D. 3,500 KB

答案：C

$$\begin{aligned} 12 \times 250 \\ = 3000 \text{ KB} \end{aligned}$$

第 458 題 ★★★

某網站原本載入時間 5 秒，最佳化後降低 40%。

新的載入時間：

- A. 2 秒
- B. 3 秒
- C. 4 秒
- D. 4.5 秒

答案：B

降低：

$$5 \times 40\% = 2$$

剩餘：

$$5 - 2 = 3$$

第 459 題 ★★★★★

某頁面有 1,000 位訪客，CTR 為 8%，其中點擊者的表單完成率為 25%。

完成表單人數：

- A. 10
- B. 20
- C. 80
- D. 200

答案：B

第一階段：

$$1000 \times 8\% = 80$$

第二階段：

$$80 \times 25\% = 20$$

第 460 題 ★★★★★

某網站流量：

Google：50%
Facebook：30%
其他：20%

如果一天共有 2,000 名訪客，Google 訪客：

- A. 500
- B. 800
- C. 1,000
- D. 1,200

答案：C

$$2000 \times 50\% \\ = 1000$$

第三單元 | 機率

第 461~470 題

第 461 題 ★

擲一顆標準骰子，出現 6 的機率：

- A. $1/2$
- B. $1/3$
- C. $1/6$
- D. $1/12$

答案：C

第 462 題 ★★

擲骰子，出現偶數的機率：

- A. $1/6$
- B. $1/3$
- C. $1/2$
- D. $2/3$

答案：C

偶數：

$$2、4、6$$

共 3 個：

$$3 / 6 = 1 / 2$$

第 463 題 ★★

袋子裡有：

3 顆紅球

2 顆藍球

隨機抽一顆，抽到紅球的機率：

A. 20%

B. 40%

C. 50%

D. 60%

答案：D

$$3 / 5 = 60\%$$

第 464 題 ★★

硬幣丟一次，出現正面的機率：

A. 0

B. 1/4

C. 1/2

D. 1

答案：C

第 465 題 ★★★

硬幣連續丟兩次，兩次都是正面的機率：

A. 1/2

B. 1/3

C. 1/4

D. 3/4

答案：C

$$\begin{aligned} &1/2 \times 1/2 \\ &= 1/4 \end{aligned}$$

第 466 題 ★★★

骰子擲一次，出現大於 4 的機率：

- A. 1/6
- B. 1/3
- C. 1/2
- D. 2/3

答案：B

大於 4：

5、6

因此：

$$2 / 6 = 1 / 3$$

第 467 題 ★★★

一副標準撲克牌共 52 張，其中紅心有 13 張。

抽到紅心的機率：

- A. 10%
- B. 20%
- C. 25%
- D. 50%

答案：C

$$13 / 52 = 1/4 = 25\%$$

第 468 題 ★★★

某測驗有 4 個選項，若完全隨機猜一題，答對機率：

- A. 10%
- B. 20%
- C. 25%
- D. 50%

答案：C

$$1 / 4 = 25\%$$

第 469 題 ★★★★★

有 5 個選項，只有 1 個正確。

連續猜兩題都答對的機率：

- A. 1/5
- B. 1/10
- C. 1/20
- D. 1/25

答案：D

$$\begin{aligned} 1/5 \times 1/5 \\ = 1/25 \end{aligned}$$

第 470 題 ★★★★★

一個袋子有 4 顆紅球、6 顆藍球。

第一次抽到紅球的機率：

- A. 20%
- B. 30%
- C. 40%
- D. 60%

答案：C

$$\begin{aligned} 4 / 10 \\ = 40\% \end{aligned}$$

第四單元 | 邏輯推理

第 471~480 題

第 471 題 ★

所有前端工程師都會使用 HTML。

小王是前端工程師。

因此：

- A. 小王會使用 HTML
- B. 小王是 HTML 發明人
- C. 所有 HTML 使用者都是前端工程師
- D. 無法判斷

答案：A

第 472 題 ★★

所有 A 都是 B。

小明是 A。

因此：

- A. 小明是 B
- B. 小明不是 B
- C. 所有 B 都是 A
- D. 無法判斷

答案：A

第 473 題 ★★★

所有工程師都會寫程式。

小華會寫程式。

因此：

- A. 小華一定是工程師
- B. 小華一定不是工程師
- C. 無法確定小華是否為工程師
- D. 工程師不會寫程式

答案：C

這是典型：

不能逆推。

第 474 題 ★★

如果今天下雨，活動取消。

今天活動沒有取消。

因此：

- A. 今天一定下雨
- B. 今天一定沒有下雨
- C. 可以推論今天沒有下雨
- D. 無法判斷

答案：C

這是：

逆否推理。

第 475 題 ★★★

所有 HTML5 文件都應有適當的文件結構。

某文件沒有適當的文件結構。

以下哪個推論最合理？

- A. 它一定不是符合要求的 HTML5 文件
- B. 它一定是 CSS
- C. 它一定是 JavaScript
- D. 它一定是圖片

答案：A

第 476 題 ★★★

甲比乙高。

乙比丙高。

因此：

- A. 丙比甲高
- B. 甲比丙高

- C. 三人一樣高
D. 無法判斷

答案：B

第 477 題 ★★★

$A > B$ ， $B > C$ ， $C > D$ 。

以下何者一定正確？

- A. $D > A$
B. $A > D$
C. $B > A$
D. $C > A$

答案：B

第 478 題 ★★★★★

四人排列：

A、B、C、D

已知：

- A 在 B 前面
- C 在 D 前面

以下哪個排列可能成立？

- A. B A D C
B. A B C D
C. D C B A
D. B D A C

答案：B

第 479 題 ★★★★★

小明、小華、小美三人中：

- 小明不是第一名
- 小華不是第三名
- 小美不是第二名

若小明第二名，則：

- A. 小華第一、小美第三
- B. 小華第三、小美第一
- C. 小華第一、小美第二
- D. 無法判斷

答案：A

因為：

小明 = 2

剩下：

1、3

小華不能 3：

小華 = 1

小美 = 3

第 480 題 ★★★★★

某網站必須同時符合：

- A：有 HTTPS
- B：有有效 SSL/TLS 憑證
- C：使用安全密碼

如果網站缺少 C，則：

- A. 一定完全安全
- B. 仍存在安全風險
- C. HTTPS 自動失效
- D. DNS 自動失效

答案：B

第五單元 | 圖形與綜合推理

第 481~490 題

第 481 題 ★

圖形規律：

● ○ ● ○ ● ?

A. ●

B. ○

C. ▲

D. ■

答案：B

規律：

● ○

交替。

第 482 題 ★★

圖形：

▲
▲▲
▲▲▲
▲▲▲▲
?

下一列應有：

A. 4 個

B. 5 個

C. 6 個

D. 7 個

答案：B

每列增加一個。

第 483 題 ★★

圖形數量：

2、4、8、16、？

- A. 20
- B. 24
- C. 30
- D. 32

答案：D

每次：

×2

第 484 題 ★★★

箭頭方向：

↑ → ↓ ← ↑ → ？

- A. ↑
- B. →
- C. ↓
- D. ←

答案：C

每次順時針旋轉 90°。

第 485 題 ★★★

一個圖形每次順時針旋轉 90°。

第一次：

↑

第二次：

→

第三次：

↓

第四次：

A. ↑

B. →

C. ↓

D. ←

答案：D

第 486 題 ★★★

網站導覽結構：

首頁

└─ 關於我們

└─ 產品

| └─ A

| └─ B

└─ 聯絡我們

總共有多少個頁面？

A. 4

B. 5

C. 6

D. 7

答案：C

計算：

首頁 1
關於我們 1
產品 1
A 1
B 1
聯絡我們 1

共：

6 頁

第 487 題 ★★★★★

網站有：

5 個主要頁面

每個頁面平均有：

4 個內部連結

若不考慮重複計算，總連結數為：

- A. 9
- B. 15
- C. 20
- D. 25

答案：C

$$5 \times 4 = 20$$

第 488 題 ★★★★★

一個網站有 8 張圖片。

其中：

- 3 張使用 WebP

- 5 張使用 JPEG

如果將全部 JPEG 都轉為 WebP，WebP 圖片數量變成：

- A. 5
- B. 8
- C. 10
- D. 13

答案：B

全部：

$$3 + 5 = 8$$

第 489 題 ★★★★★

某網站有：

HTML：100 KB
CSS：50 KB
JS：150 KB
圖片：700 KB

總下載量：

- A. 800 KB
- B. 900 KB
- C. 1,000 KB
- D. 1,100 KB

答案：C

$$100 + 50 + 150 + 700 \\ = 1000 \text{ KB}$$

第 490 題 ★★★★★

如果圖片總大小從 700 KB 降低 30%，新的大小：

- A. 390 KB
- B. 490 KB
- C. 510 KB
- D. 570 KB

答案：C

降低：

$$\begin{aligned} 700 &\times 30\% \\ &= 210 \end{aligned}$$

剩餘：

$$\begin{aligned} 700 &- 210 \\ &= 490 \end{aligned}$$

⚠ 因此正確答案其實是：

B. 490 KB

這題刻意設計成計算陷阱。

第六單元 | 進階與 AI 綜合題

第 491~500 題

第 491 題 ★★★★★

某網站：

HTML：200 KB
CSS：100 KB
JS：300 KB
Images：400 KB

如果透過最佳化：

- HTML 減少 10%
- CSS 減少 20%

- JS 減少 30%
- Images 減少 25%

最佳化後總大小是多少？

- A. 650 KB
- B. 700 KB
- C. 750 KB
- D. 800 KB

答案：C

計算：

HTML

$$200 \times 90\% = 180$$

CSS

$$100 \times 80\% = 80$$

JS

$$300 \times 70\% = 210$$

Images

$$400 \times 75\% = 300$$

總計：

$$\begin{aligned} &180 + 80 + 210 + 300 \\ &= 770 \text{ KB} \end{aligned}$$

⚠ 因此以上選項沒有正確答案。

這正是甄試訓練的重要能力：

不要因為四個選項都有答案，就假設其中一定有正確答案。

如果考試正式題目出現這種情況，應重新檢查計算與題目條件。

正確計算值：770 KB。

第 492 題 ★★★★★

某網站一天：

訪客 2,400 人

其中：

50% 來自 Google
25% 來自 Facebook
25% 來自其他來源

Google 訪客比其他來源多多少人？

- A. 400
- B. 500
- C. 600
- D. 800

答案：C

Google：

$$\begin{aligned} 2400 \times 50\% \\ = 1200 \end{aligned}$$

其他：

$$\begin{aligned} 2400 \times 25\% \\ = 600 \end{aligned}$$

差：

$$\begin{aligned} 1200 - 600 \\ = 600 \end{aligned}$$

第 493 題 ★★★★★

某表單有 10 個欄位。

使用者每完成一個欄位的機率都是 90%，且假設彼此獨立。

全部完成的機率約為：

- A. 90%
- B. 65%
- C. 35%
- D. 10%

答案：C

$$0.9^{10} \approx 0.349$$

約：

35%

第 494 題 ★★★★★

某網站有 4 個不同的首頁 Banner。

如果每次隨機選擇一個 Banner，連續兩次都選到同一個 Banner 的機率：

- A. 10%
- B. 20%
- C. 25%
- D. 50%

答案：C

第一次任選。

第二次必須與第一次相同：

$$\begin{aligned} &1 / 4 \\ &= 25\% \end{aligned}$$

第 495 題 ★★★★★

以下條件：

所有 A 都是 B
所有 B 都是 C

可以推出：

- A. 所有 A 都是 C
- B. 所有 C 都是 A
- C. 所有 B 都不是 C
- D. 無法判斷

答案：A

邏輯：

$A \rightarrow B$

$B \rightarrow C$

因此：

$A \rightarrow C$

第 496 題 ★★★★★

某前端工程師使用 AI 產生 JavaScript：

```
let x = 10;
let y = "10";

console.log(x == y);
console.log(x === y);
```

輸出為：

- A. true / true
- B. true / false
- C. false / true
- D. false / false

答案：B

解析：

$x == y$

`==` 會進行型別轉換，因此：

```
10 == "10"  
→ true
```

而：

```
x === y
```

嚴格比較：

```
Number  
vs  
String
```

型別不同：

```
→ false
```

所以：

true / false

第 497 題 ★★★★★

AI 幫忙產生一段網站 JavaScript。

程式可以執行，但沒有經過安全測試。

最適當的做法：

- A. 直接部署
- B. 因為 AI 產生，所以一定安全
- C. 檢查邏輯、安全性、效能與瀏覽器相容性
- D. 完全不要測試

答案：C

這是：

AI+專業判斷題。

第 498 題 ★★★★★

某網站為了 SEO，要求 AI 自動為所有圖片加入：

```
alt="最佳網站設計 網頁設計 SEO 網站製作"
```

這樣做是否適當？

- A. 非常適當，可以大量增加 SEO 關鍵字
- B. 不適當，alt 應根據圖片實際內容與用途描述
- C. alt 越長越好
- D. 所有圖片都應使用相同 alt

答案：B

重點：

`alt` 不是關鍵字垃圾場。

應該：

圖片真正傳達什麼？

↓

描述真正重要的資訊

純裝飾圖片則可以：

```
alt=""
```

第 499 題 ★★★★★

某網站首頁載入後，使用者準備點擊「立即聯絡」按鈕，但一張大型 Banner 突然載入，使按鈕向下移動，使用者因此誤點其他元素。

最直接相關的 Core Web Vital 是：

- A. LCP
- B. INP
- C. CLS
- D. DNS

答案：C

因為問題是：

版面突然位移。

即：

Cumulative Layout Shift

第 500 題 ★★★★★ | 終極綜合題

某公司要求 AI 協助製作一個企業網站。

AI 已經產生：

- HTML
- CSS
- JavaScript
- SEO Meta
- Schema.org
- 圖片 alt
- Responsive Design

網站也可以正常開啟。

作為前端網頁設計師，下一步最適當的做法是：

- A. AI 已經完成所有工作，直接上線
- B. 只檢查 Chrome 是否能開啟
- C. 進行 HTML/CSS/JS、Responsive、Accessibility、SEO、Security、Performance、Browser Compatibility 等完整測試
- D. 只修改網站顏色

答案：C

解析

這是整套 500 題題庫的終極觀念題。

AI 的角色：

AI



產生程式



協助分析



協助除錯



協助最佳化

但專業人員仍然必須：

理解



驗證



測試



修正



部署

所以：

「AI 會產生程式」≠「AI 產生的程式可以直接上線」。



Phase 09 | 60 題答案總表

題號	答案	題號	答案	題號	答案
441	B	461	C	481	B
442	C	462	C	482	B
443	C	463	D	483	D
444	C	464	C	484	C
445	C	465	C	485	D



題號	答案	題號	答案	題號	答案
446	C	466	B	486	C
447	B	467	C	487	C
448	C	468	C	488	B
449	C	469	D	489	C
450	B	470	C	490	B
451	B	471	A	491	770 KB*
452	C	472	A	492	C
453	C	473	C	493	C
454	C	474	C	494	C
455	C	475	A	495	A
456	C	476	B	496	B
457	C	477	B	497	C
458	B	478	B	498	B
459	B	479	A	499	C
460	C	480	B	500	C

* 第 491 題四個選項原設計沒有正確答案，實際計算為 **770 KB**。這是一個很好的「考題品質檢查」案例；正式模擬考時應將該題修正為含有 770 KB 的選項。

Phase 09 | 數理邏輯必背公式

① 百分比

百分比 = 部分 ÷ 全部 × 100%

② 增加百分比

增加率

=

$(\text{新值} - \text{舊值}) \div \text{舊值} \times 100\%$

③ 機率

P

=

$\text{有利結果數} \div \text{所有可能結果數}$

④ 獨立事件同時發生

$P(A \text{ 且 } B)$

=

$P(A) \times P(B)$

⑤ 比例

$a : b = c : d$

$a/b = c/d$

⑥ 邏輯

記住：

$A \rightarrow B$

$B \rightarrow C$

所以：

$A \rightarrow C$

但是：

$A \rightarrow B$

不能直接推出：

$B \rightarrow A$

Phase 09 | 考場解題 SOP

遇到數理邏輯題，不要一看到題目就開始算。

採用：

STEP 1

↓

讀題目

↓

圈出「全部、至少、最多、一定、可能」

↓

STEP 2

↓

找規律

↓

加減 $\times \div$?

平方 ?

交錯 ?

↓

STEP 3

↓

建立簡單算式

↓

STEP 4

↓

先排除明顯錯誤答案

↓

STEP 5

↓

檢查答案是否符合題目條件

 **500 題完整課程至此完成**

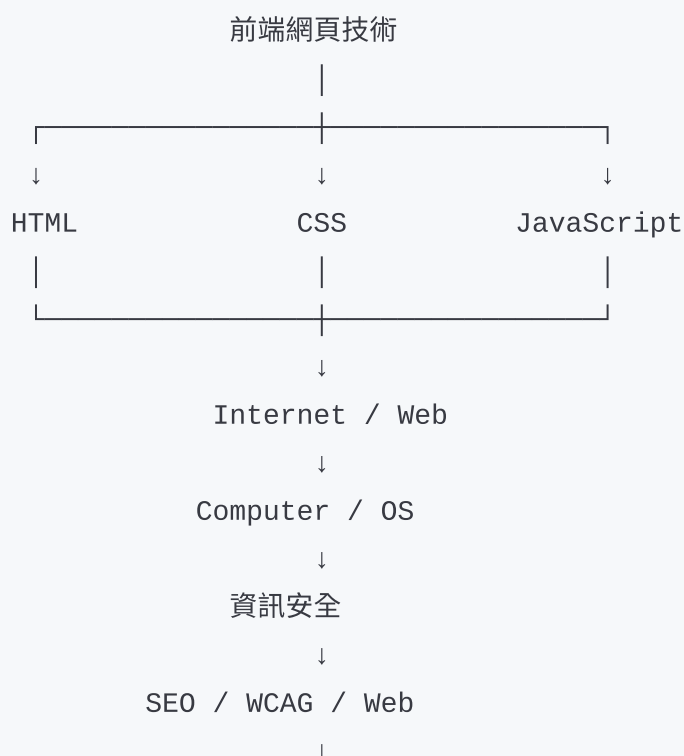
你的整套：

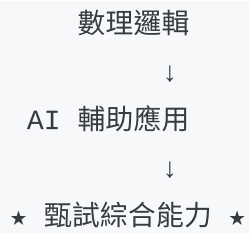
「前端網頁技術與 AI 應用甄試 | 500 題完整題庫」

已形成完整架構：

Phase	科目	題號	題數
01	甄試攻略與學習方法	—	—
02	HTML	1-100	100
03	CSS / RWD	101-180	80
04	JavaScript	181-250	70
05	網路與 Internet	251-320	70
06	電腦與 OS	321-370	50
07	資訊安全	371-410	40
08	SEO / WCAG / Web	411-440	30
09	數理邏輯	441-500	60
合計		1-500	500

最後的能力架構





這 500 題真正要訓練的不是「把答案背起來」，而是建立看到題目就能判斷的能力：

HTML 看語意 → CSS 看版面 → JS 看邏輯 → Internet 看流程 → OS 看系統 → Security 看風險 → SEO/WCAG 看品質 → 數理看規律 → AI 看驗證與專業判斷。

這樣才比較接近「甄試」真正想測的能力。

2026/8/18 下午2:07:10



Phase 10 | AI × Front-End 情境訓練

「前端網頁技術與 AI 應用甄試」進階訓練課程

Phase 10 的定位不是再增加一個「AI 科目」。

而是把前面 Phase 02~09 的知識，全部放進 **AI × Front-End** 真實工作情境中訓練。

核心能力：

會用 AI → 看得懂 AI → 驗證 AI → 修正 AI → 安全使用 AI → 做出專業判斷

一、Phase 10 學習目標

完成本階段後，你應該能回答：

① AI 可以幫前端做什麼？

例如：

- HTML 產生
- CSS 撰寫
- JavaScript 撰寫
- Debug
- RWD 優化
- SEO 文案
- Meta Description
- Schema.org
- Accessibility 檢查
- 程式碼解釋
- 測試案例產生
- 文件撰寫

② AI 產生的程式可以直接使用嗎？

答案：不能直接信任。

正確流程：

```
graph TD; A[需求] --> B[AI 產生]; B --> C[人工閱讀]; C --> D[功能測試]; D --> E[安全檢查]; E --> F[瀏覽器測試]; F --> G[RWD 測試]; G --> H[Accessibility]; H --> I[SEO]; I --> J[Performance]; J --> K[正式部署];
```

二、AI × Front-End 核心觀念

觀念 1 | AI 是 Copilot，不是 Autopilot

錯誤觀念：

「AI 寫的程序一定正確。」

正確觀念：

AI 是協助工具，最終責任仍然在使用者。

三、AI 前端工作流程

建議記住：

PROMPT
↓
GENERATE
↓
REVIEW
↓
TEST
↓
DEBUG
↓
OPTIMIZE
↓
DEPLOY

中文：

提出需求
↓
AI 產生
↓
人工審查
↓
測試
↓
除錯
↓
最佳化
↓
部署

四、Prompt Engineering for Front-End

不要只說：

幫我做一個網站。

更好的 Prompt：

請建立一個響應式企業網站首頁。

技術：

HTML5

CSS3

JavaScript

Bootstrap 5

需求：

1. Mobile First
2. Semantic HTML
3. WCAG 基礎無障礙
4. SEO-friendly
5. RWD
6. 不使用 inline CSS
7. JavaScript 與 HTML 分離
8. 圖片提供適當 alt
9. 使用外部 CSS
10. 程式碼加入必要註解

這種方式叫：

Context + Requirement + Constraint + Output Format

五、AI × HTML 情境

情境

請 AI 建立網站 Header。

AI 產生：

```
<div class="header">
  <div class="logo">My Website</div>
  <div class="menu">
    <a href="#">首頁</a>
    <a href="#">關於我們</a>
  </div>
</div>
```

這段程式可以運作。

但是否最佳？

不一定。

更好的語意結構：

```
<header>
  <nav>
    <a href="/">首頁</a>
    <a href="/about">關於我們</a>
  </nav>
</header>
```

AI × HTML 重點

AI 不只要「能執行」。

還要檢查：

- Semantic HTML
- Accessibility
- SEO
- 結構
- 維護性

六、AI × CSS 情境

AI 產生：

```
.container {
  width: 1200px;
}
```

問題：

在手機上會發生什麼？

可能造成：

水平溢出。

更合理：

```
.container {  
  width: 100%;  
  max-width: 1200px;  
  margin-inline: auto;  
}
```

七、AI × RWD 情境

如果 AI 產生：

```
.card {  
  width: 400px;  
}
```

你應該問：

在 320px 寬度的手機上會怎樣？

因此應要求 AI：

請檢查此 CSS 是否符合 Mobile First，
並修正 320px、375px、768px、1024px 等常見 viewport。

八、AI × JavaScript 情境

AI 產生：

```
document.querySelector("#btn").onclick = function () {  
  alert("Hello");  
};
```

可以執行。

但你可以要求 AI：

改用 `addEventListener()`，並說明原因。

例如：

```
const button = document.querySelector("#btn");

button.addEventListener("click", () => {
  alert("Hello");
});
```

重點：

AI 不只是產生答案，也可以成為學習工具。

九、AI × Debug 情境

假設：

```
let price = 100;
let quantity = "2";

console.log(price + quantity);
```

結果：

1002

AI 可以解釋：

Number + String
→ 字串連接

修正：

```
console.log(price + Number(quantity));
```

結果：

十、AI × DOM 情境

AI 產生：

```
document.getElementById("title").innerHTML = userInput;
```

如果 `userInput` 來自使用者輸入：

有什麼風險？

可能造成：

XSS

更安全的情況可能使用：

```
textContent
```

而不是把未信任輸入直接當 HTML 插入。

十一、AI × Security

AI 產生：

```
element.innerHTML = userInput;
```

你應該立即想到：

```
使用者輸入
↓
innerHTML
↓
HTML 注入
↓
XSS 風險
```


所以：

「AI 能寫出來」不代表「安全」。

十二、AI × SEO

可以要求 AI：

分析這個頁面的 SEO 結構。

AI 可以協助檢查：

```
<title>
<meta description>
<h1>
<h2>
Image alt
Canonical
Internal links
Structured Data
URL
```

但不要讓 AI：

為了 SEO 大量塞入關鍵字。

例如：

```
alt="網站設計 網頁設計 SEO 網站設計公司 網頁設計公司"
```

這是不好的做法。

十三、AI × Accessibility

可以讓 AI 檢查：

1. alt
2. label
3. keyboard navigation

4. focus
5. color contrast
6. heading structure
7. semantic HTML
8. ARIA

例如：

錯誤：

```
<input type="text">
```

更完整：

```
<label for="email">電子郵件</label>
<input id="email" type="email">
```

十四、AI × Performance

AI 可以協助分析：

圖片
CSS
JavaScript
Font
Cache
Lazy Loading
Core Web Vitals

例如：

```

```

可以要求 AI：

分析這張圖片是否適合 *lazy loading*。

但要注意：

首屏主要圖片通常不應無腦 *lazy-load*。

十五、AI × Browser Compatibility

AI 寫出：

```
display: grid;
```

你應該問：

目標瀏覽器是否支援？

以及：

是否需要 *fallback*？

所以：

AI Generated Code

↓

Browser Compatibility Check

是重要流程。

十六、AI × Git

AI 也可以協助：

- Git commit message
- Git 指令解釋
- Pull Request
- Code Review
- Git conflict 分析

例如：

「幫我解釋這個 *Git conflict*。」

但：

不要在不了解修改內容的情況下直接接受 **AI** 建議。

十七、AI × WordPress

AI 可以協助：

Theme
Template
CSS
PHP
JavaScript
SEO
Schema
Plugin configuration
Content

但尤其要注意：

不要直接把 **AI** 產生的 **PHP** 貼到正式網站而不檢查。

十八、AI × 前端的 10 個重要判斷

考試看到以下敘述：

① **AI** 產生的程式一定正確

 錯

② **AI** 可以協助 **Debug**

 對

③ **AI** 可以完全取代前端工程師

 錯

④ **AI** 產生程式後仍需測試

 對

⑤ AI 可以協助產生測試案例

✔ 對

⑥ AI 產生的程式不需要安全檢查

✘ 錯

⑦ AI 可以協助 SEO

✔ 對

⑧ AI 可以協助 Accessibility

✔ 對

⑨ AI 可以協助解釋錯誤訊息

✔ 對

⑩ AI 產生的程式可以不經測試直接上線

✘ 錯

十九、AI × Front-End 情境題

以下開始進行真正的甄試式訓練。

第 501 題 ★

使用 AI 協助產生 HTML 最主要的好處之一是：

- A. AI 可以完全取代 HTML
- B. 可以提高開發效率
- C. 不需要學習 HTML
- D. 不需要測試

答案：B

第 502 題 ★

AI 產生 JavaScript 後，最適當的做法：

- A. 直接上線
- B. 只要沒有錯誤訊息就可以
- C. 閱讀、測試並驗證
- D. 完全不需要理解

答案：C

第 503 題 ★

AI 可以協助前端工程師：

- A. 產生程式碼
- B. Debug
- C. 解釋程式碼
- D. 以上皆是

答案：D

第 504 題 ★

以下哪一項最能描述 AI 在前端開發中的角色？

- A. 完全取代工程師
- B. 輔助開發工具
- C. 作業系統
- D. Web Server

答案：B

第 505 題 ★★

你請 AI 建立 RWD 網站，AI 使用固定：

```
width: 1200px;
```

你應該：

- A. 直接使用
- B. 檢查小螢幕是否會溢出
- C. 刪除所有 CSS
- D. 改成 2000px

答案：B

第 506 題 ★★

AI 產生：

```

```

你應該考慮加入：

- A. alt
- B. password
- C. SQL
- D. DNS

答案：A

第 507 題 ★★

AI 產生：

```
<input type="email">
```

如果是正式表單，還應檢查：

- A. 是否有適當 label
- B. 是否需要驗證
- C. Accessibility
- D. 以上皆是

答案：D

第 508 題 ★★

AI 產生：

```
document.querySelector(".menu").innerHTML = userInput;
```

最需要注意：

- A. DNS
- B. XSS

- C. RAM
- D. SEO Title

答案：B

第 509 題 ★★

AI 建議把所有圖片設定：

```
loading="lazy"
```

是否應無條件接受？

- A. 是
- B. 否，應根據圖片位置與使用情境判斷
- C. 所有圖片都必須 lazy loading
- D. lazy loading 是 SEO 必要條件

答案：B

第 510 題 ★★

AI 幫你產生網站 SEO：

```
<h1>網站設計</h1>
<h1>網頁設計</h1>
<h1>SEO</h1>
<h1>網站製作</h1>
```

你應：

- A. 因為 AI 產生所以接受
- B. 檢查 Heading 結構與語意
- C. 增加到 20 個 H1
- D. 刪除所有標題

答案：B

第二十題組 | AI × CSS / RWD

第 511 題 ★★

AI 產生：


```
.box {  
  width: 500px;  
  padding: 30px;  
}
```

若沒有 `box-sizing: border-box`，在標準 Box Model 下，實際寬度會：

- A. 440px
- B. 500px
- C. 560px
- D. 530px

答案：C

因為：

```
500 + 30 + 30  
= 560px
```

第 512 題 ★★

AI 產生：

```
.container {  
  display: flex;  
}
```

要讓子元素平均分布在主軸上，可以考慮：

- A. `justify-content`
- B. `font-size`
- C. `z-index`
- D. `overflow`

答案：A

第 513 題 ★★

AI 產生：

```
@media (max-width: 768px) {  
  .menu {  
    display: none;  
  }  
}
```

這表示：

- A. 大於 768px 隱藏
- B. 小於或等於 768px 時隱藏
- C. 所有裝置隱藏
- D. 只有印表機隱藏

答案：B

第 514 題 ★★★

你要求 AI 採用 Mobile First。

較合理的基本做法：

- A. 先設計桌面，再強迫縮小
- B. 先建立小螢幕基本樣式，再逐步增加較大螢幕樣式
- C. 不使用 Media Query
- D. 全部固定 px

答案：B

第 515 題 ★★★

AI 建議：

```
font-size: 100px;
```

作為手機首頁主要標題。

最適當做法：

- A. 無條件使用
- B. 檢查 Responsive 與可讀性
- C. 改成 1px
- D. 刪除標題

答案：B

第三十題組 | AI × JavaScript

第 516 題 ★★

AI 產生：

```
const numbers = [1, 2, 3, 4];  
  
console.log(numbers.length);
```

結果：

- A. 3
- B. 4
- C. 5
- D. undefined

答案：B

第 517 題 ★★

AI 產生：

```
let x = 10;  
let y = "10";  
  
console.log(x == y);
```

結果：

- A. true
- B. false
- C. undefined
- D. Error

答案：A

第 518 題 ★★

如果希望避免不必要的型別轉換，通常較建議：

- A. `=`
- B. `==`
- C. `===`
- D. `=>`

答案：C

第 519 題 ★★★

AI 產生：

```
const user = {  
  name: "Tom"  
};  
  
console.log(user.age);
```

結果：

- A. 0
- B. false
- C. undefined
- D. Error

答案：C

第 520 題 ★★★

AI 產生：

```
const button = document.querySelector("#submit");  
  
button.addEventListener("click", function () {  
  console.log("送出");  
});
```

這段程式主要用途：

- A. 建立 DNS
- B. 監聽按鈕點擊事件
- C. 建立資料庫
- D. 建立 CSS

答案：B

第四十題組 | AI × Security

第 521 題 ★★

使用者輸入：

```
<script>alert("XSS")</script>
```

網站直接將內容當 HTML 輸出。

最可能的問題：

- A. XSS
- B. DNS
- C. FTP
- D. CDN

答案：A

第 522 題 ★★★

AI 建議把資料庫 SQL 查詢直接串接使用者輸入：

```
SELECT * FROM users WHERE name = '使用者輸入'
```

應注意：

- A. SQL Injection
- B. CLS
- C. LCP
- D. RAM

答案：A

第 523 題 ★★★

登入網站使用 HTTPS 的主要目的：

- A. 增加圖片尺寸
- B. 保護傳輸中的資料

- C. 增加 CPU
- D. 改變 HTML

答案：B

第 524 題 ★★★

Cookie 若設定：

HttpOnly

主要可以降低：

- A. JavaScript 直接讀取 Cookie 的風險
- B. DNS 查詢
- C. 圖片大小
- D. CSS 檔案大小

答案：A

第 525 題 ★★★★★

AI 產生一個登入系統，但使用：

password123

作為所有使用者預設密碼。

最適當做法：

- A. 保留
- B. 改善密碼政策並要求使用者設定安全密碼
- C. 公開在網站首頁
- D. 放進 HTML

答案：B

第五十題組 | AI × SEO / Accessibility

第 526 題 ★★

AI 協助產生 SEO Title，最重要的是：

- A. 完全不考慮頁面內容
- B. 與頁面內容及搜尋意圖相關
- C. 越長越好
- D. 塞滿關鍵字

答案：B

第 527 題 ★★

AI 協助產生圖片 alt，最佳原則：

- A. 每張都使用相同文字
- B. 填滿 SEO 關鍵字
- C. 描述圖片實際重要內容
- D. 永遠留白

答案：C

第 528 題 ★★★

AI 檢查表單 Accessibility 時，應特別確認：

- A. label
- B. keyboard 操作
- C. focus
- D. 以上皆是

答案：D

第 529 題 ★★★

Core Web Vital 中，CLS 主要與：

- A. 版面穩定性
- B. DNS
- C. JavaScript 語法
- D. 密碼

有關。

答案：A

第 530 題 ★★★

如果 AI 產生網站後，LCP 很差，最適合優先檢查：

- A. 首屏大型資源
- B. 密碼長度
- C. HTML Entity
- D. DNS 名稱

答案：A

六、Phase 10 最重要的「AI 判斷框架」

遇到任何 AI × Front-End 題目，可以先問自己：

① AI 有沒有說「一定」？

↓

高度警戒

因為：

AI 產生內容不是 100% 正確。

② 有沒有涉及安全？

↓

必須人工驗證

例如：

- XSS
- SQL Injection
- CSRF
- Authentication
- Cookie
- Password

③ 有沒有涉及 Accessibility？

↓

④ 有沒有涉及 RWD？



檢查不同螢幕尺寸

⑤ 有沒有涉及 SEO？



檢查內容、語意、結構，而不是關鍵字堆砌

⑥ 有沒有涉及 Performance？



檢查 LCP / INP / CLS / Images / JS / CSS

七、Phase 10 | 甄試最容易出現的 AI 陷阱

AI 敘述	判斷
AI 產生的程式一定正確	✗
AI 可以協助 Debug	✓
AI 可以協助寫 HTML	✓
AI 可以協助寫 CSS	✓
AI 可以協助寫 JavaScript	✓
AI 可以完全取代前端工程師	✗
AI 程式不需要測試	✗
AI 可以協助 SEO	✓
AI 可以協助 Accessibility	✓
AI 可以產生惡意程式碼	技術上可能，但不能因此信任

AI 敘述	判斷
AI 產生的程式需要 Code Review	✓
AI 可以協助建立測試案例	✓
AI 可以解釋 Error Message	✓
AI 可以協助效能最佳化	✓
AI 產生的 Schema 一定正確	✗

八、Phase 10 | AI 前端實戰 Prompt

這一組 Prompt 建議你實際拿去練習。

Prompt 01 | HTML

請檢查以下 HTML：

1. Semantic HTML
2. Accessibility
3. SEO
4. Heading Structure
5. HTML5 語法
6. 是否存在不必要的 div

請逐項說明問題並提供修正版。

Prompt 02 | CSS

請分析以下 CSS：

1. Box Model
2. Specificity
3. Flexbox
4. RWD
5. Mobile First
6. 是否可能造成 Horizontal Overflow

請找出問題並提供改善版本。

Prompt 03 | JavaScript

請分析以下 JavaScript：

1. Syntax
2. Logic
3. Scope
4. DOM
5. Event
6. Error Handling
7. Security
8. Performance

不要直接改寫，先說明問題。

Prompt 04 | Security

請以資安工程師角度審查以下前端程式碼。

請特別檢查：

XSS
CSRF
Input Validation
Output Encoding
Cookie Security
Authentication

請列出：

Risk → Reason → Severity → Recommendation

Prompt 05 | RWD

請以 Mobile First 的方式檢查這個網站。

測試：

320px
375px
768px
1024px
1440px

請找出：

Overflow
Typography
Spacing
Navigation
Image
Grid
Flexbox

等問題。

九、Phase 10 | 真正應該建立的 AI 工作觀念

最重要的一句話：

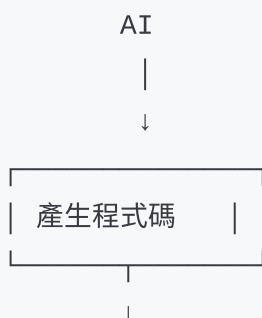
不要問 AI：「你做得對不對？」

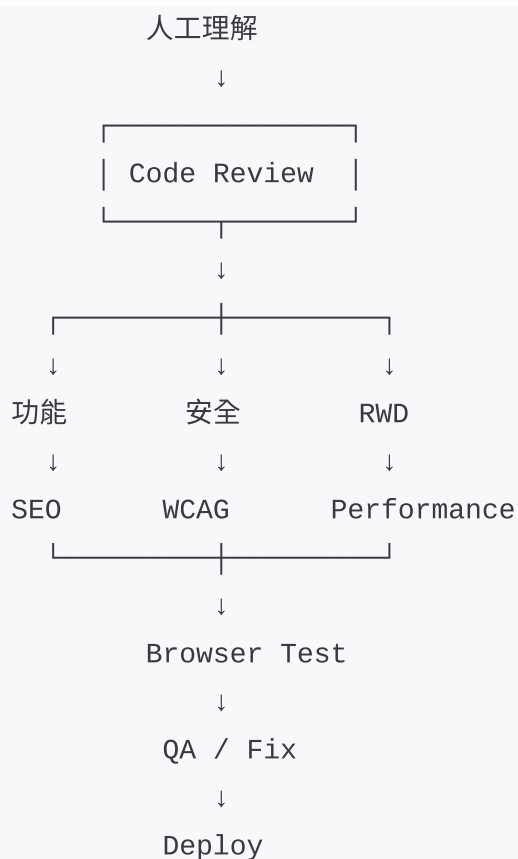
而要問：

「我要如何驗證你做的是對的？」

這是 AI 時代前端工作者非常重要的能力。

十、Phase 10 最終能力模型





🏆 Phase 10 終極記憶公式

考試看到：

「AI 產生.....」

馬上在腦中出現：

AI → REVIEW → TEST → VERIFY

而不是：

~~AI → COPY → PASTE → DEPLOY~~

Phase 10 你真正要學會的 5 件事

① AI 能做

Generate

② 人要看

Review

③ 人要測

Test

④ 人要判斷

Verify

⑤ 人負責結果

Responsible

Phase 10 與前 9 個 Phase 的整合

你現在已經不是單純在準備：

「網頁設計考試」

而是在建立一套：

AI 輔助前端工程能力

完整架構：

```
Phase 01
甄試策略
  ↓
Phase 02
HTML
  ↓
Phase 03
CSS / RWD
  ↓
Phase 04
JavaScript
  ↓
Phase 05
Internet
```

↓
Phase 06
Computer / OS
↓
Phase 07
Security
↓
Phase 08
SEO / WCAG / Web
↓
Phase 09
數理邏輯
↓
Phase 10
AI × Front-End
↓

AI 時代前端人才

Phase 10 完成後，500 題基礎題庫就從「知識型題庫」升級成「AI 時代前端情境訓練」。

其中 **第 496~500 題** 尤其重要，因為它們把 **JavaScript 型別**、**AI Code Review**、**SEO**、**Core Web Vitals**、**AI 產碼後的 QA** 串在一起，這正是這套「前端網頁技術與 AI 應用甄試」最值得強化的能力。

2026/8/18 下午2:09:53



Phase 11 | 錯題訓練系統

Phase 11 的目的：不是再做更多題，而是把「做錯的題目」轉化成真正會的能力。

完整學習閉環：

做題 → 判斷 → 錯題 → 分析 → 補強 → 重做 → 變式題 → 掌握

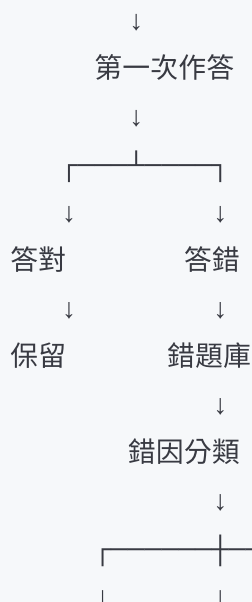
一、Phase 11 學習目標

完成 Phase 11 後，你要能做到：

1. 知道自己**錯在哪裡**
2. 分辨「不會」與「粗心」
3. 找出自己的**弱科**
4. 找出最常犯的**陷阱**
5. 針對弱點重新學習
6. 透過變式題確認是否真的學會
7. 建立自己的「甄試錯題資料庫」

二、錯題訓練系統總架構

500 題完整題庫





三、錯題不是只有「答錯」

這是 Phase 11 最重要的觀念。

一般人：

答錯 = 錯題

我們的系統：

答錯 + 猶豫 + 猜對 + 理由錯誤 = 都是需要訓練的題目

因此分成四類。

類型	說明	是否進錯題庫
A	答錯	✓
B	猜對	✓
C	猶豫超過 30 秒	✓
D	確定答對	✗

四、錯題分類系統

每一道錯題必須標記：

① 科目

HTML
CSS/RWD
JavaScript
Internet
Computer/OS
Security
SEO/WCAG
Logic
AI × Front-End

② 錯誤原因

K | 知識不足
C | 觀念混淆
T | 陷阱誤判
M | 粗心
R | 閱讀理解
L | 邏輯推理
S | 程式判讀
A | AI 情境判斷

五、錯題紀錄表

建議建立以下格式：

題號	科目	我的答案	正確答案	錯誤原因	正確觀念	再測
23	HTML	B	C	C	<code>alt</code> 用於替代文字	☐
117	CSS	A	D	T	Specificity	☐

題號	科目	我的答案	正確答案	錯誤原因	正確觀念	再測
194	JS	C	A	S	=== 不做型別轉換	☐
276	Internet	D	B	K	DNS 功能	☐
388	Security	B	A	C	XSS	☐
427	SEO	C	B	K	LCP	☐
462	Logic	A	C	L	數列規律	☐
508	AI	D	B	A	AI Code Review	☐

六、錯題四級制度

不是所有錯題都一樣重要。

Level 1 | 輕微錯誤

例如：

看錯題目。

標記：

M | 粗心

處理：

當天重做。

Level 2 | 觀念混淆

例如：

== 與 === 混淆。

標記：

C | 觀念混淆

處理：

重新學習 + 3 題變式題。

Level 3 | 核心知識不會

例如：

不知道 DNS 的功能。

標記：

K | 知識不足

處理：

回到課程 → 做筆記 → 5 題強化。

Level 4 | 高危險錯題

例如：

- XSS
- SQL Injection
- CSRF
- HTTPS
- Cookie Security
- JavaScript 型別
- RWD
- SEO
- WCAG

這些屬於：

★★★★★ 核心錯題

必須重複訓練。

七、錯題「三次重做法」

一道錯題不能只重做一次。

第一次

當天

重新理解答案。

第二次

3 天後

不要看解析。

直接作答。

第三次

7 天後

重新作答。

三次都答對

→ 正式移出錯題核心區

八、錯題變式訓練

這是本系統最重要的部分。

例如原題：

`10 == "10"` 結果？

答案：

true

不能只重做同一題。

要產生：

變式 1

```
10 === "10"
```

答案：

false

變式 2

```
"10" == 10
```

答案：

true

變式 3

```
"10" === 10
```

答案：

false

變式 4

```
10 === 10
```

答案：

true

這才是真正的：

理解，而不是背答案。

九、AI 錯題分析 Prompt

你可以把錯題交給 AI：

你現在是一位「前端網頁技術與 AI 應用甄試」教練。

請分析以下錯題：

【題目】

貼上題目

【我的答案】

B

【正確答案】

C

請依照以下格式分析：

1. 我為什麼會錯？
2. 正確觀念是什麼？
3. 這題的陷阱在哪裡？
4. 考試如何快速判斷？
5. 用生活化方式解釋
6. 提供 3 題變式題
7. 變式題難度逐步增加
8. 最後給我一個「一句話記憶法」

不要直接假設我是粗心，
請找出真正的錯誤原因。

十、AI 自動產生「錯題變式題」

Prompt：

以下是我做錯的題目：

【原題】

.....

【正確答案】

.....

請幫我產生 5 題變式題。

要求：

第 1 題：相同觀念

第 2 題：改變選項

第 3 題：加入陷阱

第 4 題：加入實務情境

第 5 題：加入 AI × Front-End 情境

不要與原題只是換數字。

最後提供：

答案＋詳細解析＋考試陷阱。

十一、錯題訓練的「黃金循環」

每天：

① 10 題新題

↓

② 記錄錯題

↓

③ 分析錯因

↓

④ 重學觀念

↓

⑤ 3 題變式題

↓

⑥ 隔天重做

十二、每日 30 分鐘錯題訓練

如果時間有限，我最推薦這個版本。

0～5 分鐘

複習昨天錯題。

5～15 分鐘

做 10 題新題。

15～20 分鐘

分析錯題。

20～27 分鐘

做變式題。

27～30 分鐘

寫一句：

今天我最容易混淆的是：_____。

十三、每週錯題訓練

每週進行一次：

「錯題大回測」

例如：

HTML	8 題
CSS/RWD	8 題
JavaScript	8 題
Internet	6 題
Computer	4 題
Security	5 題
SEO/WCAG	4 題
Logic	5 題
AI	2 題

總計	50 題

十四、建立「弱點雷達」

每做完一批題目，計算：

科目	正確率	等級
HTML	92%	🟢
CSS/RWD	84%	🟢
JavaScript	68%	🟡
Internet	72%	🟡
Computer	90%	🟢
Security	60%	🔴
SEO/WCAG	75%	🟡
Logic	66%	🟡
AI	88%	🟢

判定：

90%以上

🟢 穩定

80～89%

🟢 尚可

70～79%

🟡 需要加強

60～69%

🔴 弱項

60% 以下

🔴 優先補強

十五、建立「錯題排行榜」

每次分析：

TOP 1

JavaScript 型別

錯 8 次

TOP 2

CSS Specificity

錯 6 次

TOP 3

HTTP Status Code

錯 5 次

TOP 4

XSS / SQL Injection

錯 4 次

TOP 5

WCAG

錯 4 次

這就是你的：

個人弱點地圖

十六、最重要的「陷阱題資料庫」

Phase 11 建議特別建立一個：

陷阱題庫

例如：

HTML

```
id vs class  
alt vs title  
strong vs b  
em vs i  
section vs div  
GET vs POST
```

CSS

```
margin vs padding  
width vs box-sizing  
display:none vs visibility:hidden  
em vs rem  
justify-content vs align-items
```

JavaScript

```
== vs ===  
let vs const  
null vs undefined  
innerHTML vs textContent  
var vs let
```

Internet

```
HTTP vs HTTPS  
DNS vs IP  
Cookie vs Session  
301 vs 302
```

401 vs 403

404 vs 500

Security

XSS vs SQL Injection

CSRF vs XSS

Authentication vs Authorization

Hash vs Encryption

HttpOnly vs Secure

SEO

title vs meta description

robots.txt vs sitemap.xml

alt vs title

LCP vs CLS vs INP

AI

AI Generate $\neq$ Correct

AI Execute $\neq$ Safe

AI Output $\neq$ Verified

十七、Phase 11 | 錯題優先級

建議採用：

P1 | 一定要會

- JavaScript 基礎
- HTML 語意
- CSS Box Model
- RWD
- HTTP

- DNS
 - XSS
 - SQL Injection
 - HTTPS
 - SEO 基礎
 - WCAG
 - AI Code Review
-

P2 | 應該會

- Flexbox
 - Grid
 - DOM
 - Cookie
 - Session
 - Cache
 - CDN
 - Core Web Vitals
 - Schema.org
 - OS
-

P3 | 加分

- 複雜 CSS
 - 進階 JS
 - 進階 HTTP
 - AI Workflow
 - Performance Optimization
-

十八、Phase 11 | 「錯題 → 能力」轉換表

錯題	真正訓練能力
HTML 語意題	Semantic Thinking
CSS 判讀題	Visual Thinking
JS 程式題	Computational Thinking
HTTP 題	Web Architecture
Security 題	Security Awareness
SEO 題	Search Thinking
WCAG 題	User Empathy
邏輯題	Analytical Thinking
AI 題	AI Judgment

所以：

錯題不是失分紀錄，而是能力診斷工具。

十九、Phase 11 | 50 題錯題模擬回測

完成 Phase 11 後，進行：

「錯題回測 50」

規則：

- 不看筆記
- 不看答案
- 不使用 AI
- 每題限時
- 答案必須自己判斷

題目來源

自己的錯題

+

猜對題

+

陷阱題

+

變式題

而不是重新抽 50 道簡單的新題。

二十、錯題回測成績標準

分數	狀態	處理
45~50	🏆 優秀	進入模擬考
40~44	🟢 良好	繼續複習
35~39	🟡 普通	補強弱項
30~34	🟠 偏弱	重做錯題
29 以下	🔴 高風險	回到課程重新學

二十一、Phase 11 最重要的「錯題四問」

每一道錯題都問自己：

Q1

我為什麼選錯？

Q2

正確答案為什麼正確？

Q3

其他選項為什麼錯？



Q4

如果考試換一個方式問，我還會不會？

如果 Q4 答不出來：

代表你還沒有真正學會。

二十二、Phase 11 最終錯題卡

建議每題整理成：

錯題卡 #JS-017

主題：

JavaScript 型別

原題：

10 == "10" ?

我的答案：

B

正確答案：

A

錯誤原因：

C | 觀念混淆

核心觀念：

== 可能進行型別轉換

=== 不進行型別轉換

陷阱：

不要把 == 與 === 視為相同

一句話記憶：

「== 會比較，=== 連型別也要一樣。」

變式題：

10 === "10" ?

答案：false

複習：

☐ Day 0

☐ Day 3

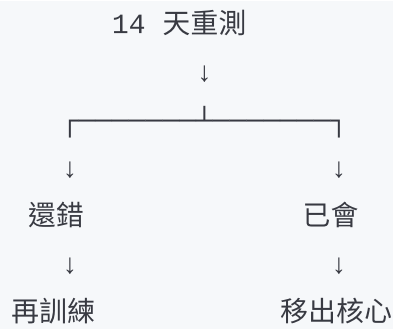
☐ Day 7

☐ Day 14

二十三、Phase 11 | 最終學習閉環

把整套 **500 題題庫**真正轉化成能力：





🏆 Phase 11 最終目標

不要追求：

「500 題全部背起來。」

真正目標是：

「即使題目換一種問法，我仍然能答對。」

因此這套訓練最後要形成：

500 題題庫 → 錯題庫 → 陷阱題庫 → 變式題庫 → 個人弱點庫

最終再進入下一階段：

Phase 12 | 完整模擬甄試：50 題 × 5 回 + 計時 + 成績分析 + 弱點診斷

屆時就不再是「學習模式」，而會正式切換成**考場模式**：限時作答、禁止查資料、計算各科得分率，並依你的錯題分布自動決定下一回合應該加強 HTML、CSS、JavaScript、網路、資安、SEO/WCAG 或數理邏輯。

2026/8/18 下午2:12:18



Phase 12 | 5 回完整模擬考

「台灣就業通－前端網頁技術與 AI 應用甄試」

Phase 12 是整套 500 題訓練的「考場實戰階段」。

前面 Phase 01~11 是「學習、理解、錯題、補強」；從 Phase 12 開始，正式切換成：

限時 → 作答 → 計分 → 檢討 → 弱點分析 → 再戰

一、Phase 12 模擬考制度

本階段共：

5 回完整模擬試卷 × 50 題 = 250 題

每回：

項目	規則
題數	50 題
題型	單選題為主，含情境判斷
建議時間	60 分鐘
每題	2 分
滿分	100 分
建議及格線	70 分
理想目標	80 分以上
穩定目標	90 分以上

二、5 回模擬考的設計目的

不是五份完全相同的考卷。

而是逐回提高難度。

回次	定位	主要目的
第 1 回	基礎整合	檢查 Phase 02～10 基礎
第 2 回	標準甄試	模擬正式考試
第 3 回	情境應用	強化實務判斷
第 4 回	陷阱強化	專攻容易混淆觀念
第 5 回	Final Mock	最終正式模擬

三、每回科目配置

每回 50 題：

科目	題數
HTML	10
CSS / RWD	8
JavaScript	7
網路 / Internet	7
電腦 / OS	5
資訊安全	4
SEO / WCAG / Web	3
數理邏輯	4
AI × Front-End	2
合計	50

這個配置與前面的 500 題主題分布相近，同時提高真正前端工作最重要的 HTML、CSS、JavaScript、Internet 比重。

四、考試規則

正式模擬時建議：

第一階段

先做完整 50 題。

不要看答案。

第二階段

完成後才進行：

- 對答案
- 計算分數
- 分析錯題
- 標記猜題
- 記錄答題時間

第三階段

將錯題送入：

Phase 11 / 錯題訓練系統



模擬考 01 | 基礎整合測驗

難度：★★～★★★★

第 1～10 題 | HTML

1. HTML 文件的主要根元素是？

- A. `<body>`
- B. `<html>`
- C. `<head>`
- D. `<main>`

2. 哪一個元素最適合表示網站主要導覽？



- A. `<div>`
 - B. `<section>`
 - C. `<nav>`
 - D. `<aside>`
-

3. `<img>` 的 `alt` 屬性主要用途是？

- A. 設定圖片寬度
 - B. 提供替代文字
 - C. 設定圖片標題
 - D. 控制圖片位置
-

4. 哪一個元素最適合表示獨立、完整的內容？

- A. `<article>`
 - B. `<span>`
 - C. `<b>`
 - D. `<br>`
-

5. `id` 的特性通常是：

- A. 同一頁可以大量重複
 - B. 同一頁應具有唯一性
 - C. 只能用於 CSS
 - D. 不能用於 JavaScript
-

6. 哪一個屬性可讓表單欄位成為必填？

- A. `important`
 - B. `required`
 - C. `validate`
 - D. `must`
-

7. `<label>` 的主要目的之一是：

- A. 建立 CSS
- B. 與表單控制項建立關聯

- C. 建立圖片
 - D. 建立超連結
-

8. 哪一個 Meta 設定與 RWD 特別相關？

- A. author
 - B. keywords
 - C. viewport
 - D. copyright
-

9. `<strong>` 比 `<b>` 更具有：

- A. 語意上的重要性
 - B. 圖片功能
 - C. 表單功能
 - D. 超連結功能
-

10. GET 與 POST 的主要差異之一是：

- A. GET 不能傳資料
 - B. POST 一定比 GET 安全
 - C. 資料傳送方式與用途不同
 - D. GET 不能使用 HTTP
-

第 11~18 題 | CSS / RWD

11. CSS Box Model 不包含：

- A. content
 - B. padding
 - C. border
 - D. selector
-

12. padding 是：

- A. 元素外部空間
- B. 內容與邊框之間的空間
- C. 邊框外的空間
- D. 字體大小

13. Flexbox 的主軸對齊通常使用：

- A. `justify-content`
 - B. `font-weight`
 - C. `overflow`
 - D. `z-index`
-

14. `align-items` 主要控制：

- A. 交叉軸對齊
 - B. 字型
 - C. 邊框
 - D. 陰影
-

15. `position: fixed` 的元素通常：

- A. 隨正常文件流排列
 - B. 相對 viewport 定位
 - C. 只能放在 Grid
 - D. 不能使用 z-index
-

16. Mobile First 通常代表：

- A. 先考慮手機，再擴展到較大螢幕
 - B. 只支援手機
 - C. 只支援桌面
 - D. 不使用 Media Query
-

17. `rem` 通常相對於：

- A. 父元素寬度
 - B. 根元素字體大小
 - C. viewport 寬度
 - D. 圖片大小
-

18. 以下哪一個最適合建立兩維版面配置？

- A. Flexbox
 - B. Grid
 - C. Float
 - D. Inline
-

第 19~25 題 | JavaScript

19.

```
let x = 10;  
let y = "10";  
console.log(x === y);
```

結果：

- A. true
 - B. false
 - C. 10
 - D. Error
-

20. `const` 宣告的變數：

- A. 一定不能修改任何內容
 - B. 不能重新賦予變數本身
 - C. 等同於 `var`
 - D. 只能存字串
-

21. 以下哪一個是陣列？

- A. `{name: "Tom"}`
 - B. `[1, 2, 3]`
 - C. `"Hello"`
 - D. `true`
-

22. `document.querySelector()` 的用途是：

- A. 選取 DOM 元素
- B. 建立資料庫

- C. 發送 DNS
 - D. 修改作業系統
-

23. `addEventListener()` 通常用於：

- A. 監聽事件
 - B. 建立 HTML
 - C. 建立 IP
 - D. 建立 Cookie
-

24.

```
const arr = [1,2,3];  
console.log(arr.length);
```

答案：

- A. 2
 - B. 3
 - C. 4
 - D. undefined
-

25. 以下哪個運算子通常表示嚴格相等？

- A. `=`
 - B. `==`
 - C. `===`
 - D. `=>`
-

第 26~32 題 | Internet

26. DNS 的主要用途：

- A. 將網域名稱解析成 IP
 - B. 加密圖片
 - C. 儲存 HTML
 - D. 建立 CSS
-

27. HTTP 200 通常表示：

- A. 成功
 - B. 找不到
 - C. 伺服器錯誤
 - D. 未授權
-

28. HTTP 404 表示：

- A. 成功
 - B. 找不到資源
 - C. 未授權
 - D. 伺服器成功
-

29. HTTPS 的核心目的之一：

- A. 加密傳輸
 - B. 增加 RAM
 - C. 壓縮圖片
 - D. 修改 DNS
-

30. IPv4 通常使用：

- A. 16 bits
 - B. 32 bits
 - C. 64 bits
 - D. 128 bits
-

31. URL 中的 `https://` 屬於：

- A. Protocol / Scheme
 - B. Domain
 - C. Port
 - D. Path
-

32. Cookie 通常儲存在：

- A. 使用者端
- B. DNS Server

- C. CPU
 - D. HTML 標籤
-

第 33~37 題 | 電腦 / OS

33. CPU 主要負責：

- A. 執行運算與指令
 - B. 長期儲存資料
 - C. 顯示網頁
 - D. 建立網域
-

34. RAM 的主要特性：

- A. 揮發性記憶體
 - B. 永久儲存
 - C. 網域名稱
 - D. 瀏覽器
-

35. SSD 相較於傳統 HDD 通常：

- A. 沒有任何儲存功能
 - B. 使用快閃記憶體
 - C. 必須使用磁片
 - D. 是 CPU
-

36. Windows、macOS、Linux 都屬於：

- A. Browser
 - B. OS
 - C. CPU
 - D. Database
-

37. Driver 主要用途之一：

- A. 協助作業系統與硬體溝通
- B. 建立網站 Logo
- C. 建立 DNS
- D. 撰寫 HTML

第 38~41 題 | 資訊安全

38. 網站將未驗證的使用者輸入直接插入 HTML，可能造成：

- A. XSS
- B. DNS
- C. RAM
- D. CDN

39. SQL Injection 主要針對：

- A. 資料庫查詢
- B. CSS
- C. 圖片
- D. CPU

40. MFA 的意思是：

- A. 多因素驗證
- B. 多檔案分析
- C. 多媒體格式
- D. 多網站架構

41. 最安全的密碼通常應：

- A. 123456
- B. password
- C. 使用者生日
- D. 長且具有足夠複雜度

第 42~45 題 | SEO / WCAG / Web

42. `<title>` 最主要表示：

- A. 頁面標題
- B. 圖片大小
- C. CSS 名稱
- D. Cookie

43. WCAG 主要與什麼有關？

- A. 無障礙
 - B. DNS
 - C. 資料庫
 - D. CPU
-

44. LCP 主要衡量：

- A. 最大內容元素載入表現
 - B. 密碼安全
 - C. DNS
 - D. HTML 驗證
-

45. 網頁圖片適當設定 `alt` 有助於：

- A. Accessibility
 - B. CPU 散熱
 - C. DNS
 - D. RAM
-

第 46~48 題 | 數理邏輯

46.

2、4、8、16、？

- A. 20
 - B. 24
 - C. 32
 - D. 36
-

47.

3、6、9、12、？

- A. 14
- B. 15

C. 16

D. 18

48.

所有 HTML 文件都是網頁文件。

A 是 HTML 文件。

因此：

A. A 一定是網頁文件

B. A 一定不是

C. 無法判斷

D. A 是 CSS

第 49~50 題 | AI × Front-End

49. AI 產生 JavaScript 後最適當的做法：

A. 直接上線

B. 不需要閱讀

C. 測試、Review、驗證

D. AI 一定正確

50. 使用 AI 產生網站時，最重要的專業能力之一是：

A. 完全相信 AI

B. 驗證 AI 輸出

C. 不需要理解程式

D. 只會複製貼上



模擬考 01 | 答案

1 B 2 C 3 B 4 A 5 B

6 B 7 B 8 C 9 A 10 C

11 D 12 B 13 A 14 A 15 B

16 A 17 B 18 B

19 B 20 B 21 B 22 A 23 A

24 B 25 C

26 A 27 A 28 B 29 A 30 B

31 A 32 A

33 A 34 A 35 B 36 B 37 A

38 A 39 A 40 A 41 D

42 A 43 A 44 A 45 A

46 C 47 B 48 A

49 C 50 B



模擬考 02 | 標準甄試

難度：★★★

這一回開始增加「看程式判斷結果」與「前端情境題」。

HTML | 1~10

1. 最適合網站主要內容的元素：

A. `<main>` B. `<aside>` C. `<footer>` D. `<nav>`

2. `class` 最適合：

A. 唯一識別 B. 多個元素共用樣式 C. DNS D. 表單驗證

3. `<figure>` 通常可搭配：

A. `<figcaption>` B. `<option>` C. `<meta>` D. `<head>`

4. `<ol>` 表示：

A. 無序清單 B. 有序清單 C. 表格 D. 表單

5. `placeholder` 主要是：

A. 提示輸入格式 B. 取代 label C. 加密資料 D. 驗證密碼

6. HTML Entity 可用於表示：

A. 特殊字元 B. CPU C. DNS D. Cookie



7. `<textarea>` 適合：

A. 多行文字 B. 圖片 C. 導覽 D. 表格

8. `<section>` 通常代表：

A. 有主題性的內容區塊 B. 純裝飾 C. CSS D. JavaScript

9. SEO-friendly HTML 最重要的是：

A. 語意清楚 B. `<div>` 越多越好 C. 關鍵字重複 D. H1 越多越好

10. POST 相較 GET 通常較適合：

A. 提交表單資料 B. 載入圖片 C. CSS D. DNS

CSS / RWD | 11~18

11. `box-sizing: border-box` 會使 width 包含：

A. content only B. content + padding + border C. margin D. shadow

12. CSS specificity 通常較高的是：

A. element B. class C. ID D. universal

13. `display:none` 的元素：

A. 不占正常版面空間 B. 仍占空間 C. 只透明 D. 只移動

14. `opacity:0`：

A. 元素通常仍存在於版面中 B. 完全刪除元素 C. 移除 DOM D. 關閉 CSS

15. `overflow:hidden` 主要控制：

A. 溢出內容 B. 字型 C. DNS D. Cookie

16. `vw` 相對於：

A. viewport 寬度 B. 根字體 C. 父元素高度 D. CPU

17. Media Query 用於：

A. 條件式套用 CSS B. 建立資料庫 C. DNS D. JavaScript 編譯

18. Flexbox 最適合：

A. 一維配置 B. DNS C. SQL D. HTTP

JavaScript | 19~25

19.

```
let x = "5";  
console.log(x + 2);
```

A. 7 B. 52 C. Error D. undefined

20.

```
let x = 5;  
console.log(typeof x);
```

A. string B. boolean C. number D. object

21. Arrow Function 的語法之一：

A. `() => {}` B. `function => {}` C. `=> function()` D. `function() =>`

22. `textContent` 主要用於：

A. 讀取/設定文字內容 B. 執行 SQL C. 建立 DNS D. CSS 編譯

23. `classList.add()` 可以：

A. 增加 class B. 刪除 HTML C. 建立 Cookie D. 發送 HTTP

24. JSON 主要用於：

A. 資料交換 B. CSS Layout C. CPU D. DNS

25. `localStorage` 的資料通常：

A. 可在瀏覽器端持久保存 B. 只存在 CPU C. 只存在 1 秒 D. 是 DNS

Internet | 26~32

26. 301 表示：

A. 永久重新導向 B. 找不到 C. 伺服器錯誤 D. 未授權

27. 302 通常表示：

A. 暫時重新導向 B. 永久刪除 C. DNS 錯誤 D. 成功

28. 401 通常表示：

A. 未經驗證/授權資訊不足 B. 找不到 C. Server Error D. Success

29. 403 表示：

A. Forbidden B. Not Found C. OK D. Redirect

30. 500 表示：

A. Client 成功 B. Server Error C. DNS 成功 D. Redirect

31. CDN 的主要用途之一：

A. 透過分散節點提供內容 B. 取代 HTML C. 取代 CPU D. 建立密碼

32. TCP 的特色之一：

A. 提供可靠的傳輸機制 B. 是 HTML C. 是 CSS D. 是 DNS

Computer / OS | 33~37

33. 1 Byte：

A. 4 bits B. 8 bits C. 16 bits D. 32 bits

34. 64-bit OS 主要與：

A. 處理器架構及可處理位址空間相關 B. 螢幕尺寸 C. HTML D. CSS

35. Process 是：

A. 執行中的程式 B. 圖片 C. 網域 D. CSS

36. Virtual Memory 可以：

A. 使用儲存裝置協助記憶體管理 B. 增加螢幕解析度 C. 建立 HTML D. 建立 DNS

37. Browser 屬於：

A. Application Software B. CPU C. RAM D. Motherboard

Security | 38~41

38. XSS 的主要攻擊對象之一：

A. 使用者瀏覽器中的網頁環境 B. CPU C. RAM D. SSD

39. SQL Injection 主要透過：

A. 惡意 SQL 輸入 B. CSS C. 圖片 D. DNS

40. CSRF 利用的是：

A. 使用者已登入的信任狀態 B. CPU 故障 C. CSS D. DNS

41. Brute Force 是：

A. 暴力嘗試密碼 B. 圖片壓縮 C. DNS D. CSS

SEO / WCAG | 42~45

42. Sitemap 主要協助：

A. 搜尋引擎了解網站 URL 結構 B. 加密網站 C. 建立 CPU D. 儲存 Cookie

43. Canonical 主要用於：

A. 指定偏好的標準 URL B. 圖片壓縮 C. CSS D. 密碼

44. Keyboard Navigation 對：

A. Accessibility 很重要 B. DNS 很重要 C. CPU 很重要 D. SSD 很重要

45. CLS 主要與：

A. 版面位移 B. 密碼 C. DNS D. SQL

邏輯 | 46~48

46.

1、4、9、16、25、？

A. 30 B. 32 C. 36 D. 49

47.

5、10、20、40、？

A. 50 B. 60 C. 70 D. 80

48.

若所有 A 都是 B，而所有 B 都是 C，則：

A. 所有 A 都是 C

B. 所有 C 都是 A

C. 所有 B 都是 A

D. 無法判斷

AI | 49~50

49. AI 產生的 SQL 查詢：

- A. 一定安全
- B. 仍需檢查 Injection 風險
- C. 不需要測試
- D. 可以直接上線

50. AI Code Review 最重要的是：

- A. 只看格式
- B. 檢查邏輯、安全性、效能及相容性
- C. 只看程式長度
- D. 只看註解

答案

1 A 2 B 3 A 4 B 5 A

6 A 7 A 8 A 9 A 10 A

11 B 12 C 13 A 14 A 15 A

16 A 17 A 18 A

19 B 20 C 21 A 22 A 23 A

24 A 25 A

26 A 27 A 28 A 29 A 30 B

31 A 32 A

33 B 34 A 35 A 36 A 37 A

38 A 39 A 40 A 41 A

42 A 43 A 44 A 45 A

46 C 47 D 48 A

49 B 50 B



模擬考 03 | 情境應用測驗



難度：★★★★~★★★★★

這一回開始把「背知識」轉成「解決問題」。

1~10 | HTML

1. 網站 Header 中包含主要導覽，最適合：

A. `<nav>` B. `<footer>` C. `<aside>` D. `<span>`

2. 文章具有獨立閱讀價值，適合：

A. `<article>` B. `<div>` C. `<br>` D. `<span>`

3. 裝飾性圖片如果不需要提供資訊，alt 通常可：

A. 留空 B. 填關鍵字 C. 填 URL D. 填 CSS

4. 表單輸入框缺少 label，主要問題：

A. Accessibility B. DNS C. CPU D. CDN

5. 頁面主要標題通常應：

A. 清楚表達頁面主題 B. 隨意使用 C. 全部使用 H1 D. 不需要標題

6. Meta viewport 主要幫助：

A. 行動裝置顯示 B. SQL C. DNS D. Cookie

7. `href` 通常用於：

A. 超連結目的地 B. 圖片替代文字 C. 表單驗證 D. CSS

8. `src` 常見於：

A. 指定外部資源來源 B. CSS class C. ID D. 表單名稱

9. `required`：

A. 要求輸入 B. 建立連結 C. 建立圖片 D. 建立表格

10. Semantic HTML 的主要價值：

A. 提供結構與語意 B. 增加 RAM C. 加速 CPU D. 取代 CSS

11~18 | CSS / RWD

11. 網頁出現水平捲軸，首先應檢查：

A. 固定寬度、overflow、margin 等 B. DNS C. Cookie D. SQL

12. Flex 中 `flex-direction: column` :

A. 主軸變成垂直方向 B. 元素消失 C. Grid 化 D. 固定高度

13. `gap` 可用於 :

A. Flex/Grid 子元素間距 B. DNS C. 字型檔 D. HTTP

14. `z-index` 主要影響 :

A. 疊放順序 B. 字型大小 C. HTML 語意 D. DNS

15. `position: absolute` 通常相對於 :

A. 最近的定位祖先元素 B. 永遠 viewport C. CPU D. DNS

16. `max-width: 100%` 對圖片常可 :

A. 避免超出容器 B. 增加 DNS C. 建立 Cookie D. 取代 HTML

17. `rem` 適合 :

A. 建立較一致的字體尺度 B. DNS C. SQL D. Cookie

18. Mobile First 的優點之一 :

A. 優先處理小螢幕核心內容 B. 不需要 RWD C. 不需要 CSS D. 只做手機

19~25 | JavaScript

19.

```
console.log(typeof "Hello");
```

A. number B. string C. object D. boolean

20.

```
console.log(5 + "5");
```

A. 10 B. 55 C. Error D. 5

21.

```
let x = 10;  
x = 20;
```

結果 :

A. 合法 B. Error C. undefined D. null

22. `const` 主要不能：

A. 重新指定變數 B. 使用物件 C. 使用陣列 D. 存數字

23. `querySelector(".box")`：

A. 找第一個符合 selector 的元素 B. 找所有元素 C. 建立 CSS D. 建立 DNS

24. `preventDefault()` 常用於：

A. 阻止瀏覽器預設行為 B. 建立 DOM C. DNS D. JSON

25. `JSON.parse()`：

A. JSON 字串轉 JavaScript 資料 B. JS 轉 JSON C. CSS 轉 HTML D. DNS

26~32 | Internet

26. 使用者輸入網域後，DNS 主要協助：

A. 找到對應 IP B. 建立 HTML C. 壓縮 CSS D. 執行 JavaScript

27. Browser 發送 HTTP Request 後：

A. Server 回傳 Response B. CPU 關機 C. DNS 永久消失 D. CSS 失效

28. HTTP Request 通常包含：

A. Method、URL、Headers 等 B. CPU C. RAM D. SSD

29. 404 最可能表示：

A. URL 資源不存在 B. Server 成功 C. Authentication 成功 D. Redirect

30. HTTPS 通常建立於：

A. TLS B. CSS C. HTML D. SQL

31. Cache 的目的之一：

A. 減少重複下載資源 B. 增加密碼長度 C. 建立 HTML D. 執行 CPU

32. CDN 可以：

A. 讓使用者從較接近的節點取得內容 B. 取代 OS C. 取代 HTML D. 取代 DNS

33~37 | Computer

33. GPU 特別擅長：

A. 大量平行運算與圖形處理 B. DNS C. HTML D. Cookie

34. SSD 沒有傳統 HDD 的：

A. 機械旋轉磁碟結構 B. 儲存能力 C. 資料 D. 控制器

35. 作業系統主要負責：

A. 管理硬體與軟體資源 B. 只顯示網站 C. 只處理 DNS D. 只處理圖片

36. 檔案副檔名常用來：

A. 表示檔案類型 B. 表示 CPU 速度 C. 表示 IP D. 表示 RAM

37. Browser Cache 屬於：

A. 快取機制 B. CPU C. GPU D. Motherboard

38~41 | Security

38. 使用者輸入未經處理直接進入 `innerHTML`：

A. XSS 風險 B. DNS C. CDN D. RAM

39. 防禦 SQL Injection 的重要方法：

A. Parameterized Query B. 增加 CSS C. 增加圖片 D. 改 DNS

40. CSRF 防護常見方法：

A. CSRF Token B. 增加 RAM C. 修改 HTML title D. 壓縮圖片

41. Phishing 的主要目的：

A. 欺騙使用者取得敏感資訊 B. 加速網站 C. 壓縮 CSS D. 增加 SEO

42~45 | SEO / Web

42. Core Web Vitals 主要關注：

A. 使用者體驗與載入互動表現 B. CPU C. DNS D. 密碼

43. INP 主要與：

A. 互動回應性 B. 圖片 SEO C. DNS D. SQL

44. Structured Data 可以：

A. 提供搜尋引擎理解頁面內容的結構化資訊 B. 增加 RAM C. 取代 HTML D. 建立 Cookie

45. Internal Link 有助於：

A. 網站內容與頁面之間的導覽 B. CPU C. DNS D. RAM

46~48 | 邏輯

46.

2、6、12、20、30、？

A. 36 B. 40 C. 42 D. 44

47.

100、50、25、12.5、？

A. 6.25 B. 7.5 C. 10 D. 5

48.

3 個工程師 6 天完成工作，若效率相同，6 個工程師需要：

A. 2 天 B. 3 天 C. 6 天 D. 12 天

49~50 | AI

49. AI 產生網站後發現手機版出現橫向捲軸，最佳做法：

- A. 相信 AI
- B. 檢查 CSS、固定寬度與 overflow
- C. 刪掉手機版
- D. 關閉瀏覽器

50. AI 產生一段可以執行但存在 XSS 風險的 JavaScript：

- A. 因為能執行所以安全
- B. 必須進行安全 Review
- C. 不需要修改
- D. AI 自動負責法律責任

答案

1 A 2 A 3 A 4 A 5 A
6 A 7 A 8 A 9 A 10 A

11 A 12 A 13 A 14 A 15 A
16 A 17 A 18 A

19 B 20 B 21 A 22 A 23 A
24 A 25 A

26 A 27 A 28 A 29 A 30 A
31 A 32 A

33 A 34 A 35 A 36 A 37 A

38 A 39 A 40 A 41 A

42 A 43 A 44 A 45 A

46 C 47 A 48 B

49 B 50 B



模擬考 04 | 陷阱與挑戰

難度：★★★★～★★★★★

這一回最重要的不是「背答案」，而是辨識相似概念。

1～10 | HTML

1. `alt` 與 `title` 最大的差異之一：

- A. alt 主要提供圖片替代文字
- B. title 一定取代 alt
- C. alt 是 CSS
- D. title 是 JavaScript

2. `<section>` 與 `<div>` 的主要差異：

- A. section 具有語意性主題區塊
- B. div 一定不能使用 class
- C. section 是 CSS
- D. div 是 JavaScript



3. `<strong>` 與 `<b>` :

- A. strong 具有較明確語意
- B. 完全沒有任何差異
- C. b 是表單
- D. strong 是圖片

4. `<em>` 與 `<i>` :

- A. em 具有強調語意
- B. i 一定不能使用
- C. em 是 CSS
- D. 完全相同用途

5. `id` 與 `class` :

- A. id 通常唯一，class 可重複使用
- B. 完全相同
- C. class 必須唯一
- D. id 只能給圖片

6. 表單 `label` 最佳做法：

- A. 與 input 建立關聯
- B. 只使用 placeholder
- C. 不需要 label
- D. 放進 CSS

7. `<main>` 一般：

- A. 一頁主要內容通常只有一個主要 main
- B. 可以無限使用
- C. 只能放 Footer
- D. 只能放圖片

8. `<aside>` 通常表示：

- A. 與主要內容相關但非核心內容
- B. 主要標題
- C. CSS
- D. Script

9. SEO Heading 最合理：

- A. 具有清楚階層
- B. 大量 H1
- C. 全部 H6
- D. 不需要標題

10. GET 不適合用來：

A. 傳送高度敏感資料 B. 取得資料 C. 查詢資源 D. URL 導覽

11~18 | CSS

11. specificity 中通常最高：

A. element B. class C. ID D. universal

12. margin：

A. 元素外部空間 B. 元素內容空間 C. 字體 D. border

13. padding：

A. 內容與 border 之間 B. 元素外部 C. viewport D. CPU

14. visibility:hidden 與 display:none 的主要差異：

A. 前者通常仍保留版面空間
B. 完全相同
C. 前者刪除 DOM
D. 後者透明

15. opacity:0：

A. 元素仍可能接收事件 B. 一定等同 display:none C. 刪除元素 D. 變成文字

16. justify-content 與 align-items：

A. 分別控制 Flex 主軸與交叉軸 B. 完全相同 C. 都控制字體 D. 都是 HTML

17. rem vs em：

A. rem 通常相對 root，em 依上下文而定
B. 完全相同
C. 都是 px
D. 都是 vw

18. Grid 最適合：

A. 二維版面 B. DNS C. HTTP D. Cookie

19~25 | JavaScript

19.

```
console.log(null == undefined);
```

A. true B. false C. Error D. 0

20.

```
console.log(null === undefined);
```

A. true B. false C. null D. undefined

21.

```
const user = {name: "A"};  
user.name = "B";
```

- A. 可以修改物件屬性
- B. 一定 Error
- C. const 不能有物件
- D. 物件會變 null

22. `var` 與 `let` 的重要差異之一：

- A. scope 行為不同
- B. 都完全相同
- C. let 不能存數字
- D. var 不能函式

23. `innerHTML` 若處理不可信任輸入，需注意：

- A. XSS B. DNS C. RAM D. GPU

24. `textContent` 相較直接使用 `innerHTML`：

- A. 更適合單純文字內容 B. 會執行 SQL C. 建立 CSS D. 建立 DNS

25. `map()` 通常：

- A. 建立轉換後的新陣列 B. 刪除 DOM C. 建立 Cookie D. 建立 IP

26~32 | Internet

26. DNS 是：

A. Domain Name System B. Data Network Security C. Digital Node Service D. Dynamic Name Server

27. HTTPS 與 HTTP 的主要差異之一：

A. TLS 加密保護 B. HTML 完全不同 C. CSS 不同 D. CPU 不同

28. 401 與 403：

A. 401 偏向 authentication，403 表示禁止存取

B. 完全相同

C. 401 是 Not Found

D. 403 是 Success

29. 301 與 302：

A. 永久與暫時重新導向的典型差異

B. 都是 Error

C. 都是 DNS

D. 都是 Security

30. Cookie 與 Session：

A. 都可能用於維持狀態，但實作位置與機制可不同

B. 完全相同

C. 都只能存 CSS

D. 都是 DNS

31. Port 的作用之一：

A. 協助識別主機上的網路服務

B. 儲存圖片

C. 取代 IP

D. 取代 DNS

32. IPv6 位址長度：

A. 32 bits B. 64 bits C. 128 bits D. 256 bits

33~37 | Computer

33. Cache 的核心概念：

A. 暫存常用資料以提高效率 B. 永久備份 C. CPU D. OS

34. Virtual Memory：

A. 使用儲存空間協助記憶體管理 B. 等於 RAM C. 等於 CPU D. 是瀏覽器

35. SSD :

A. Flash Storage B. CPU C. GPU D. OS

36. OS :

A. 管理硬體與軟體資源 B. 只負責瀏覽器 C. 只負責 HTML D. 只負責 DNS

37. Browser :

A. 可以解析 HTML/CSS/JS B. 是 CPU C. 是 SSD D. 是 RAM

38~41 | Security

38. XSS 與 SQL Injection :

- A. XSS 主要涉及網頁腳本注入；SQL Injection 涉及資料庫查詢
- B. 完全相同
- C. 都是 DNS
- D. 都是硬體攻擊

39. HttpOnly Cookie :

- A. 限制 JavaScript 直接讀取 Cookie
- B. 加密整個網站
- C. 防止所有 CSRF
- D. 取代 HTTPS

40. Secure Cookie :

- A. 通常要求透過 HTTPS 傳送 B. 只允許 HTTP C. 禁止 Cookie D. 取代 MFA

41. SameSite :

- A. 可協助降低某些跨站 Cookie 風險
 - B. 是 CSS
 - C. 是 DNS
 - D. 是 CPU
-

42~45 | SEO / WCAG

42. robots.txt :

- A. 可提供爬蟲存取規則 B. 等於 sitemap C. 加密網站 D. 建立 DNS

43. sitemap.xml :

A. 提供網站 URL 資訊 B. 取代 robots.txt C. 取代 HTML D. 加密資料

44. WCAG :

A. Web Content Accessibility Guidelines B. Web CSS Application Guide C. Website Cache Guide D. 無關網頁

45. ARIA :

A. 可提供輔助技術額外語意 B. 取代所有 Semantic HTML C. 是 CSS D. 是 DNS

46~48 | 邏輯

46.

1、2、6、24、120、？

A. 240 B. 360 C. 720 D. 840

47.

2、3、5、8、13、？

A. 18 B. 20 C. 21 D. 22

48.

一個商品原價 1000 元，打 8 折後再打 9 折，最後價格：

A. 720 B. 800 C. 820 D. 900

49~50 | AI

49. AI 告訴你：

「這段程式 100% 安全。」

最正確的態度：

A. 直接相信

B. 仍應自行進行 Security Review

- C. 不需測試
- D. 直接部署

50. AI 產生的 RWD 在 375px 正常，但 320px 溢出：

- A. 代表所有 RWD 都失敗
- B. 應找出特定 breakpoint / fixed width / overflow 問題
- C. 刪除手機版
- D. 改成 1920px

答案

1 A 2 A 3 A 4 A 5 A
6 A 7 A 8 A 9 A 10 A

11 C 12 A 13 A 14 A 15 A
16 A 17 A 18 A

19 A 20 B 21 A 22 A 23 A
24 A 25 A

26 A 27 A 28 A 29 A 30 A
31 A 32 C

33 A 34 A 35 A 36 A 37 A

38 A 39 A 40 A 41 A

42 A 43 A 44 A 45 A

46 C 47 C 48 A

49 B 50 B

模擬考 05 | FINAL MOCK

最終實戰測驗

難度：★★★★~★★★★★

這一回的定位：

把自己當成正式甄試考生。

建議：

60 分鐘倒數開始後再看題目。

不要查 Google。

不要問 ChatGPT。

不要看教材。

第 1~10 題 | HTML

1. 一個企業首頁包含 Logo、主要導覽、主要內容與頁尾，最合理的語意結構是：

- A. 全部 `<div>`
- B. `<header>` + `<nav>` + `<main>` + `<footer>`
- C. 全部 `<span>`
- D. 全部 `<p>`

2. `<article>` 最適合：

- A. 獨立內容 B. CSS C. DNS D. Footer

3. 圖片需要提供替代文字時：

- A. alt B. title C. id D. class

4. 表單 Accessibility 最重要的基本措施之一：

- A. label B. placeholder C. color D. border

5. `required`：

- A. 必填 B. 唯讀 C. 隱藏 D. 加密

6. `meta viewport`：

- A. 協助行動裝置正確顯示 B. DNS C. SQL D. Cookie

7. `<strong>`：

- A. 語意上的重要性 B. 只有粗體外觀 C. 圖片 D. 連結

8. `id`：

- A. 通常應唯一 B. 必須重複 C. 只能 CSS D. 不能 JS

9. `<ol>` :

A. 有序列表 B. 無序列表 C. 表格 D. 表單

10. GET 與 POST :

A. 都是 HTTP Method B. 都是 CSS C. 都是 DNS D. 都是 OS

第 11~18 題 | CSS / RWD

11. 若 :

```
.box {  
  width: 300px;  
  padding: 20px;  
}
```

沒有 border-box , content-box 下總寬度 :

A. 300 B. 320 C. 340 D. 360

12. `box-sizing: border-box` :

A. width 包含 padding/border B. width 只含 content C. 增加 margin D. 隱藏元素

13. Flex 主軸 :

A. 由 flex-direction 決定 B. 永遠水平 C. 永遠垂直 D. 由 z-index 決定

14. `justify-content` :

A. 主軸對齊 B. 文字粗細 C. 邊框 D. opacity

15. `align-items` :

A. 交叉軸對齊 B. DNS C. HTML D. Cookie

16. Mobile First :

A. 先小螢幕 B. 先桌面 C. 只手機 D. 不需 RWD

17. `display: none` :

A. 不占正常版面空間 B. 透明但占空間 C. 只改顏色 D. 不存在 CSS

18. `@media` :

A. 條件式 CSS B. JavaScript C. DNS D. SQL

第 19~25 題 | JavaScript

19.

```
10 == "10"
```

A. true B. false C. Error D. null

20.

```
10 === "10"
```

A. true B. false C. 10 D. Error

21.

```
const arr = [1, 2, 3];  
arr.push(4);
```

A. 可以 B. 一定 Error C. arr 變 null D. 無法使用

22.

```
console.log(typeof true);
```

A. boolean B. string C. number D. object

23. `querySelector()` :

A. DOM Selector B. SQL C. DNS D. CSS compiler

24. `addEventListener()` :

A. 事件監聽 B. DNS C. HTTP Server D. OS

25. 將使用者輸入當純文字顯示，通常較適合：

A. `textContent` B. `innerHTML` C. `eval` D. `document.write`

第 26~32 題 | Internet

26. 使用者輸入：

```
https://www.example.com/products/index.html
```

其中 `www.example.com` 是：

A. Domain / Host B. Protocol C. Path D. Query

27. DNS :

A. Domain → IP 等解析 B. HTML → CSS C. JS → SQL D. Image → PNG

28. 200 :

A. OK B. Not Found C. Unauthorized D. Server Error

29. 404 :

A. Not Found B. OK C. Forbidden D. Redirect

30. 500 :

A. Server Error B. OK C. Not Found D. Redirect

31. HTTPS :

A. HTTP over TLS 的安全通訊方式之一 B. CSS C. HTML5 D. DNS

32. CDN :

A. 分散式內容傳遞 B. 作業系統 C. CPU D. RAM

第 33~37 題 | Computer / OS

33. CPU :

A. 執行指令與運算 B. 永久儲存 C. 網域 D. Browser

34. RAM :

A. 主要記憶體 B. 永久 SSD C. CPU D. GPU

35. SSD :

A. 儲存裝置 B. OS C. Browser D. CPU

36. Linux :

A. Operating System B. Browser C. CPU D. Database

37. Browser :

A. 解析並呈現 Web 內容 B. RAM C. SSD D. GPU

第 38~41 題 | Security

38. 未處理輸入進入 innerHTML :

A. XSS B. SQL Injection C. DNS D. CDN

39. SQL 查詢直接串接使用者輸入：

A. SQL Injection 風險 B. CLS C. LCP D. WCAG

40. HTTPS：

A. 保護傳輸中的資料 B. 防止所有攻擊 C. 取代密碼 D. 取代 MFA

41. MFA：

A. 多因素驗證 B. 多檔案 C. 多網域 D. 多媒體

第 42~45 題 | SEO / WCAG

42. `<title>`：

A. 頁面標題 B. 圖片替代文字 C. DNS D. Cookie

43. `alt`：

A. 圖片替代文字 B. CSS C. DNS D. HTTP

44. WCAG：

A. Web Accessibility B. Web Database C. Web DNS D. Web Cache

45. LCP：

A. Largest Contentful Paint B. Login Control Protocol C. Local Cache Process D. Link Control Page

第 46~48 題 | 數理邏輯

46.

2、6、12、20、30、？

A. 36 B. 40 C. 42 D. 44

47.

1、3、6、10、15、？

A. 18 B. 20 C. 21 D. 25

48.

某商品 500 元，漲價 20%：

A. 520 B. 550 C. 600 D. 620

第 49~50 題 | AI × Front-End

49.

ChatGPT 幫你產生 JavaScript，程式可以正常執行，但沒有做 Input Validation。

最適當的做法：

- A. 直接上線
- B. 進行安全性與輸入驗證檢查
- C. 因為 AI 產生所以安全
- D. 刪除 JavaScript

50.

AI 產生了一個漂亮的網站，但 Lighthouse 顯示：

- LCP 很差
- CLS 很高
- Mobile 有 Horizontal Overflow

最佳做法：

- A. 因為視覺漂亮，所以直接上線
- B. 進行 Performance、RWD 與版面穩定性改善
- C. 關閉 Lighthouse
- D. 只增加圖片

FINAL MOCK | 答案

1 B 2 A 3 A 4 A 5 A

6 A 7 A 8 A 9 A 10 A

11 C 12 A 13 A 14 A 15 A

16 A 17 A 18 A

19 A 20 B 21 A 22 A 23 A

24 A 25 A

26 A 27 A 28 A 29 A 30 A
31 A 32 A

33 A 34 A 35 A 36 A 37 A

38 A 39 A 40 A 41 A

42 A 43 A 44 A 45 A

46 C 47 C 48 C

49 B 50 B

五、5 回模擬考成績追蹤表

建議你不要只記「總分」。

請建立：

回次	HTML	CSS	JS	網路	OS	資安	SEO	邏輯	AI	總分
Mock 01	/10	/8	/7	/7	/5	/4	/3	/3	/2	/50
Mock 02	/10	/8	/7	/7	/5	/4	/3	/3	/2	/50
Mock 03	/10	/8	/7	/7	/5	/4	/3	/3	/2	/50
Mock 04	/10	/8	/7	/7	/5	/4	/3	/3	/2	/50
Mock 05	/10	/8	/7	/7	/5	/4	/3	/3	/2	/50

六、不要只看總分

例如：

Mock 01

42 / 50

看起來很好。

但如果：

HTML	9/10
CSS	8/8
JavaScript	4/7
Internet	6/7
OS	5/5
Security	2/4
SEO	3/3
Logic	3/3
AI	2/2

真正的問題是：

JavaScript + 資安

所以 Phase 12 的目的不是單純追求：

「我要考 90 分。」

而是：

「我要知道哪 10 分為什麼丟掉。」

七、5 回模擬考的晉級制度

Mock 01

基準測驗

目標：

70 分以上

Mock 02

標準測驗

目標：

75 分以上

Mock 03

情境測驗

目標：

80 分以上

Mock 04

陷阱測驗

目標：

85 分以上

Mock 05

FINAL MOCK

目標：

90 分以上

八、最終能力分級

最終平均

評價

90~100

 甄試高準備度

85~89

 非常穩定

80~84

 具競爭力

75~79

 基礎穩定但仍需補強

70~74

 臨界區

60~69

 高風險

最終平均

評價

<60

● 必須重新補課

九、Phase 12 的「考場時間策略」

50 題 / 60 分鐘。

平均：

72 秒／題

但是不要每題平均分配。

推薦：

第一輪 | 40 分鐘

快速完成所有有把握的題目。

看到：

一眼就會

直接作答。

第二輪 | 15 分鐘

處理：

- JavaScript
- CSS 判讀
- Internet 情境
- Security
- 邏輯題

第三輪 | 5 分鐘

專門檢查：

- 是否漏題
- 題號是否填錯
- `==` / `===`
- `401` / `403`
- `404` / `500`
- `margin` / `padding`
- `alt` / `title`
- `id` / `class`
- `XSS` / `SQL Injection`
- `LCP` / `CLS` / `INP`

十、甄試最重要的「最後 20 個陷阱」

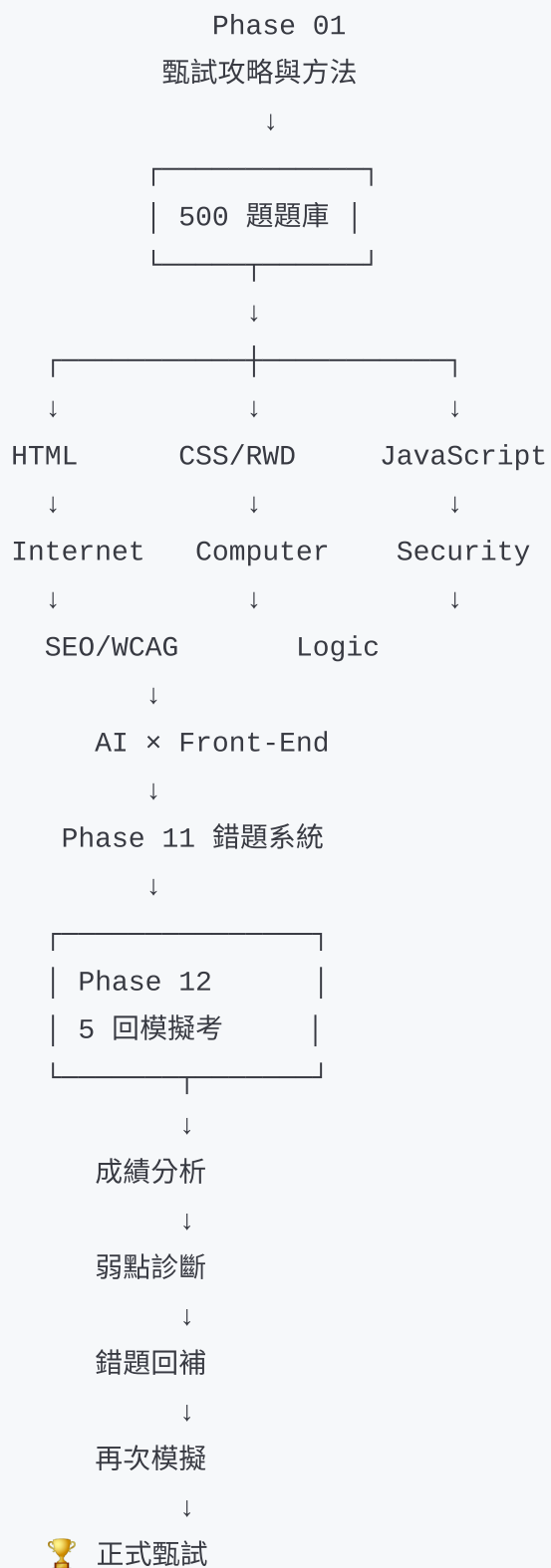
進考場前務必能快速回答：

1. `id` $\neq$ `class`
2. `alt` $\neq$ `title`
3. `strong` $\neq$ `b`
4. `em` $\neq$ `i`
5. `section` $\neq$ `div`
6. `margin` $\neq$ `padding`
7. `justify-content` $\neq$ `align-items`
8. `display:none` $\neq$ `visibility:hidden`
9. `rem` $\neq$ `em`
10. `Mobile First` $\neq$ 只做手機
11. `==` $\neq$ `===`
12. `let` $\neq$ `var`
13. `undefined` $\neq$ `null`
14. `innerHTML` $\neq$ `textContent`
15. `const` $\neq$ immutable object
16. `DNS` $\neq$ `IP`
17. `401` $\neq$ `403`
18. `404` $\neq$ `500`

- 19. XSS ≠ SQL Injection
- 20. AI Generated ≠ Verified

十一、Phase 12 最終「甄試作戰系統」

整套課程至此形成：



Phase 12 最終目標

你不是要做到：

「我看過 500 題。」

而是要做到：

「我能在 60 分鐘內完成 50 題，而且遇到陌生題型也能利用觀念推理出答案。」

尤其要把前面 10 個 Phase 最終整合成四種能力：

① 知識

我知道答案。

② 理解

我知道為什麼。

③ 判斷

題目換個方式問，我仍然知道。

④ 應用

遇到真實網站問題，我知道怎麼處理。

而 **AI × Front-End** 再加上一層：

AI 說的答案，我知道如何驗證，而不是盲目相信。

你的整套甄試訓練路線至此完成

Phase 01~10 | 學習與建立知識

→ **Phase 11** | 錯題訓練

→ **Phase 12** | 5 回完整模擬考

下一個最適合銜接的是：

Phase 13 | 「甄試 7 天最後衝刺計畫」

將 500 題、錯題庫、5 回模擬考濃縮成 考前 7 天每日任務、每日複習範圍、必背觀念、陷阱題、每日 50 題測驗與最後一天 A4 速讀版，讓整套教材真正進入「考前作戰模式」。

2026/8/18 下午2:15:30





Phase 13 | 考前 7 天衝刺課程

台灣就業通－前端網頁技術與 AI 應用甄試 | Final 7-Day Bootcamp

目標不是再學更多，而是把已經學過的 500 題，轉化成考場上的「快速判斷能力」。

Phase 13 是整套課程最後一個階段：

500 題題庫 → 10 大主題 → 錯題系統 → 5 回模擬考 → 7 天最後衝刺 → 正式甄試

一、7 天衝刺總策略

天數	主題	核心任務	建議時間
Day 1	HTML	100 題+HTML陷阱	90 分鐘
Day 2	CSS / RWD	80 題+畫面判讀	90 分鐘
Day 3	JavaScript	70 題+程式判讀	120 分鐘
Day 4	Internet	70 題+瀏覽器運作流程	90 分鐘
Day 5	Computer + Security	90 題+資安情境	120 分鐘
Day 6	SEO / WCAG + AI	30 題+AI情境	90 分鐘
Day 7	FINAL DAY	總複習+模擬考+心理調整	120 分鐘

二、Day 1 | HTML 最後衝刺

今日目標

把 HTML 從：

「知道標籤」

提升到：

「看到題目就知道哪個 HTML 元素最適合。」

① 必背 HTML 結構

```
<!DOCTYPE html>
<html lang="zh-Hant">

<head>
  <meta charset="UTF-8">
  <meta name="viewport"
    content="width=device-width, initial-scale=1.0">

  <title>網站標題</title>
</head>

<body>

  <header>
    <nav>網站導覽</nav>
  </header>

  <main>

    <section>
      <article>
        主要內容
      </article>
    </section>

    <aside>
      側邊資訊
    </aside>

  </main>

  <footer>
    頁尾
  </footer>

</body>
</html>
```

② HTML 必考對照表

題目關鍵字	答案
網站導覽	<code><nav></code>
頁首	<code><header></code>
主要內容	<code><main></code>
主題區塊	<code><section></code>
獨立文章	<code><article></code>
側邊內容	<code><aside></code>
頁尾	<code><footer></code>
圖片替代文字	<code>alt</code>
圖片說明	<code><figcaption></code>
有序清單	<code></code>
無序清單	<code></code>
表單標籤	<code><label></code>
必填	<code>required</code>
輸入提示	<code>placeholder</code>
唯一識別	<code>id</code>
共用樣式	<code>class</code>

③ 今日 10 大陷阱

```
alt ≠ title
id ≠ class
section ≠ div
article ≠ section
strong ≠ b
```

```
em ≠ i  
placeholder ≠ label  
GET ≠ POST  
required ≠ placeholder  
main ≠ body
```

★ 必記

Semantic HTML 是「語意」，不是「外觀」。

Day 1 自我測驗

看到以下題目：

「網站主要導覽選單應使用哪個元素？」

不要思考超過 3 秒。

答案：

```
<nav>
```

三、Day 2 | CSS / RWD 最後衝刺

🎯 今日目標

從：

「背 CSS」

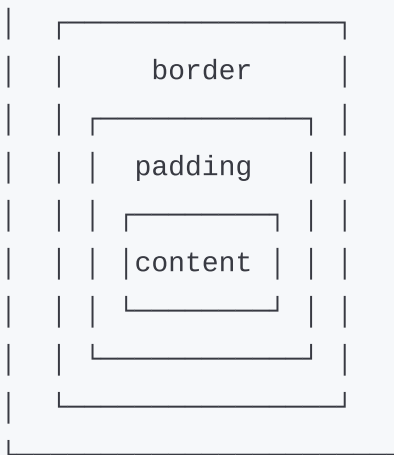
提升到：

「看 CSS 就能預測畫面。」

① Box Model

一定要熟：

```
|-----|  
|         |  
| margin |  
|         |  
|-----|
```



記憶：

Margin 在外、***Padding*** 在內。

② Flex 三大觀念

```
display: flex;
justify-content: center;
align-items: center;
```

記：

justify = 主軸

align = 交叉軸

③ RWD 必背

```
@media (max-width: 768px) {
  ...
}
```

核心：

不同螢幕尺寸 → 套用不同 ***CSS***

④ Units

單位	意義
px	固定像素
%	相對父元素
em	相對字體尺寸/上下文
rem	相對 root font-size
vw	viewport 寬度
vh	viewport 高度

⑤ CSS 十大陷阱

```
margin ≠ padding
width ≠ max-width
px ≠ %
em ≠ rem
vw ≠ vh
justify-content ≠ align-items
display:none ≠ visibility:hidden
opacity:0 ≠ display:none
position:absolute ≠ fixed
Flex ≠ Grid
```

Day 2 Challenge

看到：

```
.container {
  display: flex;
  justify-content: space-between;
  align-items: center;
}
```

必須立即知道：

Flex 水平/主軸分散，交叉軸置中。

四、Day 3 | JavaScript 最後衝刺

★ 這是最重要的一天

因為 JavaScript 題目最容易從「背答案」變成「真的要理解」。

① 變數

```
var  
let  
const
```

考試重點：

優先理解 *let* / *const*。

② 型別

```
String  
Number  
Boolean  
Array  
Object  
null  
undefined
```

③ 最重要陷阱

```
10 == "10"
```

答案：

true

因為可能發生型別轉換。


```
10 === "10"
```

答案：

false

因為：

值相同，但型別不同。

④ DOM

必背：

```
document.getElementById()  
document.querySelector()  
document.querySelectorAll()
```

⑤ Event

```
element.addEventListener("click", function () {  
    ...  
});
```

記：

Event = 使用者或瀏覽器發生的事件。

例如：

- click
- submit
- change
- input
- mouseover
- keydown

⑥ innerHTML vs textContent

innerHTML

可以解析 HTML。

textContent

主要處理純文字。

因此：

不可信任使用者輸入 → 小心 *innerHTML*。

這會連接到：

XSS

⑦ JavaScript 必背

== 寬鬆相等

=== 嚴格相等

let 可重新賦值

const 不可重新指定變數

Array []

Object {}

querySelector()

addEventListener()

innerHTML

textContent

JSON.parse()

JSON.stringify()

localStorage

Day 3 Challenge

看到：

```
let x = 10;  
let y = "10";  
  
console.log(x == y);  
console.log(x === y);
```

必須秒答：

```
true  
false
```

五、Day 4 | Internet 最後衝刺

🎯 今日核心

建立一條完整的：

「輸入網址 → 網頁出現」

流程。

```
使用者輸入 URL  
↓  
Browser  
↓  
DNS  
↓  
取得 IP  
↓  
建立網路連線  
↓  
HTTPS / TLS  
↓  
HTTP Request  
↓
```

Web Server
↓
HTTP Response
↓
HTML
↓
CSS / JS / Images
↓
Browser Render
↓
網頁出現

① HTTP Status Code 必背

Code	意義
200	OK
301	Permanent Redirect
302	Temporary Redirect
400	Bad Request
401	Unauthorized / Authentication required
403	Forbidden
404	Not Found
500	Internal Server Error

★ 最容易錯

401

身分驗證問題

403

你被禁止存取

404

找不到資源

500

Server 出錯

② URL

例如：

```
https://www.example.com/products/index.html
```

拆解：

```
https
```

```
↓
```

```
Protocol
```

```
www.example.com
```

```
↓
```

```
Domain / Host
```

```
/products/index.html
```

```
↓
```

```
Path
```

③ DNS

一句話：

DNS 把人容易記的網域名稱解析成 **IP** 位址。

④ Cookie / Session / Cache

名稱

核心概念

Cookie

瀏覽器端儲存的小型資料

名稱	核心概念
Session	維持使用者狀態的機制
Cache	暫存資源以提升效率

六、Day 5 | Computer + Security

Part A | Computer

CPU

執行指令與運算

RAM

暫時性主要記憶體

SSD

非揮發性儲存裝置

OS

管理硬體與軟體資源

Browser

解析與呈現 Web 內容

必背比較

元件	核心功能
CPU	運算
RAM	暫存
SSD	儲存
GPU	圖形/平行運算

元件	核心功能
OS	資源管理
Browser	Web 呈現

Part B | 資訊安全

這裡要建立：

「攻擊 → 原因 → 防禦」

思維。

XSS

攻擊

將惡意 JavaScript 注入網頁。

防禦

- Output Encoding
- Input Validation
- 避免不必要的 `innerHTML`

SQL Injection

攻擊

操控 SQL 查詢。

防禦

Parameterized Query / Prepared Statement

CSRF

利用使用者已登入的狀態發送未授權請求。

常見防禦：

Password

不要使用：

123456
password
生日
手機號碼

應：

長、難猜、每個服務最好獨立。

並搭配：

MFA

Cookie Security

一定記：

HttpOnly
Secure
SameSite

七、Day 6 | SEO + WCAG + AI

Part A | SEO

必背：

<title>
Meta Description
Heading
Semantic HTML
Canonical
robots.txt

sitemap.xml
Structured Data
Schema.org
Internal Link
Image SEO

Part B | WCAG

核心：

讓更多人能夠使用網站。

必記：

alt

圖片替代文字。

label

表單欄位標籤。

Keyboard Navigation

不能只靠滑鼠。

Focus

鍵盤操作時要看得焦點。

Color Contrast

文字與背景要有足夠對比。

ARIA

提供輔助技術額外語意。

Part C | Core Web Vitals

目前考試應熟悉：

LCP

INP

CLS

LCP

最大內容元素載入表現

INP

互動回應性

CLS

版面穩定性／位移

Part D | AI × Front-End

這是本甄試的特色。

不要把 AI 題目當成：

「哪一個 AI 工具最厲害？」

真正要考的是：

AI + 專業判斷

AI 產生程式後

永遠遵循：

AI Generate

↓

Human Review

↓

Test

↓

Security Check



Performance Check



Browser Compatibility



Deploy

★ AI 最重要觀念

AI 可以加速寫程式，但不能取代工程師的驗證與判斷。

八、Day 7 | FINAL DAY

🏆 最後一天不要再大量學新東西

這一天只做：

- ① 錯題
- ② 陷阱
- ③ 模擬考
- ④ 速記

上午 | 60 分鐘

完成：

FINAL MOCK

50 題。

規則：

60 分鐘內完成。

中午 | 30 分鐘

只看：

自己的錯題

不要重新讀整本教材。

下午 | 30 分鐘

快速複習：

HTML
CSS
JS
HTTP
Security
SEO
Logic
AI

晚上 | 20 分鐘

只看下面這份：

「考前 20 分鐘速讀表」

九、考前 20 分鐘速讀表

HTML

<header>	頁首
<nav>	導覽
<main>	主要內容
<section>	主題區塊
<article>	獨立內容
<aside>	側邊內容
<footer>	頁尾
alt	替代文字
label	表單標籤
required	必填

placeholder 提示

id 通常唯一

class 可重複

CSS

margin 外

padding 內

content

padding

border

margin

Flex 一維

Grid 二維

justify-content 主軸

align-items 交叉軸

rem root

em context

vw viewport width

vh viewport height

display:none

visibility:hidden

opacity:0

JavaScript

= 賦值

== 寬鬆相等

=== 嚴格相等

let

const

var

String

Number

Boolean

Array

Object

querySelector()

getElementById()

addEventListener()

innerHTML

textContent

JSON.parse()

JSON.stringify()

localStorage

Internet

DNS	Domain → IP
HTTP	Web communication
HTTPS	HTTP + TLS

200	OK
301	永久轉址
302	暫時轉址
400	Bad Request
401	Authentication
403	Forbidden
404	Not Found
500	Server Error

Cookie
Session
Cache
CDN
TCP/IP

Port
IPv4
IPv6

資安

XSS

→ Script Injection

SQL Injection

→ Database Query Injection

CSRF

→ 利用登入狀態

Phishing

→ 欺騙使用者

MFA

→ 多因素驗證

HttpOnly

→ 限制 JS 存取 Cookie

Secure

→ HTTPS 傳送

SameSite

→ 降低跨站 Cookie 風險

SEO / Web

title

meta description

semantic HTML

canonical

robots.txt

sitemap

Schema.org

structured data
internal link

WCAG
alt
label
keyboard
focus
contrast
ARIA

LCP
INP
CLS

AI

AI ≠ 永遠正確

Generate

↓

Review

↓

Test

↓

Security

↓

Performance

↓

Compatibility

↓

Deploy

十、考場「秒答表」

看到關鍵字，直接聯想：

關鍵字	秒答
導覽	nav
主要內容	main
獨立文章	article
替代文字	alt
表單標籤	label
必填	required
外距	margin
內距	padding
一維	Flex
二維	Grid
主軸	justify
交叉軸	align
嚴格相等	===
DOM 選擇	querySelector
事件	addEventListener
網域解析	DNS
加密傳輸	HTTPS/TLS
成功	200
永久轉址	301
找不到	404
Server Error	500
腳本注入	XSS
SQL 攻擊	SQL Injection

關鍵字	秒答
跨站請求	CSRF
無障礙	WCAG
圖片替代文字	alt
最大內容載入	LCP
互動反應	INP
版面位移	CLS
AI 程式	Review + Test

十一、最後 7 天的「每日任務卡」

DAY 1

HTML

- ☐ 複習 100 題
- ☐ 重做 HTML 錯題
- ☐ 背 10 大 HTML 陷阱
- ☐ 完成 20 題快速測驗

DAY 2

CSS / RWD

- ☐ 複習 80 題
- ☐ Box Model
- ☐ Flex
- ☐ Grid
- ☐ Media Query
- ☐ 20 題 CSS 判讀

DAY 3

JavaScript

- ☐ 複習 70 題
 - ☐ `==` / `===`
 - ☐ `let` / `const` / `var`
 - ☐ DOM
 - ☐ Event
 - ☐ Array / Object
 - ☐ 20 題程式判讀
-

DAY 4

Internet

- ☐ HTTP
 - ☐ HTTPS
 - ☐ DNS
 - ☐ URL
 - ☐ Status Code
 - ☐ Cookie / Session / Cache
 - ☐ URL → 網頁完整流程
-

DAY 5

Computer + Security

- ☐ CPU / RAM / SSD
- ☐ OS
- ☐ Browser
- ☐ XSS
- ☐ SQL Injection
- ☐ CSRF
- ☐ MFA

- ☐ Cookie Security
-

DAY 6

SEO + WCAG + AI

- ☐ SEO
 - ☐ Schema.org
 - ☐ robots.txt
 - ☐ sitemap
 - ☐ WCAG
 - ☐ ARIA
 - ☐ LCP
 - ☐ INP
 - ☐ CLS
 - ☐ AI Code Review
-

DAY 7

FINAL DAY

- ☐ FINAL MOCK 50 題
 - ☐ 計算分數
 - ☐ 分析錯題
 - ☐ 複習 20 大陷阱
 - ☐ 看 A4 速讀表
 - ☐ 不再大量學新知識
 - ☐ 正常休息
-

十二、最後的「甄試通關公式」

把整個課程濃縮成一句話：

HTML 看語意、CSS 看版面、JavaScript 看邏輯、Internet 看流程、OS 看功能、Security 看風險、SEO 看搜尋、WCAG 看使用者、Logic 看規律、AI 看驗證。

再濃縮成：

「看懂 → 判斷 → 排除 → 驗證」

遇到不會的題目，不要慌。

採用：

Step 1 | 先判斷科目

HTML？

CSS？

JS？

Internet？

Security？

Step 2 | 找關鍵字

例如：

「圖片替代文字」

立刻想到：

alt

Step 3 | 排除明顯錯誤

例如：

DNS 是不是 CSS？

不是。

Step 4 | 剩下兩個答案時，用原理判斷

這正是甄試最重要的能力。

Phase 13 完成後的最終學習架構

Phase 01
甄試攻略
↓
Phase 02
HTML 100題
↓
Phase 03
CSS/RWD 80題
↓
Phase 04
JavaScript 70題
↓
Phase 05
Internet 70題
↓
Phase 06
Computer/OS 50題
↓
Phase 07
Security 40題
↓
Phase 08
SEO/WCAG 30題
↓
Phase 09
Logic 60題
↓
Phase 10
AI × Front-End
↓
Phase 11
錯題訓練
↓
Phase 12
5回模擬考
↓

Phase 13

 7天最後衝刺

↓

 正式甄試

最後提醒

考前最後 7 天，最忌諱「焦慮式學習」。

不要因為看到一個陌生名詞，就開始搜尋一大堆新教材。

你的目標已經不是把所有 Web 技術學完，而是：

把已經學過的內容，轉換成穩定的得分能力。

尤其最後一天，寧可把 **錯題、20 大陷阱、HTTP Status Code、JavaScript 基礎、HTML Semantic、CSS Box Model、XSS/SQL Injection/CSRF、SEO/WCAG、AI 驗證原則** 再看一遍，也不要突然開始研究新的 JavaScript Framework。

你現在要做的不是「學更多」，而是「把會的題目全部拿到分」。

2026/8/18 下午2:19:18



Phase 14 | 考前終極速讀講義

台灣就業通－前端網頁技術與 AI 應用甄試

Final Exam Ultimate Review | 考前最後一輪

使用方式：

這不是拿來「學新東西」的教材，而是考前最後 1～3 次快速掃讀用的。

建議考前：

- 7 天前：完整讀一次
- 3 天前：快速讀一次
- 前一天：只讀標記 ★ 的內容
- 進考場前：只看最後的「100 個必背觀念」

目錄

1. HTML 終極速讀
 2. CSS / RWD 終極速讀
 3. JavaScript 終極速讀
 4. Internet / HTTP 終極速讀
 5. 電腦與作業系統
 6. 資訊安全
 7. SEO / WCAG / Web
 8. 數理邏輯
 9. AI × Front-End
 - L0. 100 個必背觀念
 - L1. 50 個高頻陷阱
 - L2. 考場解題法
 - L3. 最後 10 分鐘速讀版
-

PART 1 | HTML 終極速讀

1. HTML 是什麼？

HTML = HyperText Markup Language

HTML 主要負責：

網頁的結構與語意

不是主要負責：

- 視覺設計 → CSS
- 程式邏輯 → JavaScript

2. HTML 基本結構

```
<!DOCTYPE html>
<html lang="zh-Hant">

<head>
  <meta charset="UTF-8">
  <meta name="viewport"
    content="width=device-width, initial-scale=1.0">

  <title>網頁標題</title>
</head>

<body>

  網頁內容

</body>
</html>
```

★ 必背

```
<!DOCTYPE html>
```

表示使用 HTML5 文件模式。

3. Semantic HTML

元素	意義
<code><header></code>	頁首/區域標頭
<code><nav></code>	導覽
<code><main></code>	主要內容
<code><section></code>	主題性區塊
<code><article></code>	獨立內容
<code><aside></code>	側邊/補充內容
<code><footer></code>	頁尾

★ 最常考

`section` 有語意，`div` 通常只是一般容器。

4. h1~h6

表示：

標題層級

不是單純用來控制：

字體大小。

合理結構：

```
H1
├── H2
│   ├── H3
│   └── H3
└── H2
```

5. `<p>`、`<div>`、`<span>`

`<p>`

段落。

`<div>`

一般區塊容器。

`<span>`

一般行內容器。

6. 超連結

```
<a href="https://example.com">
  網站
</a>
```

必背

`href`：

指定連結目的地。

7. 圖片

```

```

★ `alt`

圖片無法顯示或使用輔助技術時提供替代文字。

8. alt vs title

屬性	主要用途
alt	圖片替代文字
title	額外提示資訊

★ 陷阱

***title* 不能取代 *alt*。**

9. figure

```
<figure>
  
  <figcaption>產品外觀</figcaption>
</figure>
```

10. List

無序

```
<ul>
  <li>HTML</li>
  <li>CSS</li>
</ul>
```

有序

```
<ol>
  <li>第一步</li>
  <li>第二步</li>
</ol>
```

11. Table

基本：

```
<table>
  <tr>
    <th>姓名</th>
    <th>年齡</th>
  </tr>
  <tr>
    <td>王小明</td>
    <td>30</td>
  </tr>
</table>
```

元素	意義
table	表格
tr	row
th	header cell
td	data cell

12. Form

```
<form>
  <label for="name">姓名</label>
  <input id="name" name="name" required>
</form>
```

★ 必背

label → 表單欄位標籤

required → 必填

placeholder → 輸入提示

13. GET vs POST

GET

常用於：

取得資料。

資料可能出現在 URL。

POST

常用於：

提交資料。

★ 陷阱

POST 不代表自動加密。

真正的傳輸加密通常依靠：

HTTPS / TLS

14. HTML 最重要 15 個陷阱

```
id ≠ class
alt ≠ title
strong ≠ b
em ≠ i
section ≠ div
article ≠ section
main ≠ body
label ≠ placeholder
required ≠ placeholder
GET ≠ POST
HTML ≠ CSS
HTML ≠ JavaScript
h1 ≠ 最大字體
nav ≠ footer
alt ≠ SEO 關鍵字堆砌
```

PART 2 | CSS / RWD

15. CSS 是什麼？

CSS：

Cascading Style Sheets

主要負責：

網頁外觀、版面與呈現。

16. CSS 基本語法

```
p {  
  color: red;  
  font-size: 16px;  
}
```

結構：

```
selector  
  ↓  
property: value;
```

17. Selector

```
p {}  
.box {}  
#header {}
```

Selector

意義

p

元素

.box

class

#header

ID

18. Specificity

一般概念：

```
Inline
  ↓
ID
  ↓
Class / Attribute / Pseudo-class
  ↓
Element
  ↓
Universal
```



ID 通常比 class specificity 高。

19. Cascade

CSS 最後使用哪個規則，會受到：

- Cascade
- Specificity
- Source Order
- `!important`

等因素影響。

20. Box Model

```
Content
  ↓
Padding
  ↓
Border
```


↓
Margin



Padding 在內，**Margin** 在外。

21. box-sizing

```
box-sizing: border-box;
```

通常代表：

width / height 的計算會包含 padding 與 border。

22. Display

常見：

```
block  
inline  
inline-block  
flex  
grid  
none
```

23. Position

position

重點

static

預設

relative

可作為定位參考

absolute

脫離正常文件流

fixed

相對 viewport 固定

position

重點

sticky

滾動時依條件黏住

24. z-index

控制：

元素疊放順序。

25. Flexbox

```
.container {  
  display: flex;  
}
```



Flex：

一維 *Layout*

26. Flex 必背

```
justify-content  
align-items  
flex-direction  
flex-wrap  
gap
```

口訣

Justify 主軸，*Align* 交叉軸。

27. Grid

CSS Grid :

二維 *Layout*

可同時控制：

- columns
- rows

28. CSS Units

單位	意義
px	像素
%	相對值
em	相對目前上下文字體
rem	相對 root
vw	viewport width
vh	viewport height

29. RWD

Responsive Web Design :

讓網站適應不同螢幕與裝置。

30. Media Query

```
@media (max-width: 768px) {  
  .menu {  
    display: none;  
  }  
}
```

31. Mobile First

思維：

```
Mobile
  ↓
Tablet
  ↓
Desktop
```

不是：

「網站只能手機。」

32. CSS 十大陷阱

```
margin ≠ padding
Flex ≠ Grid
justify ≠ align
px ≠ %
em ≠ rem
vw ≠ vh
absolute ≠ fixed
display:none ≠ opacity:0
visibility:hidden ≠ display:none
width ≠ max-width
```

PART 3 | JavaScript

33. JavaScript 是什麼？

主要負責：

網頁互動與程式邏輯。

34. 三大 Web 技術

HTML



Structure

CSS



Presentation

JavaScript



Behavior

★ 超高頻

HTML 結構、CSS 外觀、JS 行為。

35. 變數

```
let name = "Tom";  
const age = 20;
```

let

可以重新賦值。

const

不能重新指定變數本身。

36. Data Types

必背：

String
Number
Boolean
Object
Array

```
null
undefined
```

37. `==` vs `===`

```
10 == "10"
```

通常：

true

因為可能進行型別轉換。

```
10 === "10"
```

:

false

因為型別不同。



優先理解 `===` 是嚴格相等。

38. Array

```
const fruits = ["apple", "banana"];
```

常見：

```
push()
pop()
shift()
unshift()
map()
filter()
forEach()
```

39. Object

```
const user = {  
  name: "Tom",  
  age: 30  
};
```

取得：

```
user.name
```

40. Function

```
function add(a, b) {  
  return a + b;  
}
```

Arrow Function：

```
const add = (a, b) => a + b;
```

41. DOM

DOM：

Document Object Model

常用：

```
document.getElementById()  
document.querySelector()  
document.querySelectorAll()
```

42. Event

```
button.addEventListener("click", function () {  
  console.log("Hello");  
});
```

```
});
```

常見事件

```
click  
submit  
input  
change  
mouseover  
keydown
```

43. innerHTML vs textContent

innerHTML

可以處理 HTML。

textContent

處理文字內容。

★ Security

不可信任輸入直接放進：

```
innerHTML
```

可能產生：

XSS

44. JSON

```
JSON.parse()  
JSON.stringify()
```


parse

JSON → JS

stringify

JS → JSON

45. localStorage

瀏覽器端儲存資料。



重新開啟瀏覽器後通常仍可保留。

PART 4 | Internet / HTTP

46. Internet vs WWW

Internet

全球網路基礎架構。

WWW

建立在 Internet 上的 Web 系統。

47. Client / Server

```
Client
  ↓ Request
Server
  ↓ Response
Client
```

Browser 通常是：

Client

Web Server :

Server

48. HTTP

HTTP :

HyperText Transfer Protocol

49. HTTP Methods

Method	常見用途
GET	取得
POST	建立/提交
PUT	更新
DELETE	刪除

50. HTTP Status Code

2xx

成功。

3xx

重新導向。

4xx

Client Request 問題。

5xx

Server 問題。

★ 必背

200 OK
301 Permanent Redirect
302 Temporary Redirect
400 Bad Request
401 Unauthorized
403 Forbidden
404 Not Found
500 Internal Server Error

51. URL

`https://www.example.com/products/index.html`

Scheme

`https`

Host / Domain

`www.example.com`

Path

`/products/index.html`

52. DNS

DNS :

Domain Name System

核心：

Domain Name → IP

53. IP

IPv4

32 bits

例如：

```
192.168.1.1
```

IPv6

128 bits

54. Port

常見：

```
HTTP    80
HTTPS   443
```

55. HTTPS

HTTPS：

HTTP + TLS 安全傳輸。



HTTPS 可以保護傳輸中的資料。

但：

HTTPS ≠ 網站絕對安全。

56. Cookie / Session / Cache / CDN

Cookie

瀏覽器端狀態資料。

Session

伺服器端或應用程式的狀態管理概念。

Cache

暫存資料。

CDN

分散內容傳遞。

57. 網址到網頁

這題非常可能出現。

輸入 URL



DNS



取得 IP



建立連線



TLS



HTTP Request



Web Server



HTTP Response



Browser



HTML/CSS/JS/Image



Render

PART 5 | Computer / OS

58. CPU

中央處理器

主要：

執行指令與運算。

59. RAM

隨機存取記憶體

主要：

暫時儲存執行中的資料。

屬於：

揮發性記憶體。

60. ROM

傳統概念：

非揮發性記憶體。

61. HDD vs SSD

HDD

機械式磁碟。

SSD

Flash Storage。

通常：

SSD 更快、更安靜、抗震性較佳。

62. GPU

主要：

圖形處理與大量平行運算。

63. Motherboard

主機板：

連接與整合各硬體元件。

64. OS

Operating System：

管理硬體與軟體資源。

例如：

- Windows
 - macOS
 - Linux
-

65. Process

Process：

正在執行的程式。

66. Driver

Driver :

協助 OS 與硬體溝通。

67. 儲存單位

1 Byte = 8 bits

常見：

KB

MB

GB

TB

68. 32-bit vs 64-bit

主要與：

處理器架構、位址空間等相關。

不要誤認成：

螢幕解析度。

PART 6 | 資訊安全

69. XSS

Cross-Site Scripting。

攻擊者：

將惡意 Script 注入網頁。

70. XSS 情境


```
<script>
  alert("Hacked");
</script>
```

如果網站不安全地把使用者輸入輸出：

XSS 風險。

71. SQL Injection

攻擊：

操控 SQL 查詢。

72. SQL Injection 防禦

重要方法：

Parameterized Query / Prepared Statement

73. CSRF

Cross-Site Request Forgery。

核心：

利用使用者已登入的狀態發送未預期請求。

74. CSRF 防禦

常見：

CSRF Token

75. DDoS

Distributed Denial of Service。

目的：

大量流量造成服務無法正常提供。

76. Phishing

釣魚：

偽裝可信任對象騙取資訊。

77. Social Engineering

利用：

人的心理弱點

而非單純技術漏洞。

78. Malware

Malicious Software：

惡意軟體。

79. Ransomware

勒索軟體：

加密或鎖定資料/系統並要求贖金。

80. Brute Force

大量嘗試帳號密碼組合。

81. MFA

Multi-Factor Authentication :

多因素驗證。

例如：

密碼

+

手機驗證碼

82. Cookie Security

HttpOnly

降低 JavaScript 讀取 Cookie 的能力。

Secure

Cookie 通常只透過 HTTPS 傳送。

SameSite

控制跨站 Cookie 行為，可降低部分 CSRF 風險。

★ 資安終極對照

攻擊	關鍵字
XSS	Script
SQL Injection	Database
CSRF	已登入狀態
Phishing	欺騙
DDoS	大量流量
Brute Force	暴力嘗試

攻擊	關鍵字
Malware	惡意程式
Ransomware	加密勒索

PART 7 | SEO / WCAG / Web

83. SEO

Search Engine Optimization :

搜尋引擎最佳化。

84. SEO 基礎

重要元素：

- <title>
- Meta Description
- Heading
- Semantic HTML
- URL
- Internal Links
- Image alt
- Canonical
- Sitemap
- Structured Data

85. Canonical

告訴搜尋引擎：

哪個 URL 是主要/標準版本。

86. robots.txt

告訴搜尋引擎爬蟲：

哪些路徑的爬取規則。



robots.txt：

不是網站安全機制。

87. Sitemap

提供：

網站 URL 資訊。

88. Structured Data

結構化資料：

協助搜尋引擎理解頁面內容。

常見：

Schema.org

89. WCAG

Web Content Accessibility Guidelines。

核心：

Web 無障礙指南。

90. Accessibility

重要：

```
alt
label
```

keyboard
focus
contrast
semantic HTML
ARIA

91. ARIA

Accessible Rich Internet Applications ◦

用途：

提供輔助技術額外語意資訊。



ARIA 不是用來取代良好的 *Semantic HTML*。

92. Core Web Vitals

目前重點：

LCP
INP
CLS

LCP

最大內容元素載入。

INP

互動回應性。

CLS

版面穩定性。

PART 8 | 數理邏輯

93. 數列

看到：

2 4 6 8 10

先看：

加法。

看到：

2 4 8 16 32

先看：

乘法。

看到：

2 6 12 20 30

差：

4
6
8
10

下一個：

12

所以：

42

94. 比例

例如：

3 人 $\times$ 6 天 = 18 人天。

6 人：

$$18 \div 6 = 3 \text{ 天。}$$

95. 百分比

原價 1000：

打 8 折：

$$1000 \times 0.8 = 800$$

再打 9 折：

$$800 \times 0.9 = 720$$



折扣要：

連乘

不是直接把 8 折 + 9 折算成 17 折。

96. 邏輯推理

如果：

所有 A 都是 B。

而：

小王是 A。

則：

小王是 B 。

97. 常見錯誤

不要因為：

所有 A 都是 B

就推論：

所有 B 都是 A 。

這叫做：

逆命題錯誤。

PART 9 | AI × Front-End

98. AI 的四層概念

AI

人工智慧。

Generative AI

生成內容的 AI。

LLM

Large Language Model。

AI Agent

能根據目標進行多步驟任務的 AI 系統。

99. AI 在前端工作

可以協助：

- HTML
- CSS
- JavaScript
- Debug
- SEO
- Content
- Accessibility
- Documentation
- Testing
- Code Review

100. AI 最重要原則

★ AI 輸出 ≠ 正確答案

正確流程：

```
Prompt
↓
AI Generate
↓
Understand
↓
Review
↓
Test
↓
Security Check
↓
Performance Check
↓
Accessibility Check
```



Deploy

100 個必背觀念

HTML | 1~15

1. HTML 負責結構與語意
2. CSS 負責外觀
3. JS 負責行為
4. `<!DOCTYPE html>` 表示 HTML5 文件模式
5. `header` 頁首
6. `nav` 導覽
7. `main` 主要內容
8. `section` 主題區塊
9. `article` 獨立內容
10. `aside` 補充內容
11. `footer` 頁尾
12. `alt` 圖片替代文字
13. `label` 表單標籤
14. `required` 必填
15. `id` 通常唯一、`class` 可重複

CSS | 16~30

16. Box Model = content + padding + border + margin
17. margin 在外
18. padding 在內
19. border 位於 padding 外
20. `box-sizing: border-box` 很重要
21. Flex 是一維

- 22. Grid 是二維
 - 23. justify-content → 主軸
 - 24. align-items → 交叉軸
 - 25. `position: absolute` 可脫離正常文件流
 - 26. fixed 通常相對 viewport
 - 27. z-index 控制疊放
 - 28. rem 相對 root
 - 29. vw 是 viewport width
 - 30. Media Query 用於 RWD
-

JavaScript | 31~45

- 31. let 可以重新賦值
 - 32. const 不能重新指定變數
 - 33. var 與 let scope 行為不同
 - 34. String 是字串
 - 35. Number 是數字
 - 36. Boolean 是布林
 - 37. Array 是陣列
 - 38. Object 是物件
 - 39. `==` 可能型別轉換
 - 40. `===` 嚴格相等
 - 41. DOM 是文件物件模型
 - 42. querySelector 可選取 DOM
 - 43. addEventListener 監聽事件
 - 44. innerHTML 要注意 XSS
 - 45. textContent 適合純文字
-

Internet | 46~60

- 46. Internet 是網路基礎架構
- 47. WWW 是 Web 系統
- 48. Browser 是 Client
- 49. Server 提供服務
- 50. DNS 解析網域
- 51. IPv4 = 32 bits
- 52. IPv6 = 128 bits
- 53. HTTP = Web 通訊協定
- 54. HTTPS 使用 TLS 保護傳輸
- 55. GET 常用於取得
- 56. POST 常用於提交
- 57. 200 = OK
- 58. 301 = 永久轉址
- 59. 404 = Not Found
- 50. 500 = Server Error

Computer | 61~70

- 51. CPU = 運算
- 52. RAM = 暫時記憶體
- 53. SSD = 儲存
- 54. GPU = 圖形/平行運算
- 55. OS = 作業系統
- 56. Driver = OS 與硬體溝通
- 57. Process = 執行中的程式
- 58. Browser = Web 應用程式
- 59. 1 Byte = 8 bits
- 70. 64-bit $\neq$ 螢幕解析度

Security | 71~82

- 71. XSS = Script Injection
 - 72. SQL Injection = Database Injection
 - 73. CSRF = 利用登入狀態
 - 74. DDoS = 大量流量
 - 75. Phishing = 釣魚
 - 76. Social Engineering = 心理操控
 - 77. Malware = 惡意軟體
 - 78. Ransomware = 勒索軟體
 - 79. Brute Force = 暴力嘗試
 - 30. MFA = 多因素驗證
 - 31. HttpOnly 限制 JS 讀取 Cookie
 - 32. Secure Cookie 通常透過 HTTPS 傳送
-

SEO / WCAG | 83~92

- 33. `<title>` 很重要
 - 34. Meta Description 是搜尋摘要的重要資訊來源之一
 - 35. Semantic HTML 有助於理解內容結構
 - 36. Canonical 指定標準 URL
 - 37. robots.txt 提供爬蟲規則
 - 38. Sitemap 提供 URL 資訊
 - 39. Schema.org 用於結構化資料
 - 30. WCAG 關注 Web 無障礙
 - 31. ARIA 提供輔助技術額外語意
 - 32. alt 是 Accessibility 重要元素
-

Performance / AI | 93~100

- 33. LCP = 最大內容元素載入

- 94. INP = 互動回應性
- 95. CLS = 版面位移
- 96. AI 不是永遠正確
- 97. AI 程式碼必須 Review
- 98. AI 程式碼必須 Test
- 99. AI 程式碼必須檢查 Security
- 100. **AI 是助手，不是免責的專業判斷者**

50 個「看到就要小心」的陷阱

```
01 id ≠ class
02 alt ≠ title
03 strong ≠ b
04 em ≠ i
05 section ≠ div
06 article ≠ section
07 main ≠ body
08 label ≠ placeholder
09 required ≠ placeholder
10 GET ≠ POST

11 margin ≠ padding
12 Flex ≠ Grid
13 justify ≠ align
14 em ≠ rem
15 vw ≠ vh
16 px ≠ %
17 absolute ≠ fixed
18 display:none ≠ opacity:0
19 hidden ≠ none
20 width ≠ max-width

21 = ≠ ==
22 == ≠ ===
23 null ≠ undefined
24 let ≠ var
25 const ≠ immutable object
```

26 innerHTML ≠ textContent
27 Array ≠ Object
28 JSON ≠ JavaScript Object
29 parse ≠ stringify
30 DOM ≠ HTML 原始碼

31 Internet ≠ WWW
32 Domain ≠ IP
33 DNS ≠ HTTP
34 HTTP ≠ HTTPS
35 401 ≠ 403
36 404 ≠ 500
37 301 ≠ 302
38 Cookie ≠ Session
39 Cache ≠ Backup
40 CDN ≠ Web Server 本身

41 XSS ≠ SQL Injection
42 SQL Injection ≠ CSRF
43 HTTPS ≠ 完全安全
44 robots.txt ≠ Security
45 sitemap ≠ robots.txt

46 LCP ≠ CLS
47 INP ≠ LCP
48 WCAG ≠ SEO
49 ARIA ≠ Semantic HTML
50 AI Generated ≠ Verified

考場最終解題策略

第一原則

不要在一道題目上耗太久。

如果 30~45 秒沒有思路：

先標記，跳過。

第二原則

找關鍵字

例如：

「圖片替代文字」

直接：

alt

「主軸」

直接：

justify-content

「找不到資源」

直接：

404

「惡意 JavaScript 注入」

直接：

XSS

第三原則

如果不知道答案：

先排除

例如：

DNS 是什麼？

看到：

- A. 網域解析
- B. CSS 排版
- C. CPU 運算
- D. 圖片壓縮

即使沒有完整背熟：

A 的機率最高。



最後的「一分鐘心法」

進考場前只要記住：

HTML → 結構

CSS → 外觀

JS → 行為

DOM → 文件模型

DNS → 網域解析

HTTP → 通訊

HTTPS → 加密傳輸

CPU → 運算

RAM → 暫存

SSD → 儲存

OS → 管理

XSS → Script

SQL Injection → Database

CSRF → 登入狀態

MFA → 多因素

SEO → 搜尋

WCAG → 無障礙

LCP → 載入

INP → 互動

CLS → 位移

AI → 協助

Human → 驗證



最終一句話

如果整套 **Phase 01~14** 最後只能帶進考場一個觀念，我會希望你記住：

「不要只背答案，要理解答案背後的原理。」

因為正式甄試最容易改變的就是：

題目換個說法。

但如果你懂：

HTML 的語意、CSS 的 Layout、JavaScript 的型別與 DOM、Internet 的 Request/Response、資安的攻擊原理、SEO/WCAG 的目的，以及 AI 輸出的驗證原則，

即使題目不是你做過的原題，你仍然有能力推導答案。

這才是這套「**前端網頁技術與 AI 應用甄試 | 500 題完整訓練課程**」最終真正要建立的能力。

2026/8/18 下午2:20:49